



Elmar Wings

AI with an Arduino Nano 33 BLE Sense

Realtime Object Detection with the ArduCAM

Version: 1765
May 13, 2025

Contents

Contents	3
List of Figures	7
List of Listings	9
1. Einleitung	13
1.1. Herausforderungen	13
1.2. Lösungen	13
1.3. Struktur des Dokuments	13
I. Arduino Nano 33 BLE Sense	15
2. Arduino IDE 2.3.6	17
2.1. Beschreibung der Arduino IDE	17
2.2. Installation der Arduino IDE	18
2.2.1. Download der Arduino IDE	18
2.2.2. Installation auf Windows	18
2.2.3. Installation (MacOS)	23
2.3. Übersicht	23
2.4. Funktionen	24
2.4.1. Sketchbook	24
2.4.2. Board-Verwaltung	24
2.4.3. Bibliotheksverwalter	25
2.5. Integrating and Using a Custom Library in Arduino IDE	26
2.5.1. Creating the Library Files	26
2.5.2. Installation and Integration of the Library	27
2.5.3. Using Symbolic Links for Library Integration	27
2.5.4. Windows Tool <code>mklink</code> to Create Symbolic Links	27
2.5.5. MacOS Command <code>ln -s</code> to Create Symbolic Links	28
2.5.6. Verifying Library Availability in the Arduino IDE	29
2.5.7. Serial Monitor	30
2.5.8. Serial Plotter	30
2.6. Examples	30
2.7. Debugging	31
2.8. Autocompletion	31
2.9. Conclusion	32
3. Setup for the Arduino Nano 33 BLE Sense	33
3.1. Introduction	33
3.2. Configuration for the Arduino Nano 33 BLE Sense	33
3.2.1. Installation of the driver	33
3.2.2. Connection and Configuration of the Arduino Board to the Computer	34
3.2.3. Select the Appropriate Port	34
3.2.4. Upload and Verify a Sketch	36

3.3. Test the Configuration	37
3.3.1. Testing the Steps by an Example Sketch blink.ino	37
3.3.2. Description of Basic Sketch for Printing 'Hello'	38
4. Arduino Nano 33 BLE Sense	41
4.1. Arduino Nano 33 BLE Sense	41
4.2. Unterschied zwischen dem Arduino Nano 33 BLE Sense Rev1, dem Arduino Nano 33 BLE Sense Rev2 und dem Arduino Nano 33 BLE Sense Lite	42
4.2.1. What is the difference between Rev1 and Rev2?	42
4.2.2. Changes in the Sketch	43
4.2.3. Arduino Nano 33 BLE Sense Lite	44
4.3. On-Board Sensor Description	44
4.4. Arduino Nano 33 BLE Pin Configuration	47
 II. Applikation: Magic Faucet Fountain	 49
5. Ultraschallsensor HC-SR04	51
5.1. Einleitung	51
5.2. Ultraschallsensoren zur Abstandsmessung	51
5.3. Grundlagen zur Verwendung eines Ultraschallsensors	52
5.3.1. Grundlagen Schall	52
5.3.2. Schallgeschwindigkeit und Wellenlänge	52
5.3.3. Weg- und Abstandsmessung mit Ultraschall	53
5.4. Ultraschall-Abstandsensor HC-SR04	54
5.5. Spezifikation	55
5.5.1. Anwendungen	55
5.6. Bibliothek	55
5.6.1. Beschreibung	55
5.6.2. Installation	55
5.6.3. Funktionen	56
5.7. Kalibrierung	56
5.8. Simple Code	56
5.9. Einfache Anwendung	56
5.10. Tests	60
5.10.1. Einfacher Funktionstest	60
5.10.2. Test aller Funktionen	61
5.10.3. Lösungsansätze	62
5.11. Weiterführende Anwendungen	62
5.12. Further Readings	63
 6. Comet Tauchpumpe ELEGANT 12V	 65
6.1. Einführung	65
6.1.1. Arbeitsweise Kreislumpumpen	65
6.2. Charakteristische Größen von Pumpen	66
6.2.1. Volumenstrom	66
6.2.2. Förderhöhe	66
6.2.3. Nutzleistung	66
6.2.4. Pumpenwirkungsgrad	67
6.2.5. Kupplungsleistung	67
6.2.6. Anlagenkennlinie	67
6.3. Comet Tauchpumpe ELEGANT 12V	67
6.4. Spezifikation	68
6.4.1. Technische Daten:	68

6.4.2. Weitere Eigenschaften	68
6.5. Einfacher Code	68
6.6. Tests	70
6.7. Einfache Anwendung	70
6.8. Further Readings	72
7. Schaltnetzteil: MW RD-50A	73
7.1. Einleitung	73
7.2. Grundlegende Informationen	73
7.3. Schaltnetzteil: MW RD-50A	74
7.4. Spezifikation	74
7.5. Tests	75
7.6. Einfache Anwendung	75
7.7. Further Readings	78
8. Bluetooth Modul HC-05	79
8.1. Einleitung	79
8.2. Grundlegende Informationen	79
8.2.1. Bluetooth Varianten	79
8.2.2. Reichweite und Sendeleistung	80
8.2.3. Daten Übertragung	80
8.2.4. Anwendungen	80
8.3. Bluetooth Modul: HC-05	81
8.4. Spezifikation	81
8.5. Bibliotheken	81
8.6. Einfacher Code	82
8.7. Tests	83
8.8. Einfache Anwendung	83
8.9. Further Readings	85
9. Kippschalter: GOOBAY 10013	87
9.1. Einleitung	87
9.2. Grundlegende Informationen	87
9.3. Schalter: GOOBAY 10013	87
9.4. Spezifikation	87
9.5. Tests	88
9.6. Einfache Anwendung	88
10. MOS-FET	91
10.1. Einleitung	91
10.2. Grundlegende Informationen	91
10.2.1. N-Kanal-MOS-FET	92
10.2.2. P-Kanal-MOS-FET	92
10.3. MOS-FET IRF9540N	92
10.4. Spezifikation	92
10.5. Einfacher Code	94
10.6. Tests	95
10.7. Einfache Anwendung	95
11. Externe LED	97
11.1. Einleitung	97
11.2. Grundlegende Informationen	97
11.3. LED CML Signalleuchte, blau, ultrahell, 12 VDC	98
11.4. Spezifikation	98
11.5. Einfacher Code	98

11.6. Tests	99
11.7. Einfache Anwendung	99
11.8. Further Readings	100
12. Materialliste Magic Faucet Fountain	101
13. Projekt: Magic Faucet Fountain	103
13.1. Beschreibung	103
13.2. Aufbau	103
13.2.1. Abmaß	104
13.3. Hardware-Schnittstellen	104
13.3.1. Schaltpläne	104
13.4. Software-Schnittstellen	106
13.4.1. Ablaufdiagramm der Anwendung	106
13.4.2. Code der Anwendung	108
13.5. Installation	109
13.5.1. Mechanischer Aufbau	109
13.5.2. Elektrische Installation	109
13.5.3. Software und App-Verbindung	110
13.5.4. Software und App-Verbindung	110
13.6. Konfiguration	110
13.7. Einschränkungen	111
14. Schlussfolgerungen	113
14.1. Zusammenfassung der Projektergebnisse	113
14.2. Reflexion über die Zielerreichung	113
14.3. Mögliche Weiterentwicklungen und zukünftige Anwendungen	113
III. Anhang	115
Literaturverzeichnis	117
Stichwortverzeichnis	120
Index	121

List of Figures

2.1. Menü-Leiste	17
2.2. Arduino IDE 2.3.6	18
2.3. Option für Windows	19
2.4. Lizenzabkommen	20
2.5. Auswahl der Benutzer	21
2.6. Auswahl des Installationsverzeichnis	22
2.7. Die Arduino IDE	22
2.8. Die neue Arduino IDE mit der nützlichen Sidebar	23
2.9. Das Sketchbook auf der Arduino IDE	24
2.10. Die Board-Verwaltung auf der Arduino IDE	25
2.11. Der Bibliotheksverwalter auf der Arduino IDE	26
2.12. Command Prompt with Administrator Privileges	28
2.13. Command Prompt	28
2.14. Including the Nano33BLESenseLED Library	29
2.15. Serial Monitor	30
2.16. Examples	31
2.17. Debugging Example	31
2.18. Autocomplete	32
2.19. Menu Bar Option	32
3.1. Arduino Mbed OS Nano Boards Installation	34
3.2. Select the Connected board - here Arduino Nano 33 BLE Sense	34
3.3. Arduino Nano 33 BLE Sense Reset Button	35
3.4. green and orange Builtin-LED	36
3.5. Select Available Port for Uploading Arduino Sketch	36
3.6. Upload the Program in Arduino board	37
3.7. Setting the Port	37
3.8. LED Example Sketch	38
3.9. LED-Built Example	38
4.1. Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store	42
4.2. Arduino Nano 33 BLE Sense Rev. 1	43
4.3. Arduino Nano 33 BLE Sense Lite	44
4.4.	45
4.5. Pin assignment of the Arduino Nano 33 BLE Sense	46
4.6. Connection with BLE and smartphone	46
5.1. Frequenz der verschiedenen Schall-Arten	52
5.2. Ausbreitungsgeschwindigkeit von Schall in Luft	53
5.3. Spannungsverlauf Schallwandler bei einer Echo-Laufzeit-Messung, nach [HS23]	54
5.4. Ultraschallsensor mit Pin-Belegung	54
7.1. Aufbau eines Schaltnetzteils	74
7.2. Skizze des Netzteils	75
7.3. Spannungsmessung des Netzteils	75
7.4. Anwendung des Netzteils für 2 separate Stromkreise	76

8.1. Skizze des Bluetooth Moduls HC-05	81
9.1. Skizze der Schaltertypen	87
9.2. Messen des Widerstands	88
9.3. Einfache Anwendung des Schalters	89
10.1. Skizze des MOS-FETs	92
10.2. Pinbelegung des MOS-FETs	94
11.1. Aufbau einer LED	98

List of Listings

5.1. Einfacher Code zum Testen des Ultraschallsensors	57
5.2. Einfache Anwendung als Schwellenwert-Melder	59
5.3. Code für die Messung einer Strecke	61
6.1. Einfacher Code zur Ansteuerung einer Pumpe	69
6.2. Einfacher Code zur Fehlererkennung	71
7.1. Einfacher Code zum Verwenden des Netzteils	77
8.1. Code zur Überprüfung der Funktionalität	83
8.2. Code zur Überprüfung der Funktionalität	84
10.1. Einfacher Code zum Testen des MOS-FETs	95
10.2. Einfache Anwendung zum Verwenden des MOS-FETs	96
11.1. Einfacher Code zur Ansteuerung einer externen LED	99
11.2. Einfacher Code zum Blinken einer externen LED	100

Acronyms

GND Ground

GPIO General Purpose Input Output

I²C Inter-Integrated Circuit

IDE Integrated Development Environment

IMU Inertial Measurement Unit

SCL Serial Clock

SDA Serial Data

UART Universal Asynchronous Receiver Transmitter

VCC Voltage at the Common Collector

1. Einleitung

1.1. Herausforderungen

1.2. Lösungen

1.3. Struktur des Dokuments

Part I.

Arduino Nano 33 BLE Sense

2. Arduino IDE 2.3.6

Dieses Kapitel bietet eine umfassende Untersuchung der Arduino IDE, einschließlich ihrer Funktionen, Schnittstellenoptionen und Funktionalität. Es enthält eine detaillierte Erklärung des Zwecks und der Nutzbarkeit jeder Komponente, eine schrittweise Installationsanleitung sowie Anweisungen zur Initialisierung und Konfiguration der Umgebung. Dieser grundlegende Überblick befähigt die Leser, die IDE effektiv für die Programmierung und Fehlerbehebung von Arduino-kompatibler Hardware zu nutzen.

2.1. Beschreibung der Arduino IDE

Die Arduino IDE ist eine Open-Source-Software, die speziell für das Bearbeiten, Kompilieren und Hochladen von Code auf Arduino-Module entwickelt wurde. Sie ist plattformübergreifend und mit Betriebssystemen wie Windows, Linux und macOS kompatibel. Auf der Java-Plattform aufbauend unterstützt die IDE verschiedene Arduino-Module und die Programmierung in C und C++.

Die Mikrocontroller auf den Arduino-Boards werden so programmiert, dass sie die in Form von Code bereitgestellten Informationen verarbeiten. Programme, die in der IDE geschrieben werden, werden als Skizzen (Sketches) bezeichnet. Diese Skizzen generieren eine Hex-Datei, die dann auf den Mikrocontroller übertragen und hochgeladen wird. Die IDE-Umgebung besteht aus zwei Hauptkomponenten: dem Editor und dem Compiler. Der Editor wird zum Schreiben von Code verwendet, während der Compiler den Code kompiliert und auf das Arduino-Modul hochlädt.[FAD18] In Version 2.3.3 der Arduino IDE spielt die Menüleiste, die sich am oberen Rand der Schnittstelle befindet, eine entscheidende Rolle, da sie Zugriff auf wichtige Befehle wie Hochladen, Verifizieren/Kompilieren und Speichern bietet. Zusätzlich enthält sie das Dateimenü zum Öffnen neuer oder bestehender Dateien sowie den Beispiele-Bereich, der vorgefertigte Skizzen für verschiedene Anwendungen wie Blink und Fade bereitstellt.

Die 6 Buttons am oberen Rand des Fensters sind wie folgt in 2.1:

Figure 2.1.: Menü-Leiste



- Das Häkchen-Symbol dient zum **Überprüfen** des geschriebenen Codes. Nach der Code-Eingabe prüft diese Funktion auf Fehler und bestätigt die korrekte Struktur. Dieser Schritt stellt sicher, dass der Code für die Hardware bereit ist.
- Das **Hochladen**-Symbol kompiliert den Code und überträgt ihn auf das angeschlossene Board, sodass dieser ausgeführt werden kann.
- Das **Debug**-Symbol startet eine Debugging-Session, mit der Sie den Code analysieren können – inklusive Breakpoints, Schritt-für-Schritt-Ausführung und Variableninspektion (auf kompatiblen Boards).

-  Das **Serieller Plotter**-Symbol öffnet ein Tool zur Visualisierung von Echtzeitdaten, die über die serielle Verbindung vom Board gesendet werden, und stellt sie als Graphen dar.
-  Das **Serieller Monitor**-Symbol öffnet ein Terminal zur Anzeige und zum Versand von Textdaten über die serielle Verbindung, ermöglicht also die Echtzeit-Kommunikation mit dem Board.

2.2. Installation der Arduino IDE

2.2.1. Download der Arduino IDE

Der Arduino Nano 33 BLE Sense nutzt die Arduino Software Integrated Development Environment (IDE) für die Programmierung, die am weitesten verbreitete IDE für alle Arduino-Boards und sowohl online als auch offline verwendbar ist.

Diese Open-Source-Arduino-IDE vereinfacht das Schreiben von Code und das Hochladen auf das Board. Für jedes Betriebssystem (OS), einschließlich macOS, Linux und Windows, sind verschiedene Versionen der Software verfügbar. Die Arduino-Community bietet zudem eine Online-Plattform zum Programmieren und Speichern von Sketches in der Cloud. Dieser Online-Arduino-Editor ist die neueste Version der IDE und beinhaltet alle Bibliotheken sowie Unterstützung für neue Arduino-Boards.

Um auf diese Softwarepakete zuzugreifen, besuchen Sie die folgende Website: [Arduino-Software](#), um auf dem neuesten Stand zu bleiben, da tägliche Updates über den genannten Link verfügbar sind.

Der Editor kann direkt von der Arduino-Software-Seite heruntergeladen werden. Die Download-Optionen sind in Abbildung 2.2 dargestellt.

Figure 2.2.: Arduino IDE 2.3.6

Downloads



Arduino IDE 2.3.6

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux AppImage 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

2.2.2. Installation auf Windows

Der Installationsprozess der Arduino IDE 2 auf einem Windows-Computer wird im folgenden Abschnitt Schritt für Schritt beschrieben. So installieren Sie die Arduino

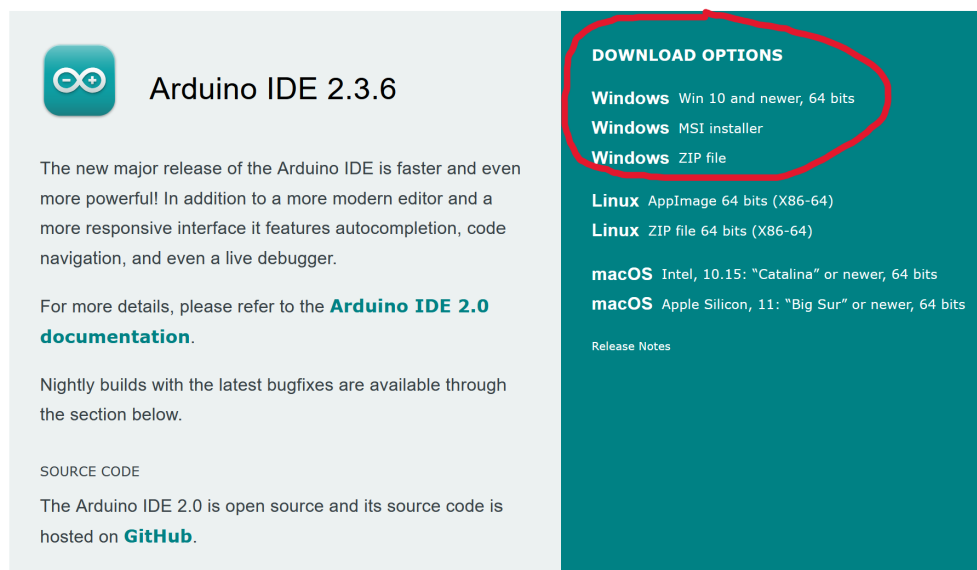
IDE 2 auf einem Windows-Computer:

Führen Sie die von der Software-Seite [Arduino-Software](#) heruntergeladene Datei aus.

Nutzen Sie die Installationsmöglichkeiten unter Windows, wie in Abbildung 2.3 dargestellt, um eine korrekte Installation sicherzustellen.

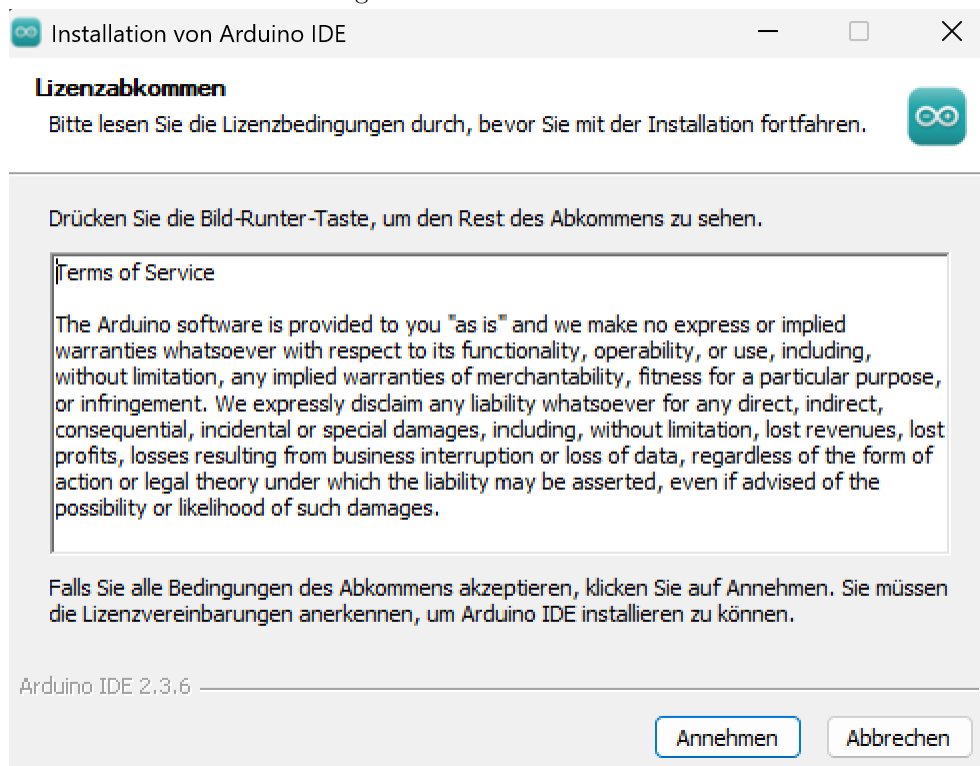
Figure 2.3.: Option für Windows

Downloads



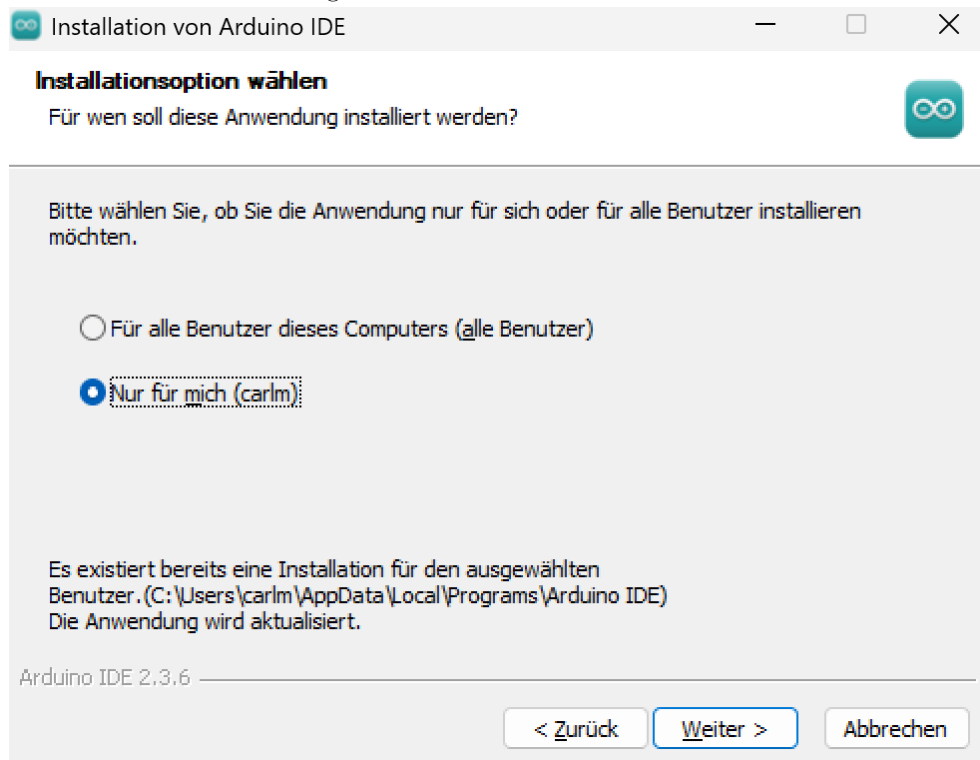
Nachdem der Download abgeschlossen ist, führen Sie die Setup-Datei aus. Akzeptieren Sie das Lizenzabkommen 2.4 und klicken sie auf [\[Annehmen\]](#):

Figure 2.4.: Lizenzabkommen



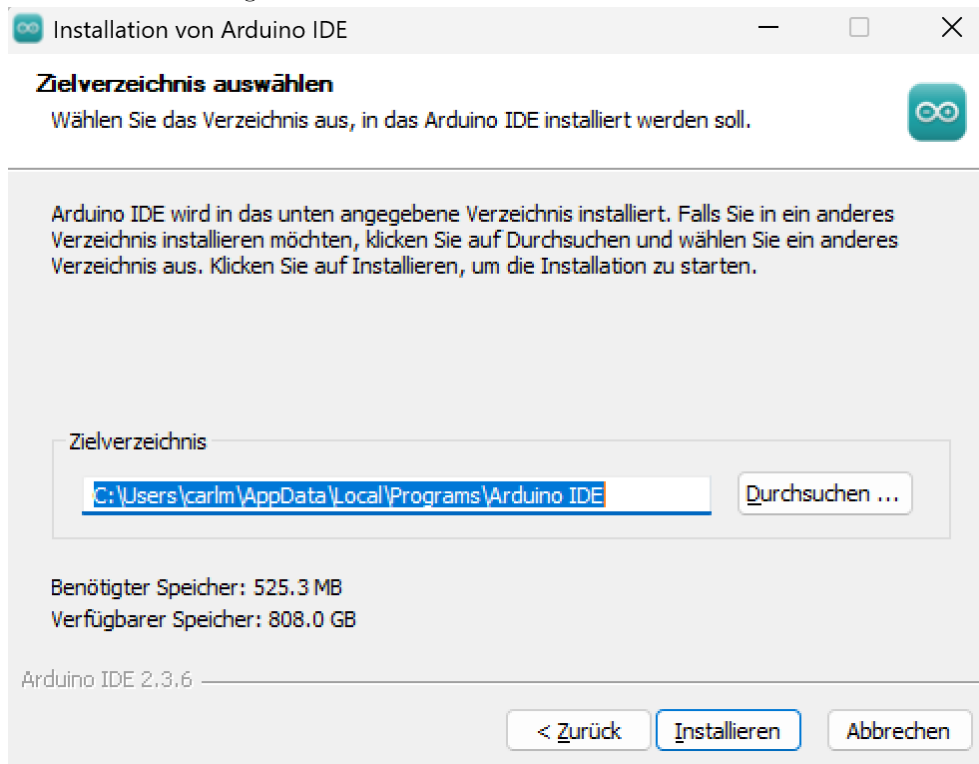
Danach wählen Sie aus, für wen die Installation auf Ihrem PC angelegt werden soll, Sie können auswählen, ob für Sie selbst oder alle Benutzer des Computers: 2.5

Figure 2.5.: Auswahl der Benutzer



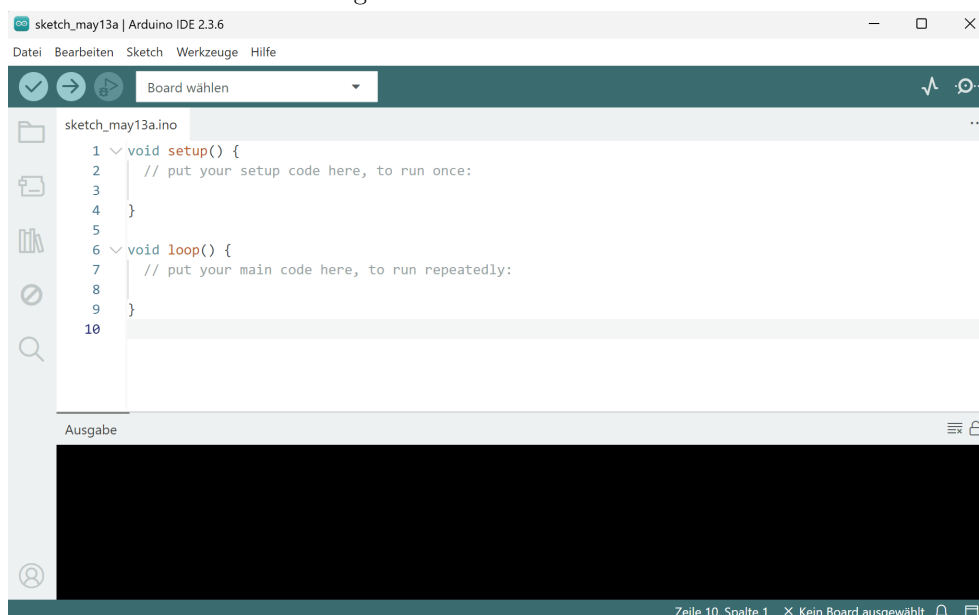
Nachdem Sie Ihre Wahl getroffen haben, klicken Sie auf **Weiter**. Nun können Sie das Verzeichnis auswählen, in das die Arduino IDE installiert werden soll. 2.6

Figure 2.6.: Auswahl des Installationsverzeichnis



Klicken Sie dann auf **Installieren** und warten Sie ab, bis der Download abgeschlossen ist. Nach Abschluss des Downloads öffnet sich die Arduino IDE automatisch. 2.7

Figure 2.7.: Die Arduino IDE



2.2.3. Installation (MacOS)

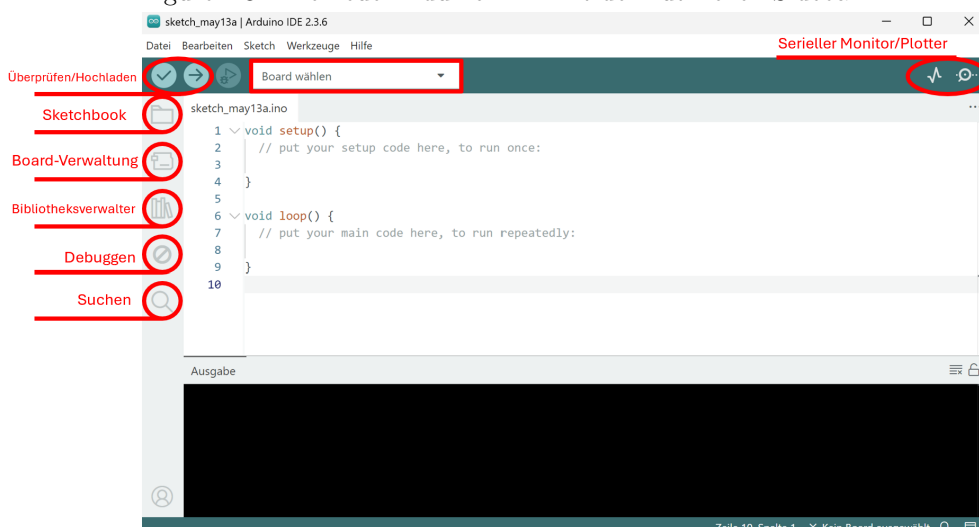
Aufgrund fehlender Hardware können wir leider keine Installation auf MacOS bereitstellen.

2.3. Übersicht

Die Arduino IDE 2 beinhaltet eine neue Sidebar 2.8, auf der die am häufigsten genutzten Werkzeuge leichter zu erreichen sind. Folgende Werkzeuge sind besonders hilfreich: [Ard25]

- **Überprüfen / Hochladen** Diese Funktionen dienen zum Kompilieren und Hochladen von Code auf ein Arduino-Board.
- **Board wählen** Erkannte Arduino-Boards werden hier automatisch angezeigt, inklusive der Port-Nummer.
- **Sketchbook** Dieser Bereich dient als zentrale Ablage für alle Sketches, die lokal auf dem Computer gespeichert sind. Das Sketchbook bietet eine strukturierte, leicht zugängliche Umgebung zur Verwaltung, Organisation und Bearbeitung von Arduino-Code. Zusätzlich ermöglicht es die Synchronisation mit der Arduino Cloud, wodurch Sketches geräteübergreifend verfügbar sind. Diese Cloud-Integration erlaubt den Zugriff und die Bearbeitung von Sketches aus der Online-Arduino-Umgebung.
- **Board-Verwaltung** Durchsuchen von Arduino- und Drittanbieter-Paketen zur Installation. Beispielsweise erfordert ein MKR WiFi 1010 Board das Arduino SAMD Boards-Paket.
- **Bibliotheksverwalter** Zugriff auf tausende Arduino-Bibliotheken, bereitgestellt von Arduino und der Community.
- **Debuggen** Echtzeit-Test und Fehlersuche in Programmen.
- **Suchen** Durchsucht den geschriebenen Code nach Schlüsselwörtern.
- **Serieller Monitor** Öffnet den seriellen Monitor als neuen Tab in der Konsole.
- **Serieller Plotter** Visualisiert serielle Daten als Echtzeit-Diagramme.

Figure 2.8.: Die neue Arduino IDE mit der nützlichen Sidebar



2.4. Funktionen

Die Arduino IDE 2 ist ein vielseitiges Entwicklungswerkzeug, das Funktionen wie die direkte Installation von Bibliotheken, Cloud-Synchronisation für Sketches und integrierte Debugging-Tools bietet. Dieser Abschnitt beleuchtet einige der Kernfunktionen, mit Verweisen auf detailliertere Ressourcen.

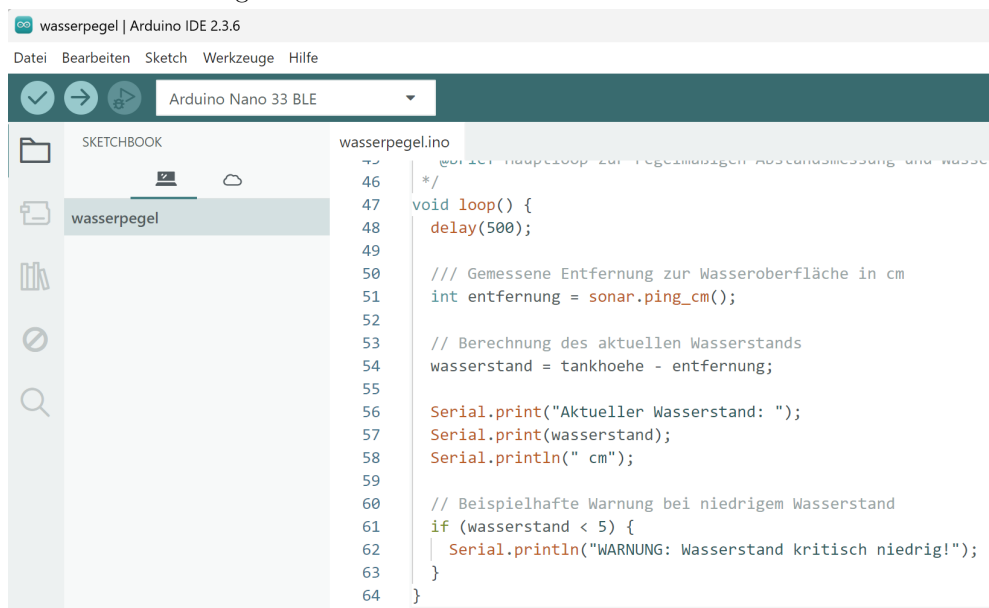
2.4.1. Sketchbook

Das **Sketchbook** ist der Speicherort für Arduino-Codedateien, die als Sketches bezeichnet werden. Diese Dateien werden mit der Endung `.ino` gespeichert. Zur ordnungsgemäßen Organisation muss jeder Sketch in einem Ordner abgelegt werden, der genau denselben Namen trägt wie die Sketch-Datei. Beispielsweise muss ein Sketch mit dem Namen `MySketch.ino` in einem Ordner namens `MySketch` gespeichert werden. Diese Namenskonvention ist entscheidend, damit die Arduino IDE die Sketches korrekt identifizieren und verwalten kann.

Standardmäßig werden Sketches in einem Ordner namens `Arduino` gespeichert, der sich im Dokumentenverzeichnis des Systems befindet.

Um auf das Sketchbook zuzugreifen, können Sie das Ordnersymbol in der Seitenleiste der Arduino IDE auswählen. Diese Aktion öffnet das Verzeichnis, in dem die Sketches gespeichert sind, und ermöglicht so eine effiziente Verwaltung und den Zugriff auf die Sketches. 2.9

Figure 2.9.: Das Sketchbook auf der Arduino IDE



2.4.2. Board-Verwaltung

Die Board-Verwaltung kann unter folgendem Pfad gefunden werden: **Werkzeuge** > **Board** > **Board-Verwaltung...**.

Der **Boards Manager** in der Arduino IDE ermöglicht die Installation verschiedener Board-Pakete, wie in Abbildung 2.10 dargestellt. Ein Board-Paket enthält die erforderlichen Dateien und Anleitungen, um Code für bestimmte Arduino-Board-Typen zu kompilieren und hochzuladen.

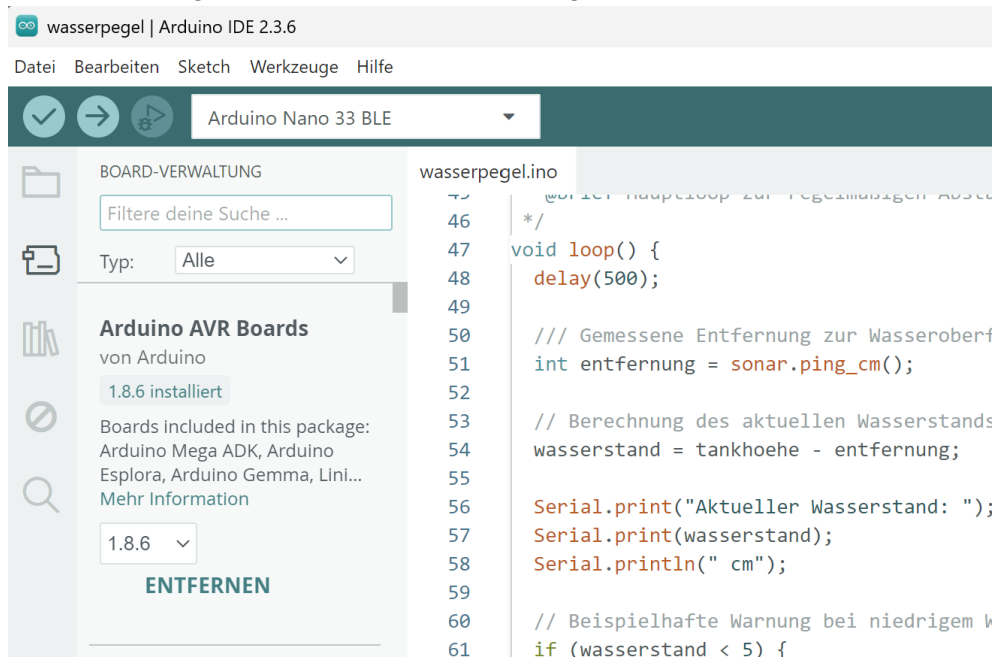
Es stehen verschiedene Board-Pakete zur Verfügung, darunter:

- **avr** - für ältere Boards wie das Arduino Uno

- **samd** - für neuere Boards wie das Arduino Zero
- **megaavr** - für die Mega-Serie

Jedes Paket unterstützt unterschiedliche Arduino-Board-Familien. Der Boards Manager stellt sicher, dass die richtigen Tools und Dateien für die Programmierung des ausgewählten Boards installiert sind.

Figure 2.10.: Die Board-Verwaltung auf der Arduino IDE



2.4.3. Bibliotheksverwalter

Den **Bibliotheksverwalter** erreichen Sie über folgenden Pfad:

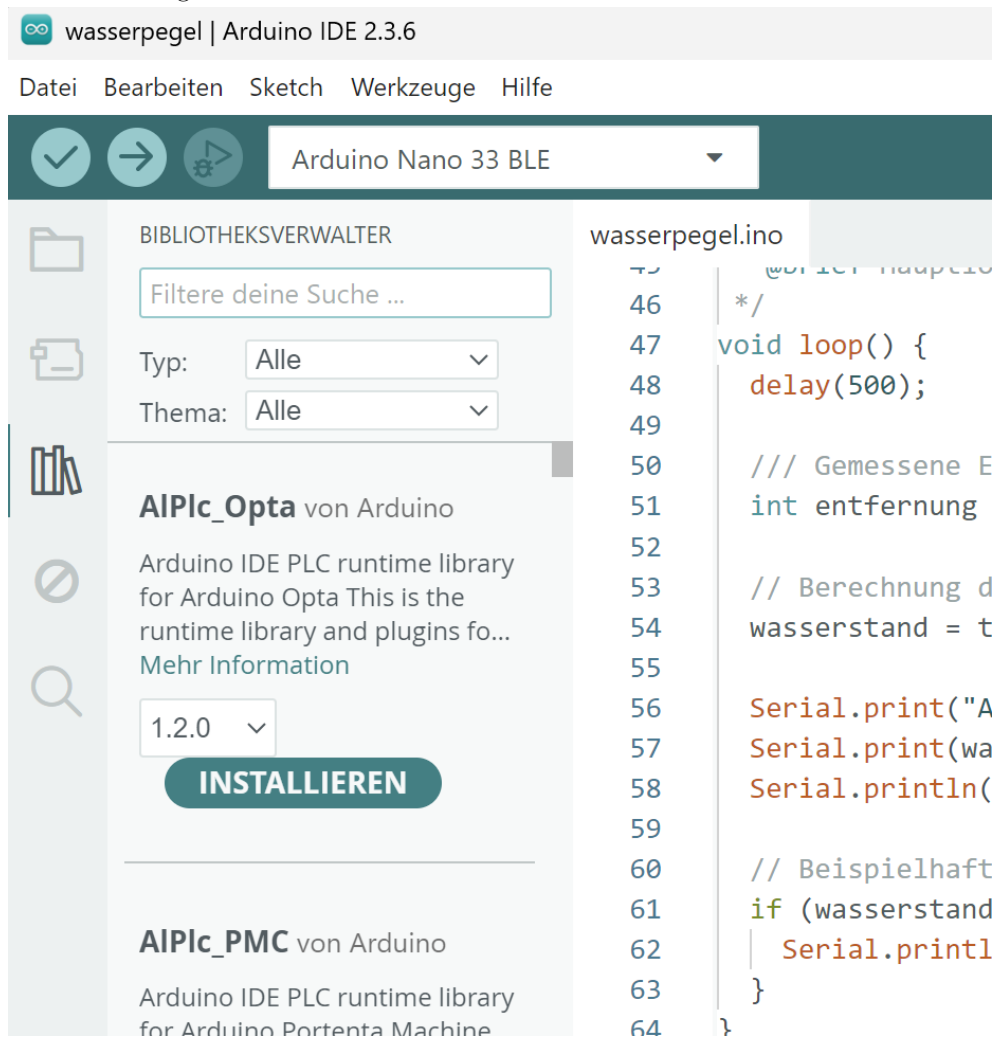
Werkzeuge > **Bibliotheken verwalten...**

Der Bibliotheksverwalter in der Arduino IDE ermöglicht das Durchsuchen und Installieren einer Vielzahl von Bibliotheken. Diese Bibliotheken erweitern die Grundfunktionen von Arduino und vereinfachen die Arbeit mit speziellen Hardwarekomponenten, wie zum Beispiel:

- Steuerung von Servomotoren
- Auslesen verschiedener Sensoren
- Kommunikation mit Modulen (z.B. Wi-Fi)

Die Bibliotheken enthalten vorgefertigten Code zur Ansteuerung spezifischer Hardware und vereinfachen komplexe Aufgabenstellungen. Dies ermöglicht eine schnellere und effizientere Entwicklung von Arduino-Projekten, wie in Abbildung 2.11 dargestellt.

Figure 2.11.: Der Bibliotheksverwalter auf der Arduino IDE



2.5. Integrating and Using a Custom Library in Arduino IDE

In the development of embedded systems using the Arduino platform, libraries are essential for abstracting and simplifying complex functionality. A custom library is particularly beneficial when a specific functionality or hardware interface is repeatedly used across different projects. This section outlines the process of creating, installing, and using a custom library within an Arduino project, in the Arduino Integrated Development Environment (IDE).

2.5.1. Creating the Library Files

A well-structured custom Arduino library typically consists of at least two key components:

Header File (.h): This file contains the declarations of classes, functions, and variables that define the interface of the library. It provides the necessary definitions for the functions and classes to be used by external programs.

Source File (.cpp): This file contains the actual implementation of the functions and

methods declared in the header file. It defines the behavior of the functions and classes. The library structure is organized in a way that allows for clear separation between the declaration and implementation of its functionalities.

2.5.2. Installation and Integration of the Library

Once the custom library has been created, it must be properly installed and integrated into the Arduino IDE to be used in projects.

To install a custom library in Arduino follow these steps:

1. Locate the Libraries Folder: The Arduino IDE searches for libraries in the libraries folder, which is located within the Arduino sketchbook directory. The typical path to this directory is:

```
C:/Users/<username>/Documents/Arduino/libraries
```

2. Copy the custom library folder, e.g., `Nano33BLESenseLE`, into the directory `libraries`. The final path should look like this:

```
C:/Users/<username>/Documents/Arduino/libraries/Nano33BLESenseLED
```

3. Restart the Arduino IDE: After placing the library in the correct directory, restart the Arduino IDE to ensure that it recognizes the newly added library.

2.5.3. Using Symbolic Links for Library Integration

In some cases, it may be more convenient or necessary to store the library files outside the default library directory. For example, if the libraries are stored in a GitHub repository or for better version control, the command `mklink` can be used to create a symbolic link. This allows the Arduino IDE to access the library files as if they were in the default directory, without duplicating the files.

2.5.4. Windows Tool `mklink` to Create Symbolic Links

Symbolic links allow libraries to be stored in a different location while still being treated by the Arduino IDE as if they reside in the standard directory `libraries`. This can be particularly useful for version-controlled environments, such as when using Git, or to organize libraries without duplicating files.

Steps for Creating a Symbolic Link:

1. **Open Command Prompt with Administrator Privileges:** Press `Win + S` and search for `cmd`, click on `Command Prompt` and select `Run as Administrator` as seen in Figure 2.12

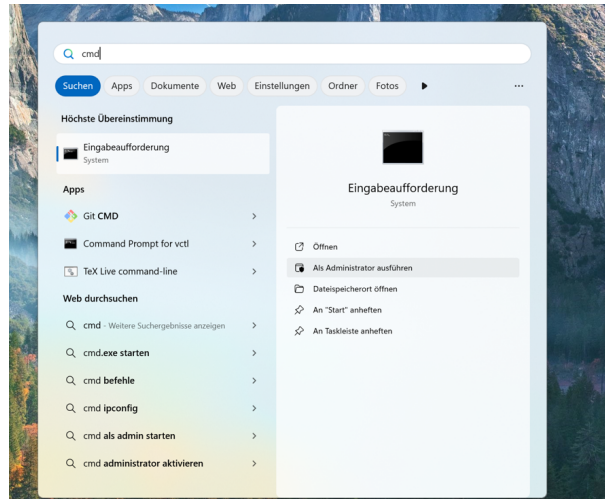


Figure 2.12.: Command Prompt with Administrator Privileges

S:Figure 1.17, 1.18 with
n english sytem, so the
figures are on english

2. **Execute the Command `mklink`** : The for creating a symbolic link is: `mklink /D "TargetPath" "SourcePath"`
 - **TargetPath:** The location where the Arduino IDE expects the library to be, here `libraries`.
 - **SourcePath:** The actual location of the library.

For this example, the library is located at:

`C:/Users/MSRLabor/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/`

The target path in the Arduino IDE's libraries folder is:

`C:/Users/MSRLabor/Documents/Arduino/libraries/Nano33BLESenseLED`

The full command can be seen in Figure 2.13.

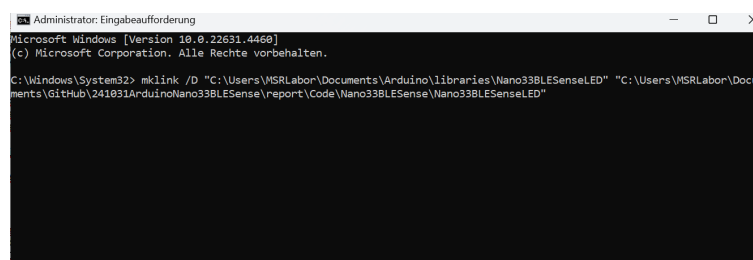


Figure 2.13.: Command Prompt

Verifying the Symbolic Link: After executing the command, navigate to the folder `libraries`. A folder named `Nano33BLESenseLED` should be present, acting as a symbolic link pointing to the actual library location.

2.5.5. MacOS Command `ln -s` to Create Symbolic Links

1. **Open Terminal:** Open the `Terminal` application by searching for it in `Applications`, go to `Utilities`.
2. **Execute the Command `ln -s`** : The command for creating a symbolic link is:
`ln -s "SourcePath" "TargetPath"`

- **TargetPath:** The location where the Arduino IDE expects the library to be, here `libraries`.
- **SourcePath:** The actual location of the library.

For this example, the library is located at:

`C:/Users/MSRLabor/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/`

The target path in the Arduino IDE's libraries folder is:

`C:/Users/MSRLabor/Documents/Arduino/libraries/Nano33BLESenseLED`

The full command would be:

```
ln -s "/Users/username/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/"
"/Users/username/Documents/Arduino/libraries/Nano33BLESenseLED"
```

Verifying the Symbolic Link: After executing the command, navigate to the folder `libraries`. A folder named `Nano33BLESenseLED` should be present, acting as a symbolic link pointing to the actual library location.

WS:here are some pictures necessary as in 1.5.4

2.5.6. Verifying Library Availability in the Arduino IDE

To ensure that the custom library has been successfully integrated and is recognized by the Arduino IDE, follow these steps:

1. Restart the Arduino IDE: If the Arduino IDE was open during the library installation or after creating the symbolic link, close and reopen it. This step ensures the IDE reloads its list of available libraries.
2. Check the Library Menu: Navigate to `Sketch` go to `Include Library` and select the Library `Nano33BLESenseLED`, as seen in Figure 2.14
3. Verify Inclusion in the List: If the library is listed, it indicates successful recognition by the IDE. If not, double-check the directory structure and symbolic link to confirm they are correctly configured.
4. Compile a Test Program: Write a simple test program using functions or classes from the library. Compile the sketch to ensure there are no errors, which confirms that the library has been successfully integrated.

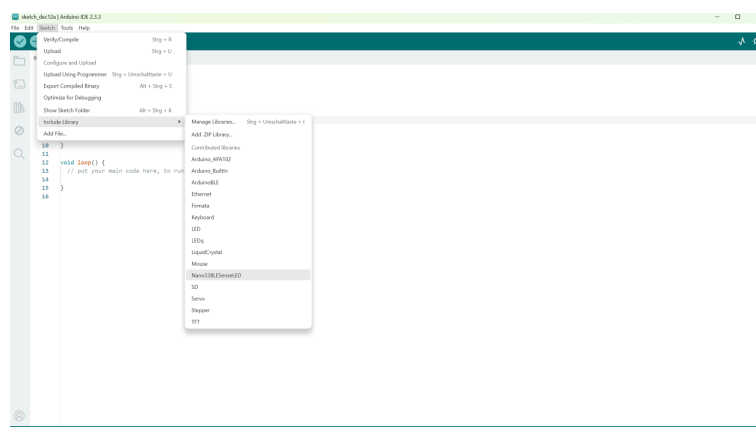


Figure 2.14.: Including the Nano33BLESenseLED Library

Once the program compiles without errors, the library is ready to use, and you can confidently include it in your Arduino projects.

2.5.7. Serial Monitor

The Serial Monitor can be found in the following steps: **Tool** **Serial Monitor**

The Serial Monitor is a tool that allows to view data streaming from the board, via for example the command `Serial.print()`.

Historically, this tool was located in a separate window but is now integrated with the editor, making it easy to run multiple instances simultaneously on your computer. The Serial Monitor can be seen in Figure 2.15.

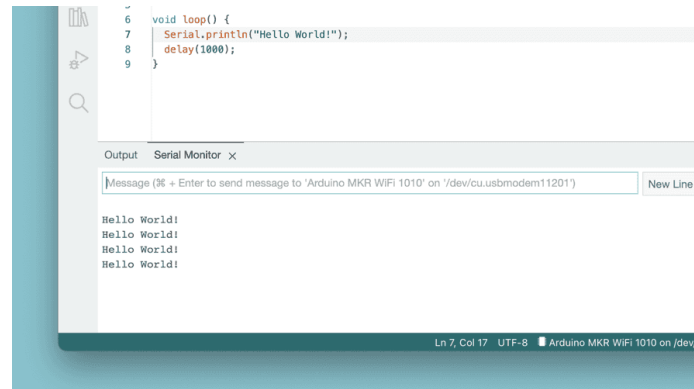


Figure 2.15.: Serial Monitor

2.5.8. Serial Plotter

The tool Serial Plotter is great for visualizing data with graphs and monitoring, such as voltage peaks. It can monitor multiple variables simultaneously, with the option to enable specific types.

2.6. Examples

A significant component of the Arduino Documentation is the set of example sketches bundled with various libraries. These examples demonstrate the practical application of library functions, showcasing their intended uses and main features. Libraries included with certain board packages may also contain their own example sketches. To access these example sketches whether from libraries installed manually or those included with board packages navigate to **File** **Examples**, and locate the desired library from the list.

For instance, when an UNO R4 WiFi board is connected, the examples list appears with options specific to that board. An example sketch demonstrating a pre-loaded Tetris animation can be accessed by navigating to **File** **Examples** **LED Matrix** **MatrixIntro** and uploading it to the connected board. This example shows how the board's bundled code can be used in practice, assisting users in learning how to control various hardware functions directly.

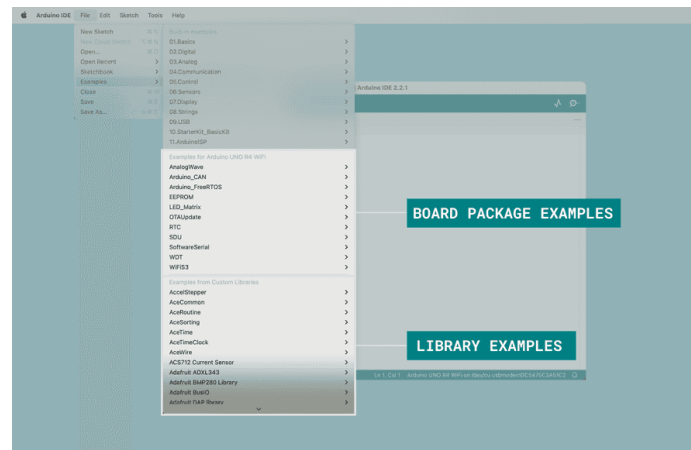


Figure 2.16.: Examples

In Figure 2.16 above, the examples list is shown when a UNO R4 WiFi board is connected to the computer.

2.7. Debugging

The tool Debugger in Arduino IDE 2.3.3 is designed to assist in testing and debugging programs by providing the ability to step through code execution in a controlled manner. This tool allows users to set breakpoints, step through code line-by-line, and inspect variables, helping to identify issues such as logic errors or incorrect variable values. The debugger enables users to pause execution at specific points in the program, allowing for a detailed examination of the program's behavior and memory state. This feature enhances the development process by making it easier to locate and fix errors during runtime, rather than relying solely on print statements or trial and error.

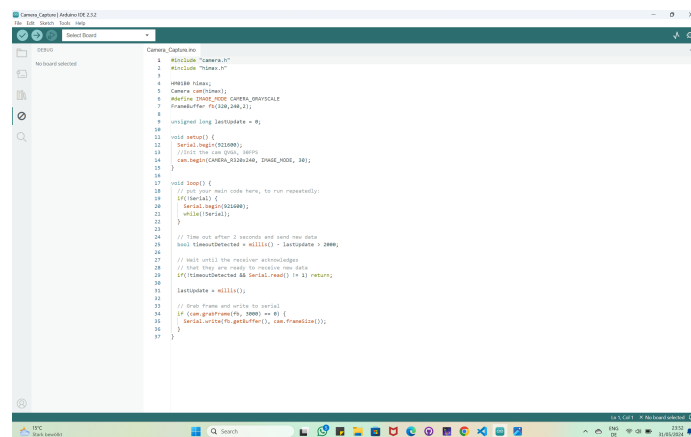


Figure 2.17.: Debugging Example

2.8. Autocompletion

Autocompletion is a key feature in Arduino IDE version 2, making it easier to code by suggesting functions and elements from the Arduino API. This feature helps identify and complete code more quickly, improving efficiency and accuracy.

WS:Debugging Bild mu
ersetzt werden, weil nic
zu lesen

For autocompletion to work, the board must be selected in the IDE. This ensures that the editor recognizes the correct functions and libraries for the specific board, allowing accurate suggestions. 2.18

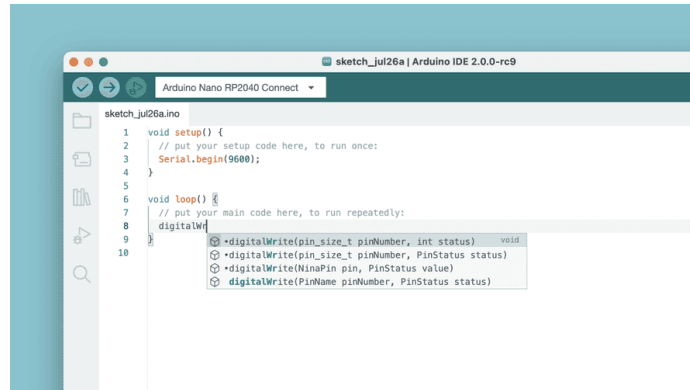


Figure 2.18.: Autocomplete

2.9. Conclusion

This guide provides an overview of key features in Arduino IDE 2, along with references to detailed articles for further exploration. Each section aims to enhance understanding and enjoyment of the wide range of functionalities included in the IDE.

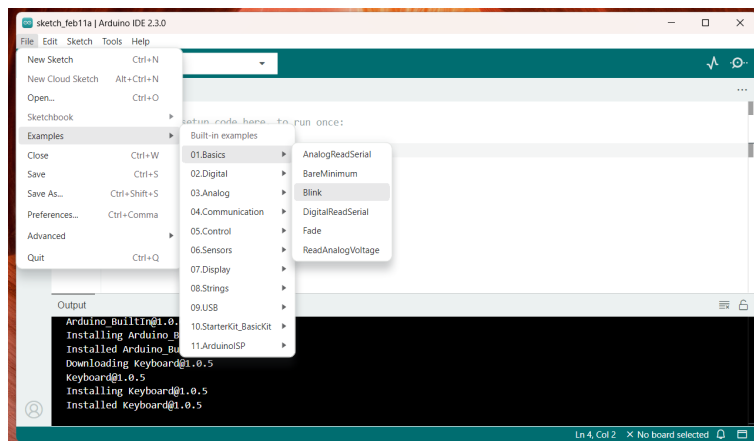


Figure 2.19.: Menu Bar Option

3. Setup for the Arduino Nano 33 BLE Sense

3.1. Introduction

In this chapter, the process of connecting the Arduino Nano 33 BLE Sense to a computer or laptop and configuring essential settings to get started is explored. The steps for installing the necessary tools and ensuring the board is correctly set up in the Arduino IDE are detailed. Finally, a simple example sketch is demonstrated to verify that the board is working as expected and ready for further development.

There are set of examples which are build in Arduino (IDE) for the testing purpose, for checking all the configuration and setting up the board we can open one of the basic example `blnik.ino` first.

3.2. Configuration for the Arduino Nano 33 BLE Sense

To program the **Arduino Nano 33 BLE Sense** in an offline environment, follow these steps:

3.2.1. Installation of the driver

1. **Installation of the driver:** Begin by installing the latest version of the Arduino IDE on your computer. Refer to Section ?? for more details.
2. **Installation of the packages:** Once the IDE installation is complete, open the `Arduino IDE` and navigate to the menu `Tools`, located in the upper-left corner.
3. In the menu `Tools`, select the `Board Manager`.
4. In the window `Board Manager`, use the search bar to locate the **Arduino Nano 33 BLE Sense** by typing its name as shown in Figure 3.1 .
5. From the search results, select `Arduino Mbed OS Nano Boards` and click `Install` to install the required package.

The Mbed OS Nano Board package includes support for several Arduino Nano family boards, including the Arduino Nano 33 BLE Sense. After completing the installation, connect the Arduino Nano 33 BLE Sense to your computer using a USB-A to USB-Micro to begin programming.

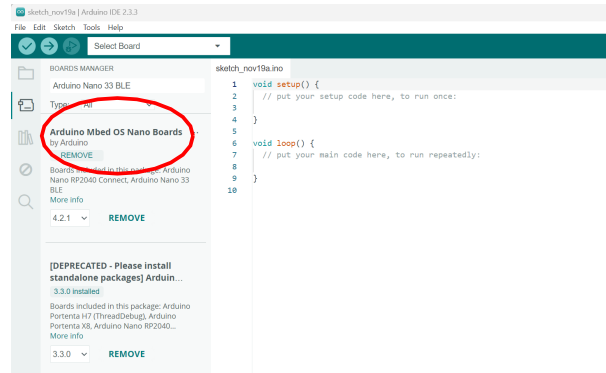


Figure 3.1.: Arduino Mbed OS Nano Boards Installation

3.2.2. Connection and Configuration of the Arduino Board to the Computer

To run either a built-in example or a custom program on the Arduino Nano 33 BLE Sense, follow these initial steps:

1. Connect the Arduino board to the computer using a USB-A-USB-micro-cable.
2. Open the Arduino IDE on the computer. A blank environment page will appear, showing the default `void setup()` and `void loop()` functions.
3. Navigate to the menu **Tools**, select **Board**, and choose the connected board, which is the **Arduino Nano 33 BLE Sense**, as shown in Figure 3.2.

This setup prepares the Arduino IDE to recognize and communicate with the Arduino Nano 33 BLE Sense.

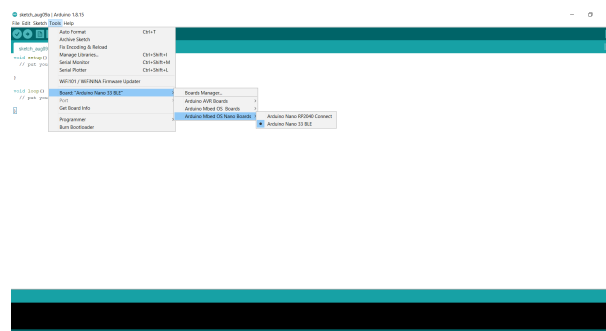


Figure 3.2.: Select the Connected board - here Arduino Nano 33 BLE Sense

3.2.3. Select the Appropriate Port

After selecting the Arduino Nano 33 BLE Sense board, the next step is to verify the connected port. To do this, the Arduino board must be set into Bootloader mode by pressing the white reset button on the board, as shown in Figure 3.4.

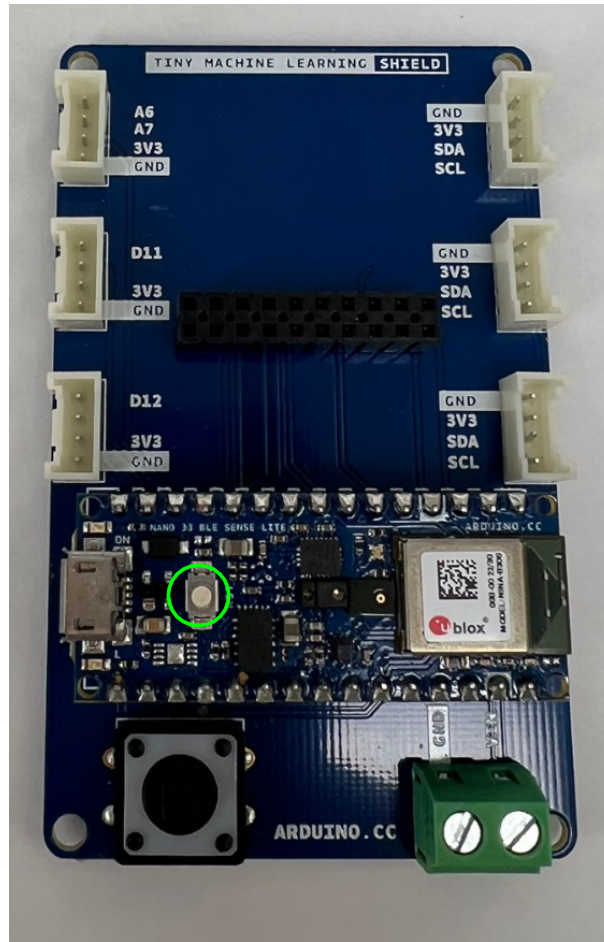


Figure 3.3.: Arduino Nano 33 BLE Sense Reset Button

By pressing the white reset button, the Arduino board will enter Bootloader mode. It is important to verify that the orange Built-in-LED is illuminated, as shown in Figure 3.4

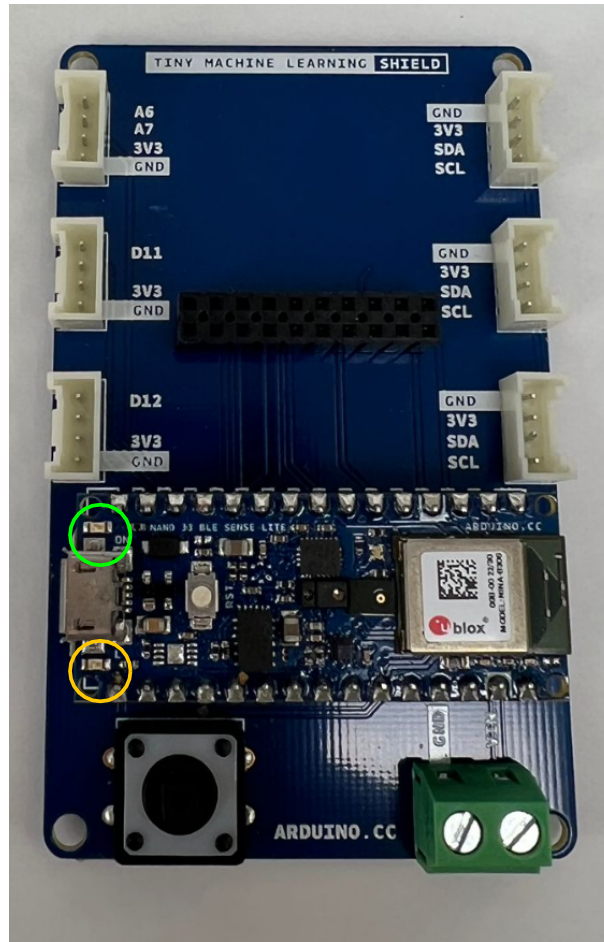


Figure 3.4.: green and orange Builtin-LED

After successfully completing the previously mentioned steps, the next task is to select the connected port before uploading the program. To do this, navigate to the menu **Tools**, select **Port**, and ensure that the available port for uploading the program is properly selected, as shown in Figure 3.5

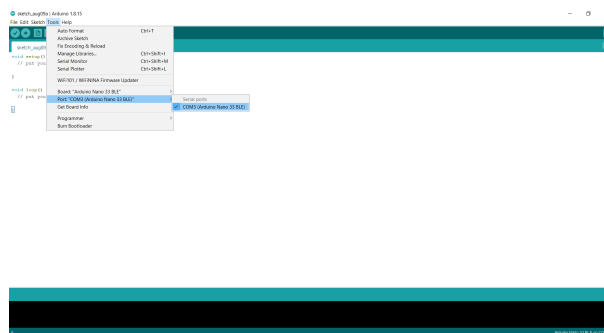


Figure 3.5.: Select Available Port for Uploading Arduino Sketch

3.2.4. Upload and Verify a Sketch

After ensuring that the appropriate port is selected, the next step is to upload the Arduino program. **Before** uploading the program, it is considered best practice to

verify it first. This process will indicate whether any errors or warnings exist in the program. Once the program is successfully verified, it can be safely uploaded by clicking the button **Upload** located below the file section, as shown in Figure 3.6. After selecting the board and port, confirm that the Arduino Nano 33 BLE Sense is properly connected to the Arduino IDE. This can be verified by checking the message in the bottom right corner of the IDE window, which should display the connected board and port.



Figure 3.6.: Upload the Program in Arduino board

After uploading, the code will be compiled, and any issues in the program will be displayed in the bottom black window. Once the code is successfully uploaded and compiled on the Arduino board, it is necessary to reselect the port, as done previously. Navigate to the **Tools menu**, select **Port**, and ensure the correct port is selected, as shown in Figure 3.7, in order to view the output in the Serial Monitor.

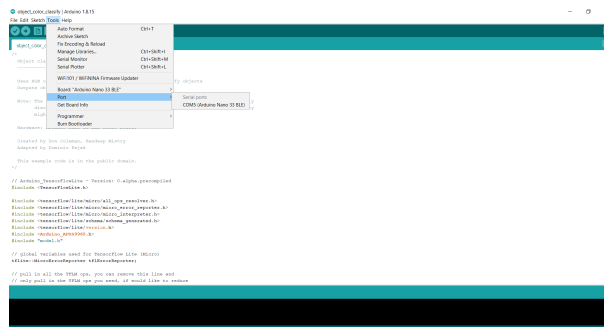


Figure 3.7.: Setting the Port

3.3. Test the Configuration

In this subsection, we will test whether the board is properly connected to the Arduino IDE by running an Example Blink Sketch. This simple test will make an onboard LED light up, verifying both the connection and basic functionality of the Arduino Nano 33 BLE Sense. By the end of this section, you will have confirmed that the hardware and software are working together seamlessly.

3.3.1. Testing the Steps by an Example Sketch `blink.ino`

The Arduino IDE contains a set of built-in examples for testing purposes. To verify the configuration and set up the board, the basic example `blink.ino` can be opened first. After uploading the example `blink.ino` the Builtin LED blinks with 2 Hz. To begin, open the Arduino IDE on the computer and follow the steps below:

1. **Open the Examples:** Once it's open, click on the **File** menu and hover over **Examples** in the dropdown menu.
2. **Selecting the Example Sketch:** A list of example categories will appear. Under the Built-in Examples, go to **01.Basics** and click on **Blink** as shown in 3.8.
3. **Blink Example Sketch:** After following steps 1 and 2, the Example Sketch **blink.ino** will open in a new window within the IDE as seen in Figure 3.9.
4. **Upload Example Sketch **blink.ino**:** Click the **Upload** button (right arrow icon) in the Arduino IDE toolbar to compile and upload the example sketch to the Arduino Nano 33 BLE Sense board. Once the upload is complete, the orange built-in LED starts blinking.

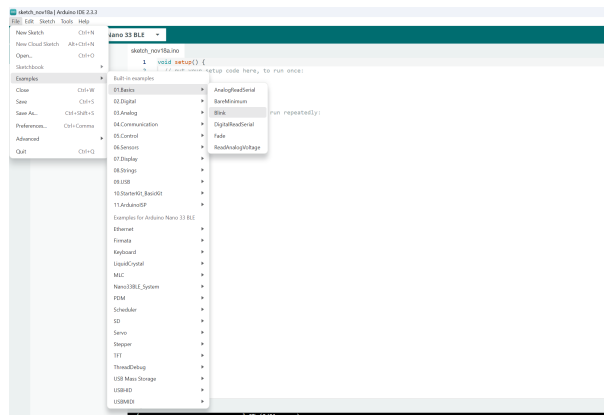


Figure 3.8.: LED Example Sketch

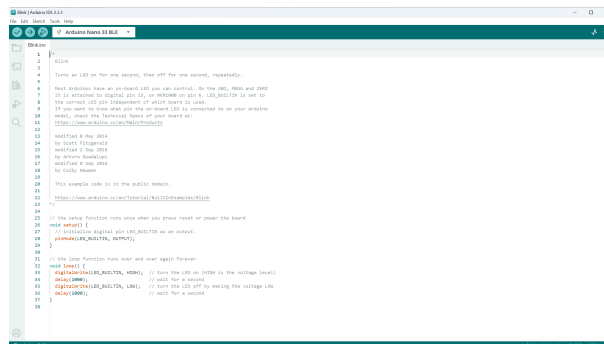


Figure 3.9.: LED-Built Example

With the successful blinking of the built-in orange LED, it is confirmed that the Arduino Nano 33 BLE Sense is properly connected to the Arduino IDE and its basic functionality has been verified. The board is now ready for further development and projects.

3.3.2. Description of Basic Sketch for Printing 'Hello'

To test the development environment and the basic functionality of the hardware, a simple example sketch that controls only one LED is suitable. This sketch provides the Arduino Integrated Development Environment (IDE). Under the path **File/Examples/01.Basics**, small example sketches can be selected. Here, the example **Fade** is used. In this case, the built-in LED, which is an RGB LED, is utilized.

```
int brightness = 0;  // how bright the LED is
int fadeAmount = 5;  // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
    // declare LED_BUILTIN to be an output:
    pinMode(LED_BUILTIN, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
    // set the brightness of LED_BUILTIN:
    analogWrite(LED_BUILTIN, brightness);

    // change the brightness for next time through
    //the loop:
    brightness = brightness + fadeAmount;

    // reverse the direction of the fading at the ends
    //of the fade:
    if (brightness <= 0 || brightness >= 255) {
        fadeAmount = -fadeAmount;
    }
    // wait for 30 milliseconds to see the dimming
    //effect
    delay(30);
}
```

As a result, the brightness of the built-in LED should gradually fade in and out.

4. Arduino Nano 33 BLE Sense

Arduino is an open-source electronics platform based on flexible, easy-to-use hardware and software. It provides us on-board microcontroller and microprocessor kits for making digital and analogue devices. The microcontroller, often called a tiny computer embedded on the Arduino board, normally in order to run these microcontrollers we need some type of electronics e.g.; diodes, resistors, capacitors, and transistors for making the voltage and current balancing. But the Arduino team makes a user-friendly environment for getting rid of these electronics complications to run the hardware on software, just power the board as per the required voltage, write the desired program and upload it in a few seconds, it will bring everything on the board and make us independent from any worry about the electronics complications. By the technological advancement in semiconductor and electronics industry, the control problems are now being solved by using these small size microcontrollers instead of mechanical and electrical switches. All Arduino boards have one thing in common which is a microcontroller, it is basically a really small computer, which helps us to make an edge computing application.[Ard21]

4.1. Arduino Nano 33 BLE Sense

The Arduino Nano Family is a set of boards with a tiny footprint, packed with features. It ranges from the inexpensive, entry model Nano, to the more feature-packed Nano BLE Sense / Nano RP2040 Connect which comprises of Bluetooth / Wi-Fi radio modules. These boards also have a set of embedded sensors, such as temperature, humidity, pressure, gesture, microphone and more. They can also be programmed with MicroPython and supports Machine Learning. [Raj19].

Arduino Nano 33 BLE Sense is one type of Arduino family board, which is come up with Bluetooth Low Energy (BLE) capability for communication and set of sensors. This compact and reliable Nano board is built around the NINA B306 module for BLE and Bluetooth 5.0 communication; Bluetooth 5.0 is the latest version of the Bluetooth wireless communication standard. Use of microcontrollers has become inevitable in almost every field of engineering. The Arduino Nano 33 BLE Sense module is based on Nordic nRF52480 processor that contains a powerful Cortex M4F and the board has a rich set of sensors that allow the creation of innovative and highly interactive designs. Its reduced power consumption, compared to other same size boards, together with the Nano form factor opens up a wide range of applications.

The model name Arduino Nano 33 BLE Sense, itself puts out some important information. It is called “Nano” because of its compact nano size. “33” is included in the model name to indicate that the board operates on 3.3V. Then the name “BLE” indicates that the module supports Bluetooth Low Energy and the name “Sense” indicates that it has on-board sensors like accelerometer, gyroscope, magnetometer, temperature and humidity sensor, Pressure sensor, Proximity sensor, Colour sensor, Gesture sensor, and even a built-in microphone. [Raj19].

Also, for making the RGB color and person detection, we need to make the interface of BLE 33 Sense with camera shield Arducam OV2640. The Arducam OV2640 camera is used to detect the RGB, object detection and also the gesture too. Arduino Nano 33 BLE Sense and camera shield Arducam is a perfect match for making ML and AI application, by having the set of sensors on board we just need to install the respective

library on Arduino board and it supports the functionality of sensors and Machine learning application.

The following figure 4.1 shows an Arduino Nano 33 BLE Sense Revision 2, showing how compact it is and small in size.

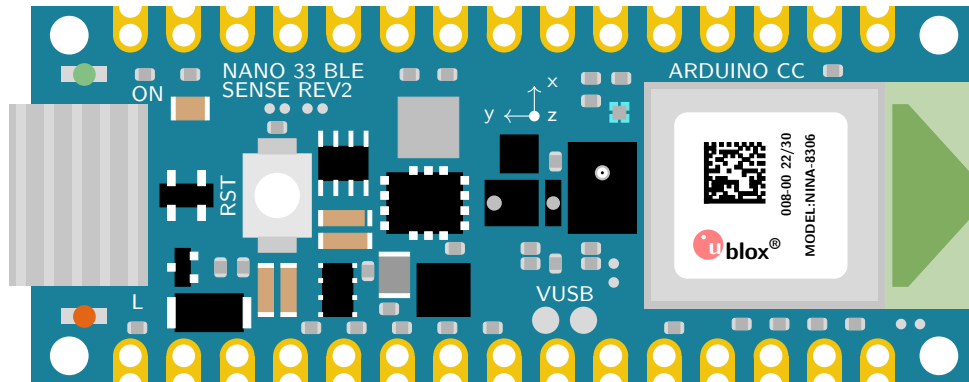


Figure 4.1.: Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store

The BLE (Bluetooth Low Energy) compact and reliable Nano board are built on the NINA B306 module for BLE and Bluetooth 5 communication; the NINA B306 module is based on the Nordic nRF52480 processor that contains a powerful Cortex M4F CPU. Its architecture is fully compatible with Arduino IDE Online and Offline. The Arduino Nano BLE 33 Sense has a following set of sensors on board: ADPS-9960, LPS22HB, HTS221, LSM9DS1, and MP34DT05-A. It is small in size, having all the required sensors on board. [Ard21]

The Arduino Nano 33 BLE Sense has the following set of sensors, BLE module, and its functionality below.

- The Bluetooth is managed by a NINA B306 module.
- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor.
- The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor.
- The LPS22HB reads barometric pressure and environmental temperature.
- The HTS221 senses relative humidity.
- The MP34DT05 supports sound detection.

4.2. Unterschied zwischen dem Arduino Nano 33 BLE Sense Rev1, dem Arduino Nano 33 BLE Sense Rev2 und dem Arduino Nano 33 BLE Sense Lite

4.2.1. What is the difference between Rev1 and Rev2?

The differences between the Arduino Nano 33 BLE Sense and the Arduino Nano 33 BLE Sense Rev2 concern the IMU, the temperature and humidity sensor, the microphone, the crypto chip and the voltage converter. In the second revision, the LSM9DS1 IMU has been replaced by two modules: the BMI270 acceleration and angular rate sensor and the BMI150 magnetometer. The HTS221 humidity sensor has been replaced by the HS3003, with the latter promising greater accuracy. The values

of the microphones have remained the same despite the change from the MP34DT05 to the MP34DT06JTR. Similarly, only the component designations of the voltage converters differ. The first generation uses the MPM3610, the Rev2 uses the MP2322. However, the crypto chip is no longer present in the Rev2.

The changes are summarized below:

- Replacement of the IMU of LSM9DS1 (9 axes) with a combination of two IMUs (BMI270 - 6 axes IMU and BMM150 - 3 axes IMU).
- Replacement of the temperature and humidity sensor from HTS221 to HS3003.
- Replacing the microphone from MP34DT05 to MP34DT06JTR.

In addition, some components and changes have been made to increase user-friendliness:

- Replacing the power supply from MPM3610 to MP2322.
- Add a VUSB solder pad to the top of the board.
- New test points for USB, SWDIO and SWCLK.

The figure 4.2 presents the layout of the first version of the Arduino Nano 33 BLE,

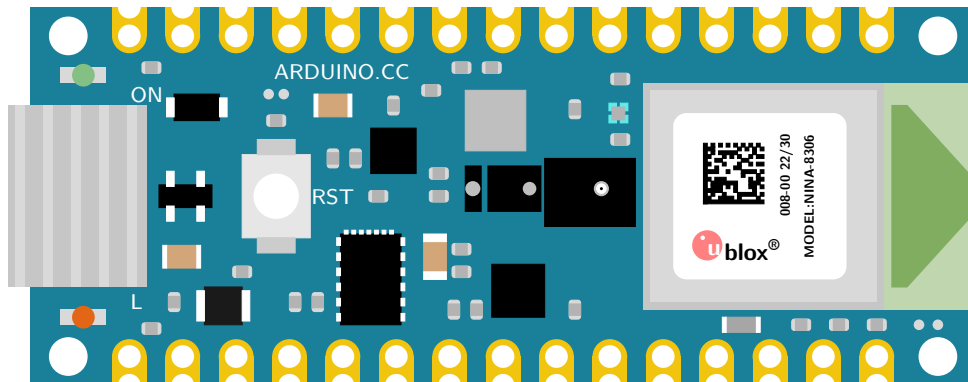


Figure 4.2.: Arduino Nano 33 BLE Sense Rev. 1

4.2.2. Changes in the Sketch

For sketches that were created with libraries such as LSM9DS1 for the IMU or HTS221 for the temperature and humidity sensor, these libraries must be changed to the following for the new revision:

- `Arduino_BMI270_BMM150` for the new combined IMU, and
- `Arduino_HS300x` for the new temperature and humidity sensor.

Rev 1 [TensorFlow Lite lib](#)

4.2.3. Arduino Nano 33 BLE Sense Lite

WS:citation The Tiny Machine Learning Kit contains only the Arduino Nano 33 BLE Sense Lite.

The Arduino Nano 33 BLE Sense Lite differs only slightly from the Arduino Nano 33 BLE Sense Revision 1. The only difference between the two models is that the Lite version does not have the HTS221, the sensor for relative humidity and temperature. The LPS22HB sensor, which is on the board, is a pressure sensor, but it can also be used to measure the temperature. It is therefore not possible to measure humidity with the Arduino Nano 33 BLE Lite. To measure relative humidity, a separate sensor must be connected. The reason for this decision is that the Arduino company has difficulties maintaining the stock. [FD22]

The figure 4.3 presents the layout of the Arduino Nano 33 BLE Lite,

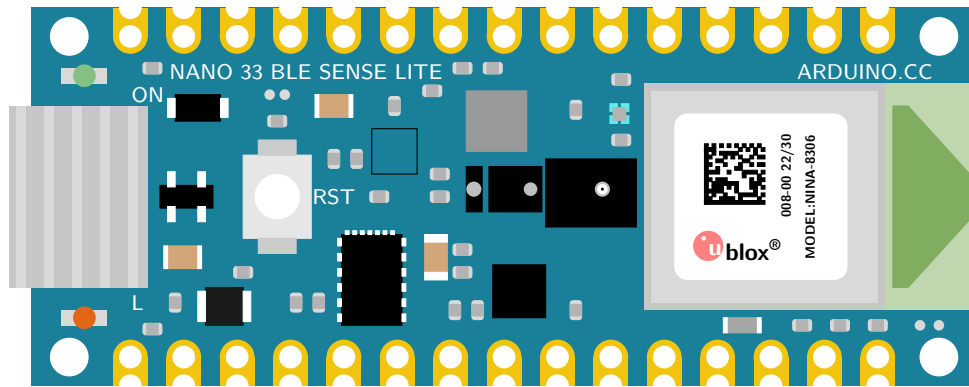


Figure 4.3.: Arduino Nano 33 BLE Sense Lite; the empty black square marks the position of the missing sensor HTS221

4.3. On-Board Sensor Description

Arduino Nano 33 BLE sense come up with the set of embed sensor on the board. The available embed sensors are commonly use for measuring both the analog and digital values around the surrounding. Arduino Nano 33 BLE sense is very small 45mm × 18mm in size, which makes it very useful for Internet of things (IOT) and Artificial intelligence (AI) application as a embed device where space is the main constrained issue. It is low power consumption board and operate normally on 3.3 V, we can say that this small size low power consumption board can operate on small batteries even for many months. Due to on-board available sensor, the low power consumption and mini architecture we can use this nano board anywhere. The Arduino Nano 33 BLE Sense is a completely new board on a well-known form factor. For getting detail information about each component of Arduino Nano 33 BLE Sense and data sheets of each sensor the following links give us a detail information [Ard21]. The short description of each sensor are as follow.

- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor. it can measure the proximity distance, light, color and gestures when moving close with the board.
- The sensor LSM9DS1 is a 9 axis Inertial Measurement Unit (IMU) use as a accelerometer, gyroscope, and magnetometer, this 9 axis sensor is ideal for wearable devices.

- The sensor LPS22HB is a barometric pressure sensor, it measures the environmental pressure which is useful for simple weather station monitoring.
- The sensor HTS221 senses the relative humidity, and temperature, to get highly accurate measurements of the environmental conditions.
- The sensor MP34DT05 is the digital microphone. it is useful for capturing, analyzing and detecting the sound in real time.
- The USB port allows you to connect Arduino Nano 33 BLE sense to your machine.
- There are 3 different LEDs that can be accessed on the Nano BLE Sense: RGB Programmable LED, the built-in orange Programmable LED and the Power LED.

The figure 4.4 show the embed sensors on the board, with powerful processor as compared to other arduino boards the nRF52840 from Nordic Semiconductors, a 32-bit ARM[®] Cortex[™]-M4 CPU processor running at 64 MHz are as follow.

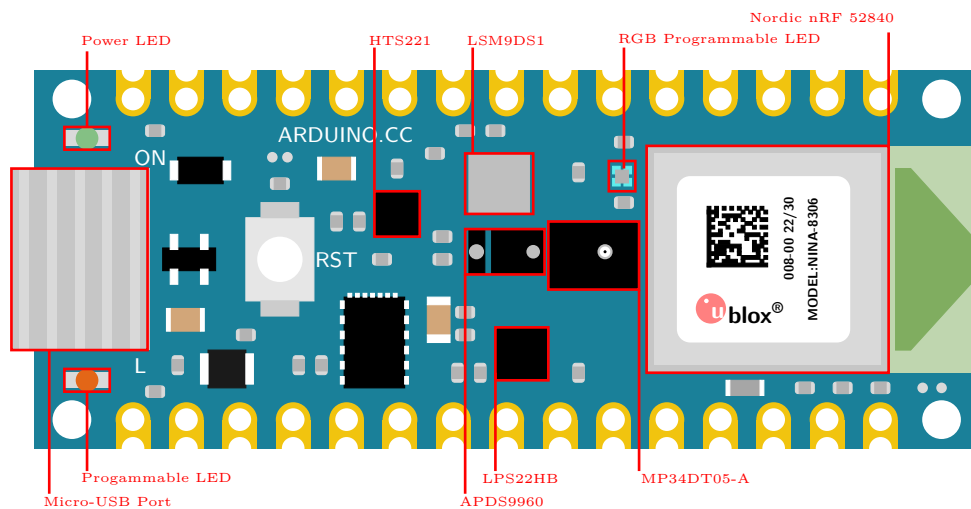


Figure 4.4.

Another point to bear in mind is the overall 'trueness' of sensor readings based on where do you place the actual sensors in the environment, as this could be quite critical. The operating temperature should not exceed 85⁰C and not lower than -40⁰C. Other factors like humidity level and air pressure values should also be kept in check.

MICROCONTROLLER	nRF52840
OPERATING VOLTAGE	3.3V
INPUT VOLTAGE (LIMIT)	21V
DC CURRENT PER I/O PIN	15 mA
CLOCK SPEED	64MHz
CPU FLASH MEMORY	1MB
SRAM	256KB
LED_BUILTIN	13
IMU (Accelerometer, Gyroscope, Magnetometer)	LSM9DS1
MICROPHONE	MP34DT05
GESTURE, LIGHT, PROXIMITY, COLOUR	APDS9960
BAROMETRIC PRESSURE	LPS22HB
TEMPERATURE, HUMIDITY	HTS221

Table 4.1.: Technical Specifications of Arduino Nano 33 BLE Sense [Ard21]

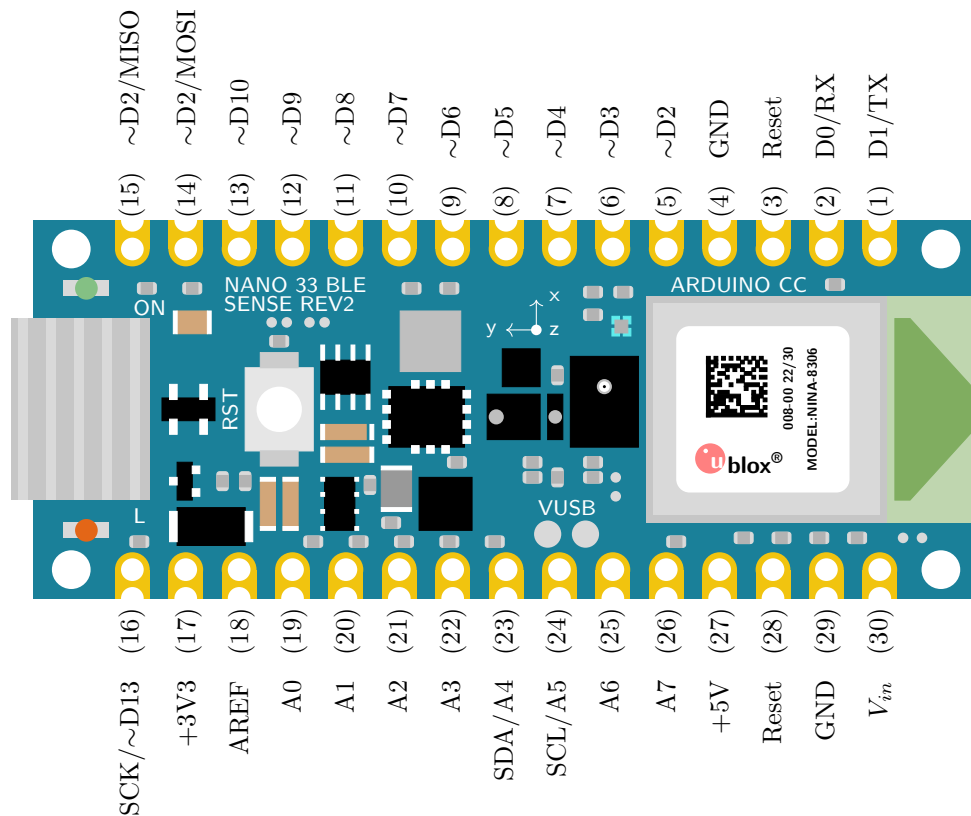


Figure 4.5.: Pin assignment of the Arduino Nano 33 BLE Sense and for the Arduino Nano 33 BLE Sense Rev 2; note that the orange built-in LED is connected to pin D13 and the power LED to pin D1. The built-in RGB LED occupies pins D18, D19 and D20.



Figure 4.6.: Connection with BLE and smartphone

4.4. Arduino Nano 33 BLE Pin Configuration

Arduino Nano 33 BLE is an advanced version of Arduino Nano board that is based on a powerful processor the nRF52840. The figure ?? shows that the board has the following pin configuration. Pin Configuration

Digital pin The number of digital I/O pins are 14 which receive only two values HIGH or LOW. These pins can either be used as an input or output based on the requirement. When these pins receive 5V, they are in a HIGH state and when they receive 0V they are in a LOW state.

Analog pin Total 8 analog pins available on the board A0 – A7. These pins get any value as opposed to digital pins that only receive two values HIGH or LOW. These pins are used to measure the analog voltage ranging between 0 to 5V.

PWM pin All digital pins can be used as PWM pins. These pins generate analog results with digital means.

SPI pin The board supports serial peripheral interface (SPI) communication protocol. This protocol is employed to develop communication between a controller and other peripheral devices like shift registers and sensors. Two pins are used for SPI communication i.e. Master Input Slave Output (MISO) and Master Output Slave Input (MOSI) are used for SPI communication. These pins are used to send or receive data by the controller.

I2C pin The board carries the I2C communication protocol which is a two-wire protocol. It comes with two pins SDA and SCL.

UART pin The board features a UART communication protocol that is used for serial communication and carries two pins Rx and Tx. The Rx is a receiving pin used to receive the serial data while Tx is a transmission pin used to transmit the serial data.

External Interrupts pin All digital pins can be used as external interrupts. This feature is used in case of emergency to interrupt the main running program with the inclusion of important instructions at that point.

LED at Pin 13 and AREF pin There is an LED connected to pin 13 of the board. And AREF is a pin used as a reference voltage for the input voltage.

Part II.

Applikation: Magic Faucet Fountain

5. Ultraschallsensor HC-SR04

5.1. Einleitung

Ein Ultraschallsensor misst berührungslos den Abstand zwischen Sensor und einer Werkstückoberfläche. Aus dieser Distanz kann der aktuelle Abstand zu der Oberfläche berechnet werden. Der Sensor arbeitet nach dem Prinzip der Laufzeitmessung von Ultraschallwellen und bietet eine einfache, kostengünstige und präzise Möglichkeit zur Pegelüberwachung, Warenerkennung und Abstandsmessung.[AG24] [TR18] [HS23]

5.2. Ultraschallsensoren zur Abstandsmessung

Ultraschallsensoren zur Abstandsmessung dienen der Ermittlung von Objektabständen über die Laufzeit von Ultraschallimpulsen und die Schallgeschwindigkeit. Da die Schallgeschwindigkeit temperatur- und luftdruckabhängig ist und zusätzlich durch die Luftzusammensetzung beeinflusst wird, ist bei der Auslegung auf solche Einflussfaktoren Rücksicht zu nehmen. Bei Nichtbeachtung sind Messungenauigkeiten in der Entfernungsmessung zu erwarten. [HS09] Umwelteinflüsse wie Feuchte, Staub und Rauch beeinflussen die Messgenauigkeit nicht. Einige Gase und Flüssigkeiten können die Sensorfunktion jedoch beeinträchtigen oder gar untüchtig machen. Beispiele hierfür sind CO₂ und Dämpfe von Lösungsmitteln. [HS+12] Ultraschallsensoren können nach ihrem Sensorprinzip in Tastbetrieb und Schrankenbetrieb unterschieden werden. Der Tastbetrieb basiert dabei auf der Auswertung der Laufzeitmessung zwischen dem Senden und Empfangen des Schalls. Der Schrankenbetrieb dahingegen beruht auf der Kontrolle, ob das gesendete Signal empfangen wurde [HLG19] Der Ultraschallsensor zur Abstandsmessung beruht auf ersterem Prinzip. Im Folgenden soll die Funktionsweise näher erläutert werden. Zunächst aktiviert die Steuerelektronik den Leistungsverstärker, welcher den Schallwandler mit einer elektrischen Leistung und einer Sinusspannung ansteuert. Dadurch arbeitet der Schallwandler als Lautsprecher und sendet einen Ultraschallimpuls, auch Burst genannt, aus. Bis der Schallwandler ausgeschwungen ist, dauert es zwei- bis dreimal so lang wie die Sendezeit. Anschließend leitet die Steuerelektronik den Empfangsbetrieb ein und der Schallwandler dient nun als Mikrofon. Sollte sich in der Schallkeule ein Objekt befinden, wird der Ultraschallimpuls zum Sender zurückreflektiert. Der Schallwandler beginnt zu schwingen und erzeugt eine Sinusschwingung, die auf einen Verstärker übertragen wird. Durch Autokorrelation wird kontrolliert, ob es sich bei dem reflektierten Schall wirklich um das Echo der ausgesandten Ultraschallwellen handelt. [HS09] Eine ganze Bandbreite an Objekten unabhängig von der Oberflächenbeschaffenheit, Farbe, Transparenz [HS+12] und Form [HS09] kann durch den Ultraschall erfasst werden. Der Ultraschall findet seine Anwendung in sehr vielen verschiedenen Bereichen 5.3, näher soll auf ihre Anwendung in den Bereichen der Medizin und Industrie eingegangen werden. In der Medizin gehört der Ultraschall mit zu den wichtigsten und häufigsten genutzten Diagnoseverfahren. Er lässt sich vielfältig einsetzen. So dient es zur Erkennung und Darstellung von Organen, gefüllten Hohlräumen, Gefäßen, Knochen, Sehnen und anderen Objekten im menschlichen Körper. Kennzeichnend für die Anwendung des Ultraschalls in der Medizin ist außerdem sein geringes Gesundheitsrisiko für Patienten. In der Industrie gibt es vielfältige Einsatzbereiche von Ultraschallsensoren. Sie können der Überwachung von Stellplätzen in der Lagerhaltung, der Füllstandbestimmung von Schüttgütern [HS09] oder der Detektion von Objekten und Grenzflächen dienen.

Beispiel für letztere Erwähnung ist die Rückfahrlilfe bei Fahrzeugen. Eine weitere sehr wichtige Anwendung findet sich in der zerstörungsfreien Prüfung von Werkstücken und Werkstoffen wieder. Vorrangig wird dabei das Materialgefüge auf Unebenheiten und Au'alligkeiten überprüft. Solche Untersuchungen werden häufig für sicherheitsrelevante Bauteile wie Eisenbahnräder oder Druckbehälter für Atomanlagen verwendet [MM17].

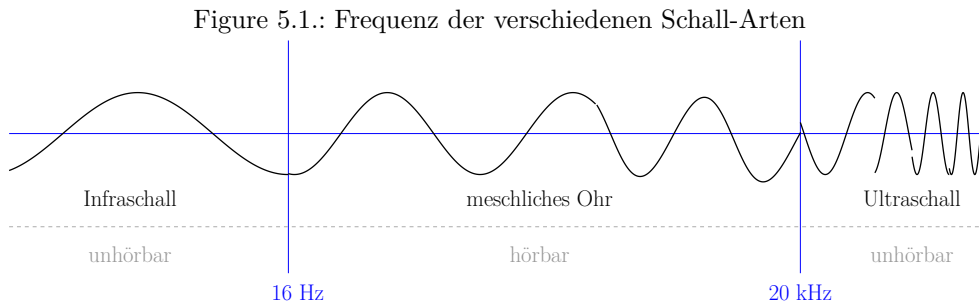
5.3. Grundlagen zur Verwendung eines Ultraschallsensors

Um einen Ultraschallsensor gezielt und effizient einsetzen zu können, ist ein grundlegendes Verständnis über die Funktionsweise, technischen Eigenschaften und Anwendungsvoraussetzungen abstandsmessender Ultraschallsensoren notwendig. Die folgenden Unterkapitel bieten eine strukturierte Einführung in die physikalischen und technischen Grundlagen des Sensors. Ziel ist es, ein tiefergehendes Verständnis für den Aufbau, die Signalverarbeitung und die praktische Handhabung eines Ultraschallsensors zu vermitteln. [AG24]

5.3.1. Grundlagen Schall

Als Schall bezeichnet man die Ausbreitung von lokalen Druckschwankungen in einem elastischen Medium. Die Schallwelle breitet sich dabei longitudinal aus, die Auslenkung der Moleküle erfolgt also in Ausbreitungsrichtung [TM24]. Die Einteilung der Schallbereiche erfolgt auf Grundlage der Frequenz, siehe außerdem 5.1:

- *Infraschall*: bis 16 Hz,
- *Hörbarer Schall*: von 16 Hz bis 20 kHz,
- *Ultraschall*: von 20 kHz bis 1 GHz,
- *Hyperschall*: über 1 GHz [HS23].



5.3.2. Schallgeschwindigkeit und Wellenlänge

Die Geschwindigkeit, mit der sich Schall in Luft ausbreitet, ist von der Lufttemperatur T und der Luftfeuchtigkeit φ abhängig. Da die Luftfeuchtigkeit einen erheblich geringeren Einfluss auf den Wert hat

$$\Delta v_{Schall,\varphi} \approx 1,2 \frac{\text{m}}{\text{s}}; 0 < \varphi < 1, T_C = 20^\circ\text{C}, \quad (5.1)$$

[HS23] lässt sich die Schallgeschwindigkeit mit ausreichender Genauigkeit mit der Formel 5.2

$$v_{Schall} = \sqrt{\kappa R T} \quad (5.2)$$

und der Annahme $\varphi = 0$ berechnen [TM24]. Dabei ist $\kappa = 1,4$ der Adiabatenexponent von Luft, $R = 287 \frac{\text{J}}{\text{kg K}}$ die spezifische Gaskonstante von Luft und T die absolute Temperatur in Kelvin [TM24; Wil22]. Für eine angenommene Temperatur von $T_C = 20^\circ\text{C}$ ergibt sich damit eine Geschwindigkeit von $v_{\text{Schall}} = 343,2 \frac{\text{m}}{\text{s}}$. In Abb. 5.2 ist die Schallgeschwindigkeit in Luft im Temperaturbereich von $T_C = -10^\circ\text{C}$ bis 60°C dargestellt.

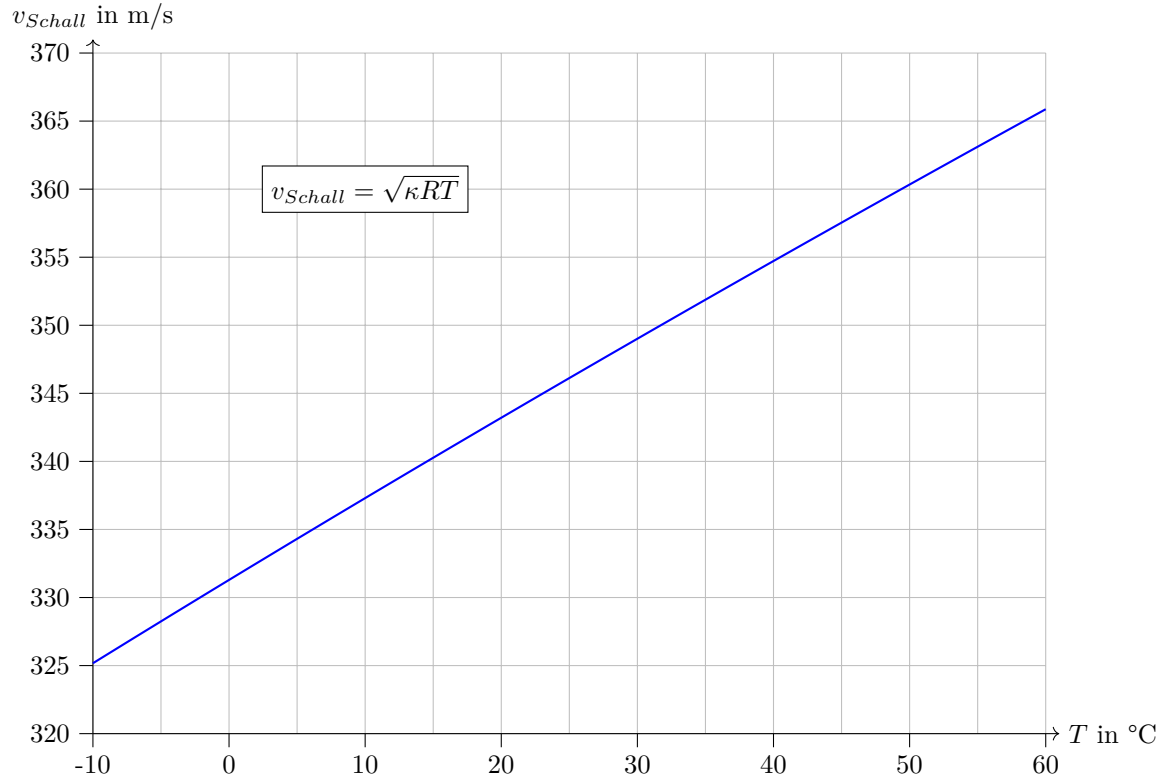


Figure 5.2.: Ausbreitungsgeschwindigkeit von Schall in Luft

Über den Formelzusammenhang 5.3

$$v_{\text{Schall}} = \lambda f \quad (5.3)$$

lässt sich bei gegebener Frequenz f und mit der berechneten Schallgeschwindigkeit v_{Schall} die Wellenlänge des Schalls berechnen [Wil22]. Diese beträgt bei einer angenommenen Frequenz von $f = 40 \text{ kHz}$ und der in Abschnitt 5.3.2 berechneten Schallgeschwindigkeit $\lambda = 8,58 \cdot 10^{-3} \text{ m}$.

5.3.3. Weg- und Abstandsmessung mit Ultraschall

Um eine Entfernung oder einen Abstand eines Objekts zu dem Sensor zu ermitteln, wird der Sensor im sog. *Tastbetrieb mit Echo-Laufzeit-Messung*, wie in Abb. 5.3 zu sehen, verwendet. Dabei sendet der Sensor zunächst ein Schallpuls in Richtung des zu messenden Objekts aus (Lautsprecher), welche zum Teil von diesem reflektiert wird und somit zurück zum Sensor gelangt (Echo). Dieses Echo kann der Sensor detektieren (Mikrofon) und über die gemessene Laufzeit des Echos sowie die Schallgeschwindigkeit im dem den Sensor umgebenden Medium die Distanz berechnen [HS23].

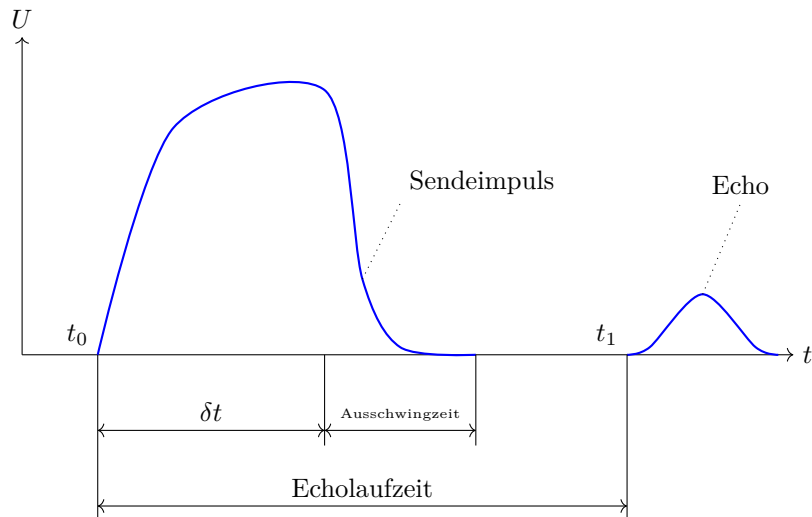
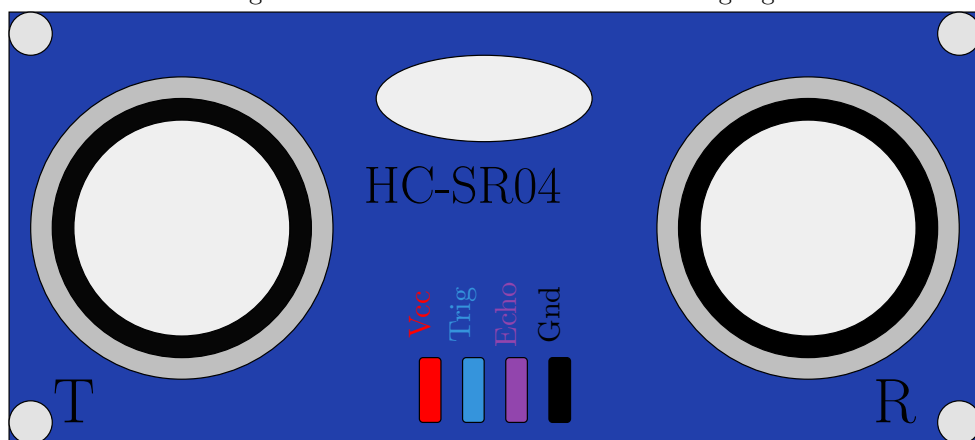


Figure 5.3.: Spannungsverlauf Schallwandler bei einer Echo-Laufzeit-Messung, nach [HS23]

5.4. Ultraschall-Abstandsensord HC-SR04

Der Ultraschallabstandssensor in Abbildung 5.4 enthält den Sensor HC-SR04. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt und ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm spezifiziert. Dafür nutzt er eine Schallfrequenz von 40 kHz. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm. Die Auflösung des Sensors beträgt ± 1 mm. Angeschlossen wird der Abstandssensor über den Voltage at the Common Collector (VCC)-Pin, Trig/Serail Clock (SCL)-Pin, Echo/Serail Data (SDA)-Pin und dem Ground (GND)-Pin. Der Sensor kann über die drei Schnittstellen General Purpose Input Output (GPIO), Universal Asynchronous Receiver Transmitter (UART) und Inter-Integrated Circuit (I²C) verbunden werden [Sim].

Figure 5.4.: Ultraschallsensor mit Pin-Belegung



5.5. Spezifikation

Der Sensor HC-SR04 ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm mit einer Auflösung von ± 1 mm spezifiziert. Dafür nutzt er eine Schallfrequenz von 40 kHz. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm.

Angeschlossen wird der Abstandssensor über den SCL-Pin für das Triggersignal, den SDA-Pin fürs Echo, den VCC-Pin zur Stromversorgung und dem GND-Pin.

Der Sensor kann über die drei Schnittstellen GPIO, UART und I²C verbunden werden [Sim].

Bei den folgenden Beispielprogrammen wird der Arduino Nano 33 BLE Sense verwendet. In den Beispielen werden die Pins D6 und D9 für die Datenübertragung verwendet. In der Tabelle 5.1 ist dies für die Grove-Schnittstelle festgehalten und zudem auch in der Abbildung 5.4 zu sehen.

Table 5.1.: Pinbelegung für den Ultraschallabstandssensor [Sim]

Arduino Pin	Ultraschall Abstandssensor
D9	Trig
D6	Echo
3V3	VCC
GND	GND

5.5.1. Anwendungen

Ultraschallsensoren finden in der Praxis Anwendung in verschiedenen Bereichen, die auf die Ausbreitung des Schalls über unterschiedliche Medien basieren [HS23]. Es lassen sich drei Hauptkategorien unterscheiden: In der ersten Kategorie breitet sich der Schall über Luft aus. Typische Anwendungen sind hier Füllstandmessungen, Durchmessererfassungen bei Auf- und Abwickelsteuerungen sowie Kollisionsschutz und Objekterkennung [HS23]. Die zweite Kategorie umfasst Sensoren, bei denen sich der Schall über Flüssigkeiten ausbreitet. Diese finden vor allem im maritimen Bereich Verwendung, beispielsweise in Sonarsystemen zum Orten von Unterseebooten oder Fischschwärmen sowie in Echoloten zur Kartografie des Meeresbodens [HEG21]. Die dritte Kategorie umfasst Körperschallsensoren, die in der Industrie zur zerstörungsfreien Materialprüfung sowie in der Medizin zur Ultraschalldiagnostik verwendet werden. [HEG21]

5.6. Bibliothek

5.6.1. Beschreibung

Zur Ansteuerung des Ultraschallsensors HC-SR04 wird eine Arduino-kompatible Bibliothek `NewPing` verwendet, die die Initialisierung des Sensors sowie das Senden und Empfangen von Ultraschallsignalen abstrahiert. Diese Bibliothek kapselt die Funktionen zum Triggern, zur Laufzeitmessung und zur Berechnung der Distanz [AG24].

5.6.2. Installation

Die Installation der Bibliothek `NewPing`, welche die Funktionalität des Ultraschallsensors HC-SR04 unter Arduino abstrahiert, erfolgt über den Bibliotheksverwalter der Arduino IDE. Diese bietet eine benutzerfreundliche Oberfläche zur Integration externer Bibliotheken. [AG24]

Schrittweise Anleitung:

1. Öffne die Arduino IDE (Version 1.8.x oder neuer empfohlen).

2. Navigiere zu: Sketch > Bibliothek einbinden > Bibliotheken verwalten...
3. Im Suchfeld oben rechts den Begriff `NewPing` eingeben
4. Wähle den Eintrag `NewPing` by Tim Eckel aus (Version 1.9.7 oder neuer empfohlen).
5. Klicke auf „Installieren“.

5.6.3. Funktionen

Die `NewPing`-Bibliothek bietet eine Vielzahl von Funktionen, die die Nutzung von Ultraschallsensoren erleichtern und erweitern. Die wichtigsten Funktionen sind: [AG24]

`NewPing()`: Initialisierung eines Sensors durch Angabe der Pins für Trigger- und Echo-Signale sowie der maximalen Messdistanz.

`ping()`: Senden eines Ultraschallimpulses und Rückgabe der gemessenen Laufzeit in Mikrosekunden.

`getDistance()`: Direkte Ausgabe der gemessenen Entfernung in Zentimetern basierend auf der Standard-Schallgeschwindigkeit in Luft.

`pingMedian()`: Durchführung mehrerer Messungen und Rückgabe des Medianwertes zur Reduktion von Ausreißern.

Diese Funktionen ermöglichen eine einfache und präzise Nutzung von Ultraschallsensoren in verschiedenen Anwendungen, von der Einzelmessung bis hin zur Durchführung mehrerer Messungen zur Reduzierung von Ausreißern.

5.7. Kalibrierung

Damit der HC-SR04 präzise misst, muss er kalibriert werden. Die wichtigste Einflussgröße ist die Schallgeschwindigkeit, da sie von der Umgebungstemperatur abhängt. Eine Kalibrierung erfolgt durch den Vergleich mit einer bekannten Referenzdistanz. Ergibt sich eine systematische Abweichung, kann diese durch einen festen Korrekturwert (Offset) oder durch Anpassung der Schallgeschwindigkeit ausgeglichen werden. [TR14] Der Ultraschallabstandssensor wird für die Verwendung bei der Temperatur von 20 °C ausgelegt. Die berechnete Schallgeschwindigkeit $v_{Schall} = 343,2 \frac{\text{m}}{\text{s}}$ bei 20 °C wird für die Umrechnung der Schalldauer in die Distanz zur Reflexionsfläche verwendet.

5.8. Simple Code

Zur Überprüfung der grundlegenden Funktionalität des Ultraschallsensors wurde ein Simple Code 5.1 verwendet. Das Programm dient zum einfachen Test mithilfe der `NewPing`-Bibliothek. Es misst die Laufzeit (Echozeit) des Schallsignals. Die Werte werden anschließend über den Seriellen Monitor ausgegeben.

5.9. Einfache Anwendung

In der Industrie ist es oft wichtig zu wissen, wann ein bestimmter Pegel einer Flüssigkeit unterschritten wird. Dafür können Ultraschallsensoren eingesetzt werden. Sie messen kontinuierlich den Pegel und können dem Programm so permanent einen Messwert bereitstellen. Wird ein bestimmter Messwert über- oder unterschritten, kann das Programm eine Warnmeldung erzeugen. Dieses Arduino-Programm 5.2 nutzt einen HC-SR04 Ultraschallsensor, um die Entfernung zu einem Objekt kontinuierlich zu

Listing 5.1.: Einfacher Code zum Testen des Ultraschallsensors

```

1000 /**
1001  * @file laufzeitMessung.ino
1002  * @brief Simple test program for measuring the echo time of an HC-SR04
1003  *        ultrasonic sensor using the NewPing library.
1004  *
1005  * This program initializes the ultrasonic sensor and performs a basic
1006  * runtime measurement (ping)
1007  * to determine the echo time in microseconds. The result is printed to
1008  * the Serial Monitor.
1009  */
1010 #include <NewPing.h>
1011
1012 // Define pins and maximum distance
1013 #define TRIGGER_PIN 9    ///< Pin connected to the trigger pin of the
1014   HC-SR04
1015 #define ECHO_PIN 10     ///< Pin connected to the echo pin of the HC-
1016   SR04
1017 #define MAX_DISTANCE 200 ///< Maximum distance to measure (in cm)
1018
1019 // Create a NewPing object named 'sonar'
1020 NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1021
1022 /**
1023  * @brief Arduino setup function.
1024  *
1025  * Initializes serial communication for output.
1026  */
1027 void setup() {
1028   Serial.begin(9600);
1029   Serial.println("Ultrasonic runtime measurement started...");
1030 }
1031
1032 /**
1033  * @brief Arduino main loop function.
1034  *
1035  * Measures the echo time using the ping() function and prints it in
1036  * microseconds.
1037  */
1038 void loop() {
1039   delay(1000); // Wait 1 second between measurements
1040
1041   // Measure echo time in microseconds
1042   unsigned int echoTime = sonar.ping();
1043
1044   // Print result
1045   Serial.print("Echo time: ");
1046   Serial.print(echoTime);
1047   Serial.println(" mikro-sekunden");
1048 }

```

../Code/Arduino/UltraSonicHCSR04/laufzeitMessung/laufzeitMessung.ino

überwachen. Es erkennt, ob eine kritische Schwelle (25 cm) unterschritten wurde und gibt in diesem Fall eine Warnmeldung im Seriellen Monitor aus.

Listing 5.2.: Einfache Anwendung als Schwellenwert-Melder

```

1000 /**
1001  * @file schwellenwertMeldung.ino
1002  * @brief Simple application using an HC-SR04 ultrasonic sensor to
1003  *        monitor a distance threshold.
1004  *
1005  * This program continuously measures the distance using the HC-SR04
1006  * sensor and triggers a warning
1007  * when a critical threshold of 25 cm is reached or undershot. The
1008  * distance is measured using
1009  * pingMedian() for stable values, and getDistance() is also
1010  * demonstrated for direct measurement.
1011  */
1012
1013 #include <NewPing.h>
1014
1015 #define TRIGGER_PIN    9    ///< Pin connected to the trigger pin of
1016   the HC-SR04
1017 #define ECHO_PIN       10   ///< Pin connected to the echo pin of the
1018   HC-SR04
1019 #define MAX_DISTANCE   100  ///< Maximum distance to measure (in cm)
1020
1021 #define TARGET_DISTANCE 30  ///< Reference distance (in cm)
1022 #define CRITICAL_THRESHOLD 25 ///< Critical threshold for warning (in
1023   cm)
1024
1025 /// Create a NewPing object named 'sonar'
1026 NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1027
1028 /**
1029  * @brief Arduino setup function.
1030  *
1031  * Initializes serial communication and outputs startup message.
1032  */
1033 void setup() {
1034   Serial.begin(9600);
1035   Serial.println("Distance threshold monitoring started...");
1036 }
1037
1038 /**
1039  * @brief Arduino main loop function.
1040  *
1041  * Continuously measures the distance using pingMedian().
1042  * If the measured distance is less than or equal to the critical
1043  * threshold,
1044  * a warning message is printed.
1045  */
1046 void loop() {
1047   delay(500); // Measurement interval
1048
1049   // Use pingMedian() for stable distance reading (average of multiple
1050   // pings)
1051   unsigned int distance = sonar.pingMedian();
1052
1053   // Print measured distance
1054   Serial.print("Measured distance: ");
1055   Serial.print(distance);
1056   Serial.println(" cm");
1057
1058   // Check if distance is at or below critical threshold
1059   if (distance > 0 && distance <= CRITICAL_THRESHOLD) {
1060     Serial.println("WARNING: Critical distance reached!");
1061   }
1062
1063   // Optional: demonstrate use of getDistance() (less accurate, uses
1064   // default single ping)
1065   // unsigned int quickDistance = sonar.getDistance();
1066   // Serial.print("Quick distance (getDistance): ");
1067   // Serial.print(quickDistance);
1068   // Serial.println(" cm");
1069 }

```

../Code/Arduino/UltraSonicHCSR04/schwellenwertMeldung/schwellenwertMeldung.ino

5.10. Tests

5.10.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird überprüft, ob der Sensor korrekt anspricht und brauchbare Messwerte liefert. Dazu wird der HC-SR04 an einen Mikrocontroller (z. B. Arduino Nano 33 BLE Sense Lite) angeschlossen und ein einfacher Sketch 5.3 aufgespielt, welcher die gemessene Entfernung über die serielle Schnittstelle ausgibt. [HS23] Ein Testobjekt wird in einem bekannten Abstand, typischerweise 20 cm, orthogonal vor den Sensor gehalten. Die gemessene Entfernung wird mit einem Maßband überprüft. Stimmen beide Werte überein, ist die Grundfunktion des Sensors sichergestellt. Dieser Test eignet sich insbesondere zur ersten Inbetriebnahme oder zur Überprüfung von Lötverbindungen, Spannungsversorgung und Signalverarbeitung.

Listing 5.3.: Code für die Messung einer Strecke

```

1000 /**
1001  * @file entfernungMessung.ino
1002  * @brief Simple program to measure and display distance in centimeters
1003  *        using the HC-SR04 ultrasonic sensor.
1004  *
1005  * This program initializes the HC-SR04 sensor and continuously measures
1006  * the distance to an object.
1007  * The distance is displayed in centimeters on the Serial Monitor using
1008  * the NewPing library.
1009  */
1010 #include <NewPing.h>
1011
1012 // Define pins and maximum measurement distance
1013 #define TRIGGER_PIN 9    ///< Pin connected to the trigger pin of the
1014   HC-SR04
1015 #define ECHO_PIN 10     ///< Pin connected to the echo pin of the HC
1016   -SR04
1017 #define MAX_DISTANCE 200 ///< Maximum distance to measure (in cm)
1018
1019 /// Create a NewPing object for distance measurement
1020 NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1021
1022 /**
1023  * @brief Arduino setup function.
1024  *
1025  * Initializes serial communication and prints startup message.
1026  */
1027 void setup() {
1028   Serial.begin(9600);
1029   Serial.println("Simple distance measurement started...");
1030 }
1031
1032 /**
1033  * @brief Arduino main loop function.
1034  *
1035  * Measures the distance using getDistance() and prints it in
1036  * centimeters.
1037  */
1038 void loop() {
1039   delay(1000); // Wait 1 second between measurements
1040
1041   // Measure distance in cm (automatically calculated by NewPing)
1042   unsigned int distance = sonar.getDistance();
1043
1044   // Print result
1045   Serial.print("Distance: ");
1046   Serial.print(distance);
1047   Serial.println(" cm");
1048 }

```

../Code/Arduino/UltraSonicHCSR04/entfernungMessung/entfernungMessung.ino

5.10.2. Test aller Funktionen

Ein umfassender Funktionstest geht über die reine Abstandsmessung hinaus. Hierbei werden systematisch verschiedene Einflüsse auf die Messgenauigkeit überprüft: [HS23]

- **Oberflächenbeschaffenheit:** Glatte, harte Oberflächen liefern bessere Reflexionen als weiche oder absorbierende Materialien wie Schaumstoff oder Stoff
- **Ausrichtungswinkel:** Objekte, die schräg zur Sensorachse stehen, reflektieren das Signal schlechter oder gar nicht. Der Test prüft, in welchem Winkelbereich

zuverlässige Ergebnisse erzielt werden.

- **Entfernungsbereich:** Es wird getestet, ob der Sensor sowohl im Nahbereich (unter 10 cm) als auch im Fernbereich (bis 400 cm) verlässlich misst.
- **Umwelteinflüsse:** Staub, leichte Luftströmungen oder Temperaturveränderungen werden simuliert, um die Stabilität des Sensors unter realistischen Bedingungen zu bewerten.
- **Reproduzierbarkeit:** Mehrfache Messungen unter gleichen Bedingungen sollten ähnliche Ergebnisse liefern. Eine Standardabweichung der Messwerte kann berechnet werden, um die Präzision zu bewerten.

5.10.3. Lösungsansätze

Die Genauigkeit von Ultraschall-Abstandsmessungen hängt unter anderem von der Schallgeschwindigkeit ab, welche wiederum temperaturabhängig ist. Wird die Geschwindigkeit im Auswertungsprozess als konstanter Wert angenommen, kann es bei Abweichungen von der tatsächlichen Umgebungstemperatur zu Messfehlern kommen. Eine Möglichkeit zur Korrektur besteht darin, die aktuelle Temperatur zu erfassen und die Schallgeschwindigkeit entsprechend anzupassen. In vielen Systemen wird hierfür ein interner Temperatursensor verwendet. Liegt dieser jedoch direkt auf der Platine und ist in einem Gehäuse untergebracht, kann es durch Eigenwärme und Isolation zu einer zeitverzögerten oder verfälschten Temperaturmessung kommen. Die Verwendung eines separaten Temperatursensors außerhalb des Gehäuses wäre in solchen Fällen eine mögliche Lösung, ist jedoch mit zusätzlichem Aufwand verbunden. Für das vorliegende Projekt, bei dem ein visueller Effekt eines scheinbar schwebenden Wasserhahns erzeugt werden soll, ist eine sehr hohe Messgenauigkeit nicht zwingend erforderlich. Der Wasserstand im Behälter wird zusätzlich durch Bohrungen geregelt, die ein Überlaufen verhindern. Die Ultraschallmessung dient primär der groben Überwachung des Füllstandes, beispielsweise zur rechtzeitigen Nachfüllung. Geringe Messabweichungen durch temperaturbedingte Schwankungen der Schallgeschwindigkeit sind daher tolerierbar und haben keinen wesentlichen Einfluss auf die Funktionalität der Installation.

5.11. Weiterführende Anwendungen

Ultraschallsensoren wie der HC-SR04 finden in folgenden Gebieten ihre Anwendung:

- **Robotik:** Hinderniserkennung und Kollisionsvermeidung bei autonomen Systemen [TR14]
- **Industrieautomation:** In Tanks und Silos, auch bei Staub oder Dampf [HS23].
- **Fahrzeugtechnik:** Einparkhilfen und Abstandswarnsysteme [HS23].
- **Medizin:** Kontaktlose Abstandsmessung in sterilen Umgebungen, z. B. bei Desinfektionsstationen [HS23]

Die Stärken des Sensors liegen in der einfachen Integration, dem günstigen Preis und der Robustheit gegenüber Lichtverhältnissen oder Verschmutzung. Die genannten Eigenschaften werden auch in der Fachliteratur als Vorteile von Ultraschallsensoren gegenüber optischen Verfahren hervorgehoben [HS23]

5.12. Further Readings

Tränkler, H.-R.: mSensortechnik – Handbuch für Praxis und Wissenschaft. 2. Aufl., Springer Vieweg, 2018 [TR18]

- Ein schöner Einstieg und Überblick über die Sensortechnik und ihre Anwendung heutzutage.

Hering, E.: Sensoren in Wissenschaft und Technik – Funktionsweise und Einsatzgebiete. 3. Aufl., Springer Vieweg, 2023 [HS23]

- Ein tiefer Einblick in die Sensortechnik, besonders schöne Abbildungen und Erklärungen. Von Grund auf an eine Einführung für Ultraschallsensoren

Arduino AG: NewPing – Library Documentation, Arduino.cc, abgerufen 2025. [AG24]

- Auf der Arduino Website kann man sich viele Extra-Informationen zum Arduino und weiterführende Anwendungen, Beispiele und Projekte anschauen.

6. Comet Tauchpumpe ELEGANT 12V

6.1. Einführung

Pumpen sind Einrichtungen zur Förderung von flüssigen Medien von einem niedrigeren auf ein höheres statisches Druckniveau. Dieser Vorgang wird über verschiedene Wege erreicht. Ein höheres statisches Druckniveau kann durch folgende Verfahren erreicht werden:

- das Einwirken einer Druckkraft,
- die Übertragung mechanischer Arbeit,
- den Impulsaustausch,
- das Zumischen von Druckluft zum Förderwasser,
- die Stoßwirkung einer in ihrer Bewegung gehemmten Wassersäule oder
- durch Ausnutzung der Zähigkeit des Mediums unter Verwendung eines Fördergewindes

All die verschiedenen Arbeitsweisen beeinflussen die Pumpenbauart. Die häufigsten vorkommenden Pumpen sind die Verdränger- und Kreiselpumpe. Im Folgenden wird näher auf die Kreiselpumpe eingegangen. Kreispumpen gibt es in ein- oder mehrstufigen Ausführungen. Ihre Arbeitsweise ist gleich, sie unterscheiden sich lediglich in ihrem Aufbau.[Sch13] Pumpen sind oft Komponenten von komplexen Prozessen, deren Ausfall erhebliche Auswirkungen verursachen kann. Die Anforderung an maximale Betriebssicherheit ist daher von besonderer Bedeutung.[GG20]

6.1.1. Arbeitsweise Kreiselpumpen

Die Kreiselpumpe verfügt über ein rotierendes Laufrad mit Schaufeln. Dieses Rad über-trägt mechanische Arbeit auf die in den Laufradkanälen befindliche Flüssigkeit, die da-bei durch Fliehkräfte aus dem Laufrad verdrängt wird. Durch den Vorgang kommt es zur Druckerhöhung und Geschwindigkeitszunahme des Mediums. Die gleichzeitige Zunahme der Absolutgeschwindigkeit des Fördermittels stellt eine unerwünschte Strömungsbegleiterscheinung dar, da in der Pumpe vorrangig eine Druckerhöhung erzielt werden soll. Daher muss diese anschließend in Druckenergie umgewandelt werden. Die Umwandlung wird durch das um das Laufrad befindliche ringförmige Leitrad realisiert. Zusammen bilden Lauf- und Leitrad eine Stufe (daher einstufige Kreiselpumpe). Die kontinuierliche Aufrechterhaltung der Strömung erfolgt dadurch, dass durch die aus dem drehenden Laufrad verdrängte Flüssigkeit eine Saugwirkung erzeugt und das gleiche Volumen an Flüssigkeit über Saugstutzen wieder in die Pumpe eintritt. Die Wahl einer Kreiselpumpe erfolgt nach Förderhöhe und Druckhöhe. So kann nach der Förderhöhe in Nieder-, - Mittel – und Hochdruckpumpen unterteilt werden. Im Hin-blick auf die Druckhöhe, wählt man bei großen Druckhöhen mehrstufige Kreiselpumpen. Sie sind im Aufbau wie die einstufigen Kreiselpumpen mit dem einzigen Unter-schied, dass mehrere Stufen (gleichartige Lauf- und Leiträder) hintereinandergeschaltet sind. Dabei wird die Anzahl der Stufen durch die Förderhöhe, Drehzahl der Pumpe und die Größe des Volumenstromes bestimmt. Bei geringen Förderhöhen und großen Volumenströmen entspricht die Laufradform, die dem Franciscrade bei Wasserturbinen. Die Schaufeln des

Laufrades sind bei dieser Bauart räumlich gekrümmt bzw. verwunden. Eine Erhöhung des Volumenstroms bei gering bleibender Förderhöhe erfordert eine mehrströmige Ausführung der Pumpe. Bei dieser Bauart sind mehrere Laufräder parallelgeschaltet. Verwirklicht wird das Ganze durch zwei Einzlräder, welche zu einem Radpaar mit doppelseitigem Wassereintritt vereinigt werden. Beispiele für mehrströmige Pumpen sind Kühlwasserpumpen für Kondensationsanlagen. Eine weitere Anwendung bei gleichbleibenden Bedingungen (Vergrößerung Volumenstrom, klein bleibende Förderhöhe) findet die Pumpe halbaxialer Bauart. Diese Bauart zeichnet sich dadurch aus, dass die axial eintretende Flüssigkeit nur noch eine geringe Umlenkung nach der radialen Richtung erfährt. Sie ersetzt die mehrstufige und bildet den Übergang von der Radial- zur Axialpumpe. Reine Axialpumpen eignen sich besonders zur Überwindung großer Volumenströme bei geringen Strecken.[Sch13]

6.2. Charakteristische Größen von Pumpen

Pumpen werden durch charakteristische Größen beschrieben. Im Folgenden sollen die Hauptparameter näher erläutert werden.

6.2.1. Volumenstrom

Der Volumenstrom oder Durchsatz gibt an, wie viel Raum ein Medium pro Zeiteinheit durchströmt. Er wird definiert durch:

$$\dot{V} = \frac{\dot{m}}{\rho} \quad (6.1)$$

Die Einheit ist beispielsweise m^3/h , m^3/s , l/s oder l/min .

Der Volumenstrom setzt sich aus dem effektiv geförderten Volumenstrom und dem Leckvolumenstrom zusammen. Daraus ergibt sich der äußere und innere Dichtheitsgrad:

$$\lambda = \frac{\dot{V}_s}{\dot{V}_i} \quad (6.2)$$

Der Dichtheitsgrad ist dimensionslos und liegt typischerweise zwischen 0,92 und 0,98.

6.2.2. Förderhöhe

Die Förderhöhe (auch Nutzförderhöhe) ergibt sich aus der Summe der Saughöhe H_s und der Druckhöhe H_D :

$$H = H_s + H_D \quad (6.3)$$

Für eine genauere Beschreibung der Förderhöhe werden zusätzlich Druckverluste und Geschwindigkeitsänderungen berücksichtigt. Die vollständige Formel lautet:

$$H = h_{\text{geo}} + \frac{p_D - p_S}{g \cdot \rho} + \frac{c_D^2 - c_S^2}{2g} + H_V \quad (6.4)$$

Diese setzt sich zusammen aus der geodätischen Höhe, den statischen Drücken im Druck- und Saugbehälter, den mittleren Strömungsgeschwindigkeiten im Aus- und Eingangsquerschnitt der Pumpe und den Druckverlusten.

6.2.3. Nutzleistung

Die Nutzleistung ergibt sich aus dem Massenstrom und der spezifischen Nutzarbeit der Pumpe:

$$P_N = g \cdot \rho \cdot \dot{V} \cdot H \quad (6.5)$$

6.2.4. Pumpenwirkungsgrad

Der Pumpenwirkungsgrad berechnet sich als Verhältnis von Nutzleistung zur Kupplungsleistung:

$$\eta_P = \frac{P_N}{P_K} \quad (6.6)$$

Ist der hydraulische, volumetrische, Radreibungs- und mechanische Wirkungsgrad bekannt, so ergibt sich der gesamte Pumpenwirkungsgrad als Produkt:

$$\eta_P = \eta_h \cdot \eta_{\text{vol}} \cdot \eta_R \cdot \eta_m \quad (6.7)$$

Für Kreiselpumpen gelten typischerweise folgende Wirkungsgradbereiche:

- Hydraulischer Wirkungsgrad: $\eta_h = 0,82 - 0,95$
- Mechanischer Wirkungsgrad: $\eta_m = 0,95 - 0,98$
- Motorwirkungsgrad: $\eta_M = 0,82 - 0,95$
- Kupplungswirkungsgrad: $\eta_K = 0,82 - 0,95$

6.2.5. Kupplungsleistung

Die Kupplungsleistung ist die mechanische Leistung, die zum Antrieb der Pumpe erforderlich ist. Sie setzt sich aus allen Leistungsverlusten zusammen:

$$P_K = P_N + P_h + P_{\text{vol}} + P_R + P_m \quad (6.8)$$

Dazu gehören hydraulische und mechanische Verluste, Radseitenreibungsverluste, Wirbel- und Wandreibungsverluste, Spaltverluste an den Dichtflächen, Ventilverluste sowie Lagerreibungsverluste.

6.2.6. Anlagenkennlinie

Alle Pumpenanlagen besitzen eine Anlagenkennlinie $H_A = f(\dot{V}, d)$, die sich aus der Bernoulli-Gleichung ergibt:

$$H_A = \frac{p_D - p_S}{g \cdot \rho} + \frac{c_D^2 - c_S^2}{2g} + H_{\text{geo}} + H_V \quad (6.9)$$

Bei der Verwendung eines offenen Behälters entfällt der erste Term, da der Umgebungsdruck konstant ist. Die Gleichung vereinfacht sich dann zu:

$$H_A = \frac{c_D^2 - c_S^2}{2g} + H_{\text{geo}} + H_V \quad (6.10)$$

[Sur21]

6.3. Comet Tauchpumpe ELEGANT 12V

Die COMET ELEGANT 12V ist eine kompakte, leistungsfähige Niedervolt-Tauchpumpe für sauberes Wasser, auch geeignet für Trinkwasser. Sie ist dauerlaufeigentlich und ist für den Betrieb mit einem 12V DC-Netzteil oder Akku ausgelegt. [Com04] [Con25]

6.4. Spezifikation

6.4.1. Technische Daten:

- Betriebsspannung: 12 Volt DC
- Stromaufnahme: max. 2,2 A
- Leistungsaufnahme: 15 – 24 W
- Förderleistung: max. 10 l/min
- Förderhöhe: max. 5,5 m
- Fördermenge: max. 600 l/h
- Förderdruck: max. 0,55 bar
- Durchmesser: 38 mm
- Höhe: 104 mm
- Kabellänge: 1 m
- Anschlussfarben: Braun (+) / Blau (-)

6.4.2. Weitere Eigenschaften

- Trockenlaufsicher bis zu 2 Stunden
- Dauerlaufgeeignet mit einer Lebensdauer von ca. 450-500 Betriebsstunden
- Wasserverträglichkeit bis zu 60°, auch für Trinkwasser geeignet
- Passend für Schläuche mit 10 mm Innendurchmesser
- Optional erweiterbar mit: Rückschlagventil mit Entlüftung, Entlüftungsstutzen, Feinfilter

[Com04] [Con25]

6.5. Einfacher Code

Ein einfacher Beispielcode ist eine Wasserstandkontrolle. Es wird eine Meldung ausgegeben, wenn der Wasserstand zu niedrig ist. 6.1

Listing 6.1.: Einfacher Code zur Ansteuerung einer Pumpe

```

1000 /**
1001  * @file ansteuerungPumpe.ino
1002  * @brief Control of the COMET submersible pump ELEGANT 12V using a
1003  *        MOSFET and an Arduino Nano 33 BLE Sense.
1004  *
1005  * The pump is powered by an external 12V power supply and switched via
1006  * an N-channel MOSFET.
1007  * The control signals (start/stop) are given via a digital Arduino
1008  * output pin.
1009  */
1010
1011 /// GPIO pin on the Arduino to control the MOSFET gate
1012 const int PUMP_PIN = 2;
1013
1014 /// Optional maximum pump runtime in milliseconds (e.g., 60 seconds)
1015 const unsigned long MAX_RUNTIME_MS = 60000;
1016
1017 /// Internal variable for time tracking
1018 unsigned long pumpStartTime = 0;
1019
1020 /**
1021  * @brief Initializes the pump control pin as output and ensures the
1022  *        pump is off at startup.
1023  */
1024 void setup() {
1025     pinMode(PUMP_PIN, OUTPUT);
1026     stopPump(); // Ensure the pump is off at startup
1027 }
1028
1029 /**
1030  * @brief Main program loop with a simple start/stop test.
1031  */
1032 void loop() {
1033     // Example: Start the pump for a defined time, then stop
1034     startPump();
1035     delay(10000); // Run for 10 seconds
1036     stopPump();
1037     delay(10000); // Pause for 10 seconds
1038 }
1039
1040 /**
1041  * @brief Activates the pump by setting the MOSFET gate to HIGH.
1042  * Also stores the start time for monitoring.
1043  */
1044 void startPump() {
1045     digitalWrite(PUMP_PIN, HIGH);
1046     pumpStartTime = millis();
1047 }
1048
1049 /**
1050  * @brief Deactivates the pump by setting the MOSFET gate to LOW.
1051  */
1052 void stopPump() {
1053     digitalWrite(PUMP_PIN, LOW);
1054     pumpStartTime = 0;
1055 }
1056
1057 /**
1058  * @brief Returns whether the pump is currently active.
1059  * @return true if the pump is running, false otherwise.
1060  */
1061 bool isPumpActive() {
1062     return digitalRead(PUMP_PIN) == HIGH;
1063 }

```

../Code/Arduino/Pumpe/ansteuerungPumpe/ansteuerungPumpe.ino

6.6. Tests

6.7. Einfache Anwendung

Eine Anwendung ist die Fehlererkennung für den Fall, dass kein Wasser durch die Pumpe befördert wird. Ein Ultraschallsensor misst vor dem Start der Pumpe den Wasserstand. Nach dem Start wird der Wasserstand erneut gemessen. Ändert sich der Wasserstand nicht, gibt es eine Meldung.^{6.2}

Listing 6.2.: Einfacher Code zur Fehlererkennung

```

1000 /**
1001  * @file fehlererkennungPumpe.ino
1002  * @brief Control of the COMET pump with water level monitoring and
1003  *        delivery verification.
1004  *
1005  * Detects whether water is actually being pumped while the pump is
1006  *        running
1007  * (e.g., due to air intake, blockage, or dry running).
1008  */
1009
1010 const int PUMP_PIN = 2;
1011 const int TRIG_PIN = 3;
1012 const int ECHO_PIN = 4;
1013
1014 const int MIN_WATER_LEVEL_CM = 15;          ///< Minimum water level
1015         required to start the pump
1016 const int DELIVERY_TOLERANCE_CM = 1;        ///< Minimum level change
1017         expected during pumping
1018 const unsigned long MEASUREMENT_INTERVAL = 3000;
1019 const unsigned long DELIVERY_TEST_TIME = 3000; ///< Max time allowed for
1020         pump test
1021
1022 unsigned long lastMeasurementTime = 0;
1023 bool pumpIsRunning = false;
1024
1025 /**
1026  * @brief Initializes the GPIO pins and serial communication.
1027  */
1028 void setup() {
1029     pinMode(PUMP_PIN, OUTPUT);
1030     pinMode(TRIG_PIN, OUTPUT);
1031     pinMode(ECHO_PIN, INPUT);
1032     stopPump();
1033
1034     Serial.begin(9600);
1035     while (!Serial);
1036     Serial.println("System start with error monitoring.");
1037 }
1038
1039 /**
1040  * @brief Main loop that checks water level and verifies pump delivery.
1041  */
1042 void loop() {
1043     if (millis() - lastMeasurementTime >= MEASUREMENT_INTERVAL) {
1044         lastMeasurementTime = millis();
1045
1046         int levelBefore = measureWaterLevel();
1047         Serial.print("Water level (before): ");
1048         Serial.print(levelBefore);
1049         Serial.println(" cm");
1050
1051         if (levelBefore > MIN_WATER_LEVEL_CM) {
1052             Serial.println("WARNING: Water level too low.");
1053             stopPump();
1054             return;
1055         }
1056
1057         // Start pump and allow test run
1058         startPump();
1059         delay(DELIVERY_TEST_TIME);
1060
1061         int levelAfter = measureWaterLevel();
1062         Serial.print("Water level (after): ");
1063         Serial.print(levelAfter);
1064         Serial.println(" cm");
1065
1066         // Check for significant change in water level
1067         if (abs(levelAfter - levelBefore) < DELIVERY_TOLERANCE_CM) {
1068             Serial.println("ERROR: No water delivered! Please check the system
1069 .");
1070             stopPump();
1071         } else {
1072             Serial.println("Pump is operating correctly.");
1073         }
1074
1075         delay(5000); // Pause before next cycle
1076     }
1077 }

```

6.8. Further Readings

7. Schaltnetzteil: MW RD-50A

7.1. Einleitung

Netzteile sind elektronische Bauteile welche dazu genutzt werden, eine Eingangsspannung (oder auch Strom) in eine gewünschte Ausgangsspannung (oder Strom) umzuwandeln. Sie sind besonders wichtig für eine stabile und sichere Versorgungsspannung elektrischer Bauteile. Übergreifend lassen sich Netzteile zu den Spannungswandlern zuordnen. [Her+24]

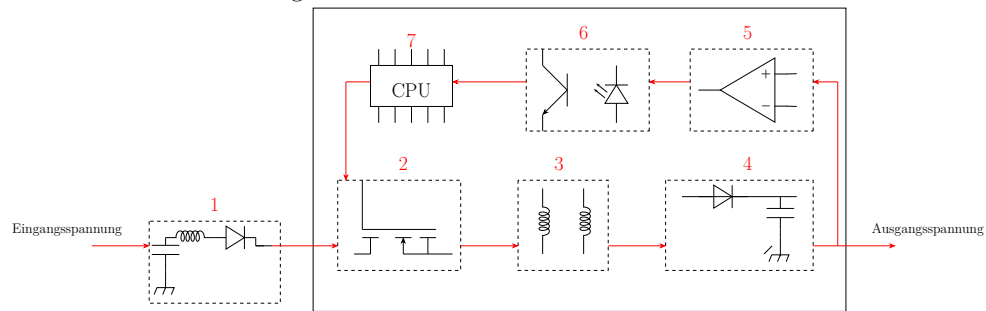
7.2. Grundlegende Informationen

Ein Netzteil wandelt in den meisten Fällen eine bekannte Wechselspannung, wie die Netzspannung im deutschen Netz (230 V / 50 Hz), in eine gewünschte Gleichspannung um. Wechselspannungen lassen sich leichter als Gleichspannungen über weite Strecken transportieren, jedoch wird für die meisten elektrischen Geräte eine Gleichspannung benötigt. Man teilt Netzteile in 2 Gruppen ein: [Her+24]

1. **Trafonetzteile:** groß und schwer, bestehen aus Transformator, Gleichrichter und Glättungskondensator
2. **Schaltnetzteile:** klein und leicht, sehr effizient, nutzen Halbleitertechnik

Für die meisten elektronischen Bauteile werden heutzutage nur noch Schaltnetzteile eingesetzt. Sie sind leichter, effizienter, günstiger und deutlich kleiner. Im Folgenden wird daher vertieft auf das Schaltnetzteil eingegangen.[HI18] Der Aufbau eines Schaltnetzteils ist wie folgt: (vgl. 7.1) Die Eingangs-Netzfrequenz von 230 V wird im Netzgleichrichter gleichgerichtet und gesiebt. Der Schalter ist ein Halbleiterbauelement (Hochfrequenzschalter wie MOSFET) welcher aus der Gleichspannung eine Rechteckwechselspannung mit hoher Schaltfrequenz umwandelt. Der Transformator ist ein klassischer Ferritkern-Trafo und dient zur Spannungsübersetzung und zur galvanischen Trennung der Stromkreise. Im Ausgangsgleichrichter wird aus der hochfrequenten Wechselspannung eine kleine Gleichspannung gleichgerichtet und gesiebt. Der Regelverstärker dient zur Verstärkung der Ausgangsspannung und führt diese in den eingebauten Regelkreis des Schaltnetzteils. Um den Regelkreis und die empfindliche Elektronik vom Ausgangsstromkreis zu trennen, wird eine Potentialtrennung (Optokoppler) verwendet. Die Steuer- und Überwachungsschaltung wird mit einem Microcontroller (CPU) realisiert. Dieser überwacht und steuert den Regelkreis und sorgt somit für eine konstante und stabile Ausgangsspannung. [HI18]

Figure 7.1.: Aufbau eines Schaltnetzteils



7.3. Schaltnetzteil: MW RD-50A

Das Schaltnetzteil in unserem Projekt ist das „MeanWell RD-50A“. Dabei handelt es sich um ein Schaltnetzteil mit variabler Eingangsspannung und zwei Ausgangsspannungen: 5 V und 12 V. Als Eingangsspannung wird die Netzspannung von 230 V und 50 Hz genutzt, das Netzteil stellt daraufhin die Versorgungsspannung für Arduino und Sensor (5 V) her und ebenfalls die Lastspannung von 12 V für die Pumpe. [Mea] [Rei25d]

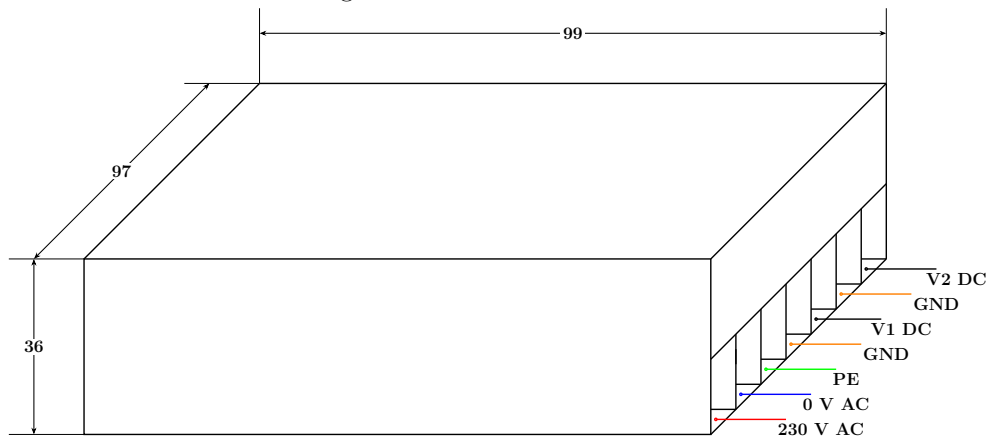
7.4. Spezifikation

Das MW RD-50A, zu sehen in 7.2, verfügt über eine variable Ausgangsspannung von 5/12 V und eine Ausgangsleistung von 54 W bei einem Ausgangsstrom von 6 A. Das Schaltnetzteil hat einen Wirkungsgrad von 79% und verfügt über einen integrierten Kurzschlusschutz, Überlastschutz und einen Überspannungsschutz. Es hat mit seiner kompakten Bauform ein Gewicht von 410 g und ist verfügt über Schraubanschlüsse. In der Tabelle 7.1 ist die Pinbelegung zu sehen. Die Pins 1-3 sind die L1 Phase vom AC Netz, Neutraleiter und PE. Sie bilden die Versorgungsspannung. Die Pins 4 und 6 sind für die Erdung der beiden Gleichstromkreise zuständig. Pin 5 sorgt für die 5 V DC Spannung und Pin 7 ist für das 12 V DC Netz zuständig.[Mea] [Rei25d]

Table 7.1.: Pinbelegung des Schaltnetzteils

1	L1 – 230 V AC
2	N – 0 V AC
3	PE
4	GND (DC)
5	V1 – 5 V DC
6	GND DC
7	V2 – 12 V (DC)

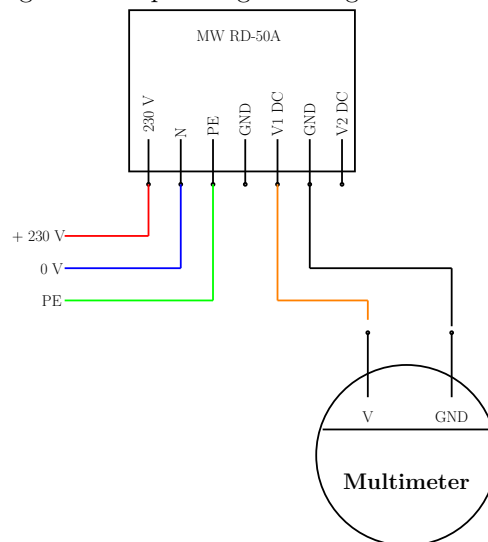
Figure 7.2.: Skizze des Netzteils



7.5. Tests

Mit einem einfachen Multimeter kann man die Ausgangsspannungen des Schaltnetzteils überprüfen. Dazu schließt man das Netzteil am AC-Netz an und verbindet die Messspitzen des Multimeters mit dem GND und jeweils dem V1 und V2 Anschluss, siehe dazu 7.3. Am Multimeter muss jedoch das Wahlrad auf DC 20 V eingestellt werden. Zu beachten ist, dass das Netzteil eine Toleranz der Ausgangsspannung von $\pm 2\%$ hat und das Multimeter auch eine interne Messabweichung hat. Dies ist zu berücksichtigen, daher empfiehlt es sich, eine Versuchsreihe von mindestens 3 Versuchen durchzuführen und das arithmetische Mittel der 3 Anzeigewerte zu nehmen.

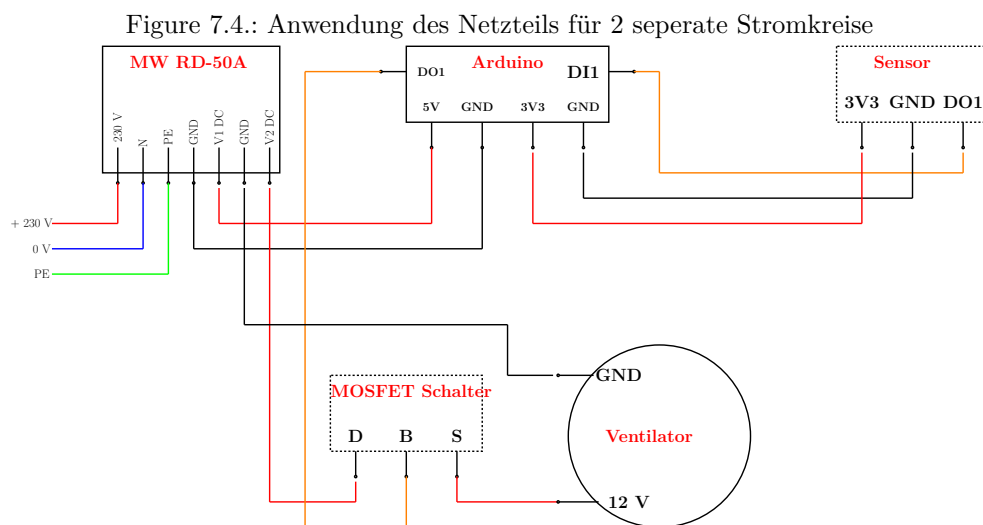
Figure 7.3.: Spannungsmessung des Netzteils



7.6. Einfache Anwendung

Als eine einfache Anwendung kann man zum Beispiel das Netzteil verwenden, um einen Steuer- und einen Laststromkreis aufzubauen. Mit dem Steuerstromkreis werden Sensoren und Mikrocontroller versorgt und der Laststromkreis ist für größere Aktoren wie ein Ventilator zuständig. In diesem Anwendungsfall (7.4) ist der Mikrocontroller

ein Arduino und der Sensor ein Luftfeuchtigkeitssensor. Sinkt die Luftfeuchtigkeit im Raum registriert dies der Sensor. Der Arduino schaltet dann den Laststromkreis mit einem MOSFET frei und der Ventilator beginnt sich zu drehen. Mit folgendem Code lässt sich das realisieren: 7.1



Listing 7.1.: Einfacher Code zum Verwenden des Netzteils

```

1000 /**
1001  * @file anwendungSchaltnetzteil.ino
1002  * @brief Controls a 12V fan via a MOSFET based on room humidity.
1003  *
1004  * If the room humidity drops below 40%, the Arduino activates DO1,
1005  * which switches a MOSFET to power a 12V fan.
1006  *
1007  * @date 2025-04-28
1008  */
1009
1010 #include <DHT.h>
1011
1012 /// Digital pin connected to the DHT sensor
1013 #define DHT_PIN 2
1014
1015 /// Digital pin to control the MOSFET (DO1)
1016 #define FAN_CONTROL_PIN 3
1017
1018 /// DHT sensor type: DHT11 or DHT22
1019 #define DHT_TYPE DHT22
1020
1021 /// Humidity threshold in percent
1022 #define HUMIDITY_THRESHOLD 40.0
1023
1024 /// DHT sensor instance
1025 DHT dht(DHT_PIN, DHT_TYPE);
1026
1027 /**
1028  * @brief Arduino setup function.
1029  * Initializes serial communication, DHT sensor, and fan control pin.
1030  */
1031 void setup() {
1032     Serial.begin(9600);
1033     dht.begin();
1034     pinMode(FAN_CONTROL_PIN, OUTPUT);
1035     digitalWrite(FAN_CONTROL_PIN, LOW); // Ensure fan is off at startup
1036 }
1037
1038 /**
1039  * @brief Arduino main loop.
1040  * Reads humidity and activates fan if below threshold.
1041  */
1042 void loop() {
1043     float humidity = dht.readHumidity();
1044
1045     // Check if sensor reading is valid
1046     if (isnan(humidity)) {
1047         Serial.println("Failed to read humidity from sensor.");
1048         return;
1049     }
1050
1051     Serial.print("Humidity: ");
1052     Serial.print(humidity);
1053     Serial.println(" %");
1054
1055     // Control fan based on humidity
1056     if (humidity < HUMIDITY_THRESHOLD) {
1057         digitalWrite(FAN_CONTROL_PIN, HIGH); ///< Turn on fan
1058         Serial.println("Fan ON (humidity below threshold)");
1059     } else {
1060         digitalWrite(FAN_CONTROL_PIN, LOW); ///< Turn off fan
1061         Serial.println("Fan OFF (humidity above threshold)");
1062     }
1063
1064     delay(2000); // Wait before next reading
1065 }

```

../Code/Arduino/Schaltnetzteil/anwendungSchaltnetzteil/anwendungSchaltnetzteil.ino

7.7. Further Readings

Tränkler, H.-R.: Sensortechnik – Handbuch für Praxis und Wissenschaft. 2. Aufl., Springer Vieweg, 2018 [TR18]

- Ein schöner Einstieg und Überblick über die Sensortechnik und ihre Anwendung heutzutage.

E. Hering et al. *Elektrotechnik und Elektronik in Maschinenbau und Mechatronik*. 5. Auflage. Springer, 2024. ISBN: 978-3-662-67538-0.[Her+24]

- Eine sehr spannende Einführung in die Elektrotechnik für Maschinenbauer. Viele grundlegende Informationen und gute Anwenderbeispiele.

Haeberle and Isele. Tabellenbuch Elektrotechnik. 28th ed. Europa Lehrmittel, 2018. isbn: 978-3-808-53490-8.[HI18]

- Ein tolles Tabellenbuch in dem viel Grundlagenwissen vermittelt wird. Zudem sind die einfache Skizzen und Erklärungen hervorzuheben.

8. Bluetooth Modul HC-05

8.1. Einleitung

Bluetooth-Module sind elektronische Komponenten, die drahtlose Kommunikation zwischen Geräten ermöglichen. Sie wandeln elektrische Signale in Funkwellen um und sorgen so für eine zuverlässige Datenübertragung über kurze Distanzen. Grundsätzlich unterscheidet man Bluetooth Classic (für kontinuierliche Verbindungen wie Audio-Streaming) und Bluetooth Low Energy (BLE) (energiesparend für Geräte mit begrenzter Akkulaufzeit). Ihre Hauptaufgabe ist es, Geräte wie Kopfhörer, Sensoren oder Smart-Home-Systeme einfach und kabellos zu vernetzen – ohne komplexe Infrastruktur [Sau22].

8.2. Grundlegende Informationen

Bluetooth-Module sind spezialisierte Hardwarekomponenten, die eine drahtlose Kommunikation zwischen Geräten über kurze Distanzen ermöglichen. Typischerweise decken sie Reichweiten von 10 bis 30 Metern ab, abhängig von Umgebungsfaktoren wie Wänden oder Störquellen. Im Kern bestehen diese Module aus einem Bluetooth-Chip, einer Antenne sowie unterstützender Elektronik wie Spannungsreglern oder Schnittstellen-ICs. Um Platz und Energie zu sparen, werden sie häufig als System-on-a-Chip (SoC) realisiert, bei dem alle notwendigen Komponenten in einem einzigen, kompakten Bauteil integriert sind. Die Technologie nutzt das 2,4-GHz-ISM-Band (2400–2483,5 MHz), ein lizenzfreies Frequenzspektrum, das auch von WLAN, Mikrowellen und anderen Funkstandards genutzt wird. Um Interferenzen zu vermeiden, setzt Bluetooth auf Frequency Hopping: Das Modul wechselt bis zu 1600-mal pro Sekunde zwischen 79 definierten Kanälen, wobei jeder Kanal eine Bandbreite von 1 MHz besitzt. Die genaue Frequenz eines Kanals k lässt sich durch die Gleichung

$$f_k = 2402 + k \text{ [MHz]} \quad (8.1)$$

berechnen, wobei k Werte von 0 bis 78 annimmt. Dieser dynamische Kanalwechsel sorgt für eine robuste Verbindung, selbst in frequenzbelasteten Umgebungen [Sau22].

8.2.1. Bluetooth Varianten

Grundsätzlich unterscheidet man zwei Bluetooth-Varianten: Bluetooth Classic und Bluetooth Low Energy (BLE).

- Bluetooth Classic ist für Anwendungen mit hohen Datenraten ausgelegt, etwa Audio-Streaming für Kopfhörer oder Lautsprecher, und erreicht bis zu 3 Mbps (Megabit pro Sekunde) durch Erweiterungen wie EDR (Enhanced Data Rate).
- BLE hingegen wurde für Geräte entwickelt, die extrem energieeffizient arbeiten müssen, wie Wearables oder Sensoren in IoT-Netzwerken. Hier liegt die maximale Datenrate bei 1 Mbps, während der Stromverbrauch im Standby-Modus oft unter 15 μA bleibt – ideal für Batteriebetrieb über Monate oder Jahre.

8.2.2. Reichweite und Sendeleistung

Die Reichweite eines Bluetooth-Moduls hängt maßgeblich von seiner Sendeleistung (TX Power) ab.

- Klasse 1 Geräte (bis 100 mW Sendeleistung) können theoretisch bis zu 100 m überbrücken.
- Klasse 2 Geräte (2,5 mW Sendeleistung) sind auf etwa 10 m begrenzt.
- Klasse 3 Geräte (1 mW Sendeleistung) sind auf etwa 1 m begrenzt.

In der Praxis reduzieren Hindernisse wie Wände oder elektronische Störquellen diese Werte jedoch deutlich [Sau22].

8.2.3. Daten Übertragung

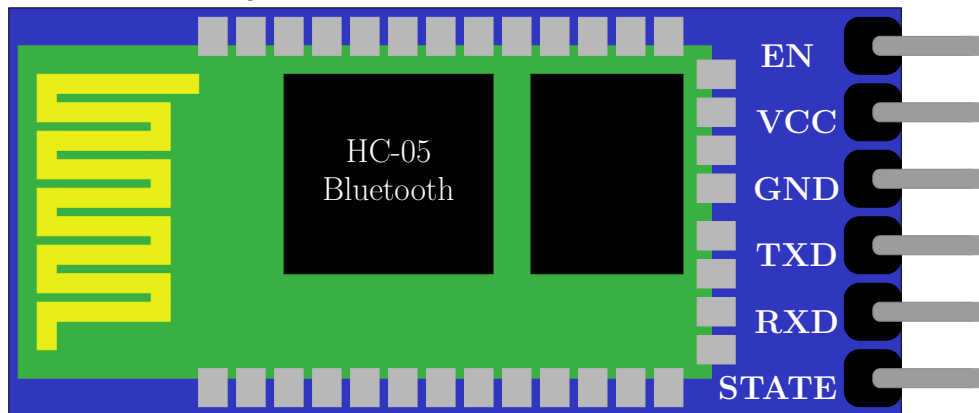
Ein kritischer Aspekt ist die Sicherheit. Bluetooth verwendet AES-128-Verschlüsselung, um Daten vor unbefugtem Zugriff zu schützen. Beim Verbindungsaufbau (Pairing) kommen zudem Protokolle wie Elliptic Curve Diffie-Hellman (ECDH) zum Einsatz, die einen sicheren Schlüsselaustausch ermöglichen und Man-in-the-Middle-Angriffe erschweren. Dank standardisierter Profile – vordefinierter Regeln für spezifische Anwendungen – lassen sich Geräte unterschiedlicher Hersteller problemlos koppeln. Beispiele sind das *A2DP-Profil* für Audio-Streaming oder das HID-Profil für Tastaturen und Mäuse [Sau22].

8.2.4. Anwendungen

Bluetooth-Module sind heute in nahezu allen Consumer-Geräten zu finden, von Smartphones über Smart-Home-Steuerungen bis hin zu industriellen Sensornetzwerken. Ihre Stärke liegt in der einfachen Integration: Über Schnittstellen wie UART oder SPI können sie direkt mit Mikrocontrollern wie Arduino oder Raspberry Pi verbunden werden, um Projekte schnell drahtlos zu machen. Dennoch stößt die Technologie an Grenzen, wenn es um hohe Reichweiten, extrem hohe Datenmengen (z. B. Video-Streaming) oder komplexe Mesh-Netzwerke geht – hier sind Alternativen wie WLAN oder Zigbee oft besser geeignet. Zusammenfassend bilden Bluetooth-Module das technische Rückgrat für unzählige kabellose Anwendungen des täglichen Lebens. Sie kombinieren Energieeffizienz mit standardisierter Kompatibilität und bleiben dabei für Nutzer weitgehend unsichtbar. Mathematische Grundlagen wie Frequenzberechnung, Verschlüsselungsalgorithmen oder Leistungsbilanzierung sorgen im Hintergrund für Zuverlässigkeit, während Anwender lediglich die simplicity der „Plug-and-Play“-Vernetzung schätzen [Sau22].

8.3. Bluetooth Modul: HC-05

Figure 8.1.: Skizze des Bluetooth Moduls HC-05



Das HC-05 ist ein weit verbreitetes Bluetooth-Modul für die drahtlose Serial-Communication (UART), zu sehen in 8.1. Es eignet sich besonders für Projekte, bei denen Mikrocontroller wie Arduino oder Raspberry Pi kabellos mit Smartphones, PCs oder anderen Geräten kommunizieren sollen. Im Folgenden werden die technischen Spezifikationen, Funktionen und typischen Einsatzgebiete des Moduls vorgestellt.

8.4. Spezifikation

Das HC-05 Bluetooth-Modul ist ein serielles Kommunikationsmodul, das auf der Bluetooth-Spezifikation V2.0+EDR basiert und für drahtlose Datenübertragung über das Serial Port Protocol (SPP) konzipiert ist. Es ermöglicht sowohl Master- als auch Slave-Betriebsmodi und wird häufig in Embedded-Systemen zur Integration von Bluetooth-Funktionalität eingesetzt [[empty citation](#)]

8.5. Bibliotheken

Um die Kommunikation zwischen Arduino und HC-05 zu ermöglichen wird die Bibliothek `SoftwareSerial.h` benötigt. Durch die Verwendung der Bibliothek kann der HC-05 über beliebige Pins kommunizieren. Die Bibliothek wird hier mit der Installation der aktuellsten Version von Arduino-IDE (Version 2.2.1) mitinstalliert. Falls die Bibliothek dennoch fehlen sollte kann sie wie folgt installiert werden:

- Bibliothek über Arduino Library List Herunterladen
- Öffnen Sie die Arduino-IDE
- Gehen Sie zu Sketch > Bibliothek einbinden > .ZIP-Bibliothek hinzufügen.
- Wählen Sie die heruntergeladene ZIP-Datei aus
- Die Bibliothek wird automatisch im Ordner `Documents/Arduino/libraries/` installiert

Optional kann man noch die Bibliothek `HC05Library` installieren mit dieser Bibliothek wird das Senden von AT-Befehlen vereinfacht.

Installation: Download:

- GitHub: <https://github.com/evankale/ArduinoHC05>
- Klicke auf Code > Download ZIP.
- In der Arduino-IDE: Sketch > Bibliothek einbinden > .ZIP-Bibliothek hinzufügen.

8.6. Einfacher Code

Um die grundlegende Funktionsweise des HC-05 Bluetooth-Moduls zu testen kann der folgende Code verwendet werden. Der Code überprüft ob das HC-05 mit dem Smartphone verbunden ist und ob die Übertragung von Daten funktioniert. Um das Smartphone mit dem HC-05 zu verbinden wird ebenfalls ein Code benötigt dieser ist falls er nicht verändert wurde: 1234 oder 0000 falls der Code geändert wurde kann dieser über ein I2C und einem entsprechenden Code ausgelesen werden. 8.1

Listing 8.1.: Code zur Überprüfung der Funktionalität

```

1000 /**
1001  * @file bluetoothBeispiel.ino
1002  * @brief Basic Bluetooth connection indicator using internal LED.
1003  *
1004  * This sketch lights up the internal LED (pin 13) when a Bluetooth
1005  * connection is active.
1006  * It also sends a confirmation message "OK!" over Bluetooth.
1007  */
1008 #include <SoftwareSerial.h> ///< Include SoftwareSerial library for
1009                               Bluetooth communication
1010
1011 ///< Create a SoftwareSerial object for Bluetooth communication (RX = pin
1012    2, TX = pin 3)
1013 SoftwareSerial bluetooth(2, 3);
1014
1015 /**
1016  * @brief Arduino setup function.
1017  *
1018  * Sets pin 13 as output and initializes Bluetooth communication.
1019  */
1020 void setup() {
1021     pinMode(13, OUTPUT);          ///< Use internal LED (pin 13) as output
1022     bluetooth.begin(9600);        ///< Start Bluetooth communication at 9600
1023                                   baud
1024 }
1025
1026 /**
1027  * @brief Arduino main loop function.
1028  *
1029  * Turns the internal LED on when data is available from the Bluetooth
1030  * module,
1031  * sends a response message, and prevents message spamming with a short
1032  * delay.
1033  */
1034 void loop() {
1035     if (bluetooth.available()) {   ///< Check if Bluetooth data is
1036         available
1037         digitalWrite(13, HIGH);    ///< Turn on internal LED
1038         bluetooth.println("OK!");  ///< Send response message
1039         delay(1000);               ///< Wait to avoid message flooding
1040     } else {
1041         digitalWrite(13, LOW);     ///< Turn off internal LED if no
1042         Bluetooth data
1043     }
1044 }

```

../Code/Arduino/Bluetooth Modul/bluetoothBeispiel/bluetoothBeispiel.ino

8.7. Tests

8.8. Einfache Anwendung

Als Beispielhafte Anwendung für den HC-05 kann das Modul über einen Arduino und einem Relais mit einer Stehlampe verschaltet werden. Zu sehen in folgendem Code: 8.2. Wenn man das System dann mit einem Smartphone verbindet kann die Stehlampe über das System mit den HC-05 ein und ausgeschaltet werden. Das System kann folgenderweise verbunden werden 8.1:

Die Stehlampe wird dann noch mit dem Relais verbunden. Dann kann auch schon der Code für die Anwendung Installiert werden.

Table 8.1.: Pin Belegung

Arduino	HC-05	Realis
RX(Pin 2)	TX	-
TX(Pin 3)	RX	-
5V (oder 3.3V)	VCC	-
GND	GND	-
Pin 7	-	IN(Steuereingang)

Listing 8.2.: Code zur Überprüfung der Funktionalität

```

1000 /**
1001  * @file bluetoothAnwendung.ino
1002  * @brief Bluetooth-controlled relay for switching a floor lamp on and
1003  *       off.
1004  *
1005  * This sketch uses a Bluetooth module connected via SoftwareSerial to
1006  * control a relay.
1007  * Commands 'E' and 'A' turn the relay ON and OFF, respectively.
1008  */
1009
1010 #include <SoftwareSerial.h> ///< Include SoftwareSerial library for
1011                               Bluetooth communication
1012
1013 #define RELAY_PIN 7 ///< Arduino pin connected to the relay module
1014
1015 ///< Create a SoftwareSerial object for Bluetooth communication (RX = pin
1016 2, TX = pin 3)
1017 SoftwareSerial bluetooth(2, 3);
1018
1019 /**
1020  * @brief Arduino setup function.
1021  *
1022  * Initializes the relay pin and starts the Bluetooth serial
1023  * communication.
1024  * Sends an initial message to the Bluetooth device to indicate
1025  * readiness.
1026  */
1027 void setup() {
1028     pinMode(RELAY_PIN, OUTPUT);          ///< Set the relay pin as output
1029     digitalWrite(RELAY_PIN, LOW);        ///< Start with the relay turned OFF
1030     bluetooth.begin(9600);                ///< Start Bluetooth communication
1031     at 9600 baud
1032     bluetooth.println("Floor lamp ready! Send 'E' to turn ON, 'A' to turn
1033 OFF.");
1034 }
1035
1036 /**
1037  * @brief Arduino main loop function.
1038  *
1039  * Continuously checks for incoming Bluetooth commands and toggles the
1040  * relay accordingly.
1041  */
1042 void loop() {
1043     if (bluetooth.available()) {
1044         char cmd = bluetooth.read(); ///< Read the incoming command
1045         character
1046
1047         if (cmd == 'E') {
1048             digitalWrite(RELAY_PIN, HIGH); ///< Turn the relay ON
1049             bluetooth.println("Floor lamp ON");
1050         }
1051         else if (cmd == 'A') {
1052             digitalWrite(RELAY_PIN, LOW);  ///< Turn the relay OFF
1053             bluetooth.println("Floor lamp OFF");
1054         }
1055     }
1056 }

```

../Code/Arduino/Bluetooth Modul/bluetoothAnwendung/bluetoothAnwendung.ino

8.9. Further Readings

9. Kippschalter: GOOBAY 10013

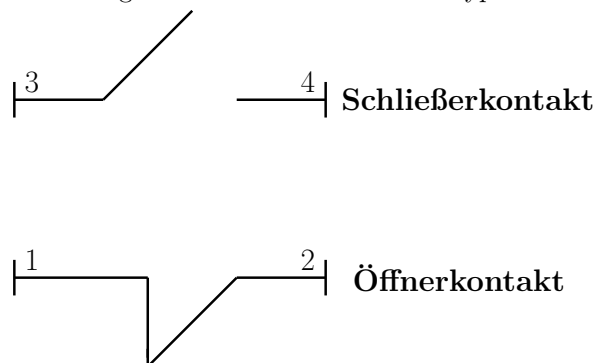
9.1. Einleitung

Kippschalter sind elektrische Bauteile, die durch ihre einfache Handhabung und Vielseitigkeit in elektronischen Systemen und Anlagen ihre Verwendung finden. Sie ermöglichen das Ein- und Ausschalten von Stromkreisen oder das Umschalten zwischen verschiedenen Betriebsmodi. [NKK25] [Ind25]

9.2. Grundlegende Informationen

Ein Kippschalter besteht typischerweise aus einem Hebel, der mechanisch zwei oder mehr elektrische Kontakte verbindet oder trennt. Beim Betätigen des Hebels bewegt sich ein beweglicher Kontakt, der entweder einen Stromkreis schließt oder öffnet. Diese mechanische Bewegung ermöglicht eine klare und stabile Schaltposition, was Kippschalter besonders zuverlässig macht. Je nach Ausführung können Kippschalter ein- oder mehrpolig sein und unterschiedliche Schaltfunktionen wie EIN/AUS, EIN/EIN oder EIN/AUS/EIN realisieren. Intern können sie entweder als Schließer oder Öffner funktionieren, zu sehen in 9.1.[NKK25] [Ind25]

Figure 9.1.: Skizze der Schaltertypen



9.3. Schalter: GOOBAY 10013

Der Goobay Miniatur-Kippschalter ist ein kompakter Schalter mit drei Pins und der Schaltfunktion EIN–AUS–EIN. Diese Konfiguration ermöglicht es, zwischen zwei Stromkreisen umzuschalten oder einen Stromkreis vollständig zu unterbrechen. Der Schalter ist ideal für Anwendungen im DIY-Bereich, Modellbau oder in elektronischen Projekten, bei denen Platzersparnis und Zuverlässigkeit gefragt sind. [NKK25] [goo23] [Rei25b]

9.4. Spezifikation

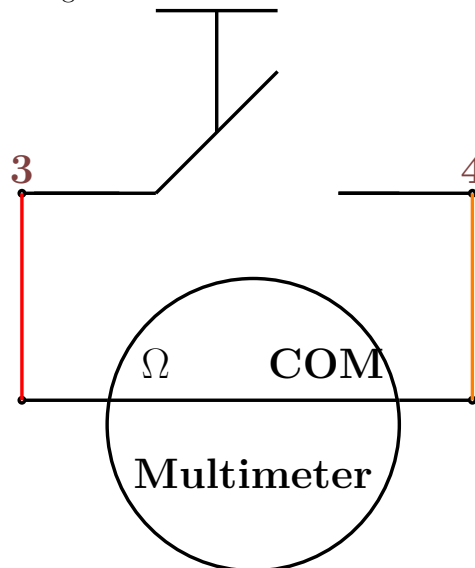
- Kippschalter mit 2 Pins (Lötösen)
- LxBxH: 8 x 5 x 7 mm

- Nennstrom: 6 A bei 125 V AC oder 3 A bei 250 V AC
- RoHS konform
- Gewicht: 3 g
- Farbe: Blau

9.5. Tests

Man kann die Funktionalität des Kippschalters leicht testen. Da er als Schließer (EIN-AUS) ausgeführt ist, kann man eine einfache Durchgangsprüfung machen, wie in 9.2 zu sehen. Man stellt das Wahlrad des Multimeters zuerst auf Widerstandsmessung. Danach schließt man das Multimeter mit den Anschlüssen für Widerstand und COM an den beiden Kontakten des Schalters an. Ist der Schalter geschlossen, zeigt das Multimeter einen hochohmigen Messwert. Schließt man jedoch den Schalter, wird der Widerstand sehr gering. Damit hat man überprüft, dass der Schalter einen Durchfluss des Stroms ermöglicht. [NKK25] [FLU25]

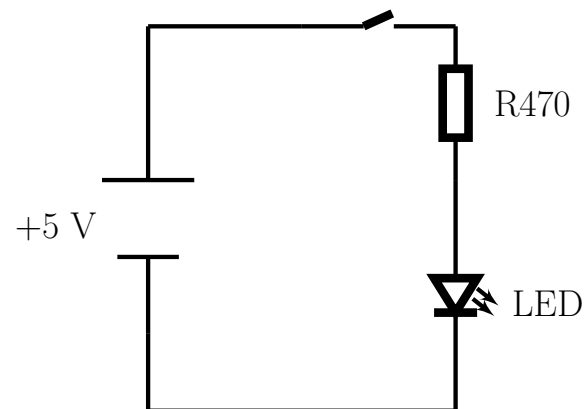
Figure 9.2.: Messen des Widerstands



9.6. Einfache Anwendung

Als einfache Anwendung kann man einen einfachen Lichtschalter betrachten. Eine Gleichstromquelle versorgt eine LED mit Vorwiderstand mit einer 5 V Spannung. Vor dem Widerstand ist ein einfacher Kippschalter als EIN-AUS eingebaut. Dieser kann den Stromkreis ein- und ausschalten. 9.3

Figure 9.3.: Einfache Anwendung des Schalters
Schalter



10. MOS-FET

10.1. Einleitung

MOSFETs (Metal-Oxide-Semiconductor Field-Effect Transistors) sind spannungsgesteuerte Halbleiterbauelemente, bei denen das Gate durch eine isolierende SiO_2 -Schicht vom Halbleiter getrennt ist. Sie ermöglichen das Schalten und Regeln von Strömen mit sehr geringem Steuerstrom und werden daher häufig in digitalen und analogen Schaltungen eingesetzt. [All24]

10.2. Grundlegende Informationen

Der Metall-Oxid-Halbleiter-Feldeffekttransistor (MOS-FET) ist ein spannungsgesteuerter Transistor, der in vier Grundtypen unterteilt wird: n-Kanal- und p-Kanal-Typen, jeweils als Anreicherungs- oder Verarmungstyp. Der am häufigsten verwendete Typ ist der n-Kanal-MOS-FET im Anreicherungstyp, der bei einer Gate-Source-Spannung von 0 V gesperrt ist (selbstsperrend). [All24]

Ein MOS-FET besteht aus zwei n-dotierten Bereichen – Source und Drain –, die auf einem p-dotierten Substrat liegen. Zwischen diesen Bereichen befindet sich ein isoliertes Gate. Wird eine positive Spannung am Gate angelegt, zieht sie Elektronen an und bildet einen leitfähigen Kanal zwischen Source und Drain. Die Stromleitung erfolgt dabei nahezu verlustfrei über diesen Kanal, wobei der Steuerstrom sehr gering ist, da das Gate durch eine Oxidschicht elektrisch isoliert ist. [All24]

Je nach Höhe der Gate-Source-Spannung (UGS) im Verhältnis zur Schwellenspannung (U_{Th}) ergeben sich drei Betriebsbereiche [All24]:

- Sperrbetrieb ($U_{GS} < U_{Th}$): Kein leitender Kanal – der Transistor sperrt.
- Abschnürbereich ($U_{GS} > U_{Th}$, aber $U_{GS} - U_{Th} < U_{DS}$): Der Kanal ist nur teilweise ausgebildet – Strom fließt eingeschränkt.
- Ohmscher Bereich ($U_{GS} - U_{Th} > U_{DS}$): Der Kanal reicht vollständig von Source bis Drain – der Transistor verhält sich wie ein variabler Widerstand.

Die Eingangswiderstände von MOS-FETs sind extrem hoch (im $G\Omega$ -Bereich), was sie ideal für Anwendungen mit geringer Steuerleistung macht. Jedoch kann bei hohen Frequenzen die Gate-Kapazität zu Nachteilen führen, da sie zu Umladeströmen führt, die wiederum die Geschwindigkeit der Schaltung begrenzen. [All24]

Die Kennlinienfelder, die das Verhalten von Drainstrom (ID) in Abhängigkeit von UGS und UDS zeigen, weisen zum Teil deutliche Unterschiede zwischen Simulation und realen Messungen auf. Diese Abweichungen entstehen u.a. durch Parameterstreuungen bei der Fertigung, insbesondere bei der Thresholdspannung und der sogenannten Kanallängenmodulation (Early-Effekt). Dies spielt in digitalen Schaltungen oft eine untergeordnete Rolle, kann jedoch in analogen Anwendungen zu Problemen führen, vor allem bei hohen Temperaturen. [All24]

Insgesamt bietet der MOS-FET viele Vorteile wie hohe Energieeffizienz und einfache Ansteuerung, erfordert aber eine sorgfältige Berücksichtigung seiner physikalischen Eigenschaften, insbesondere bei hohen Frequenzen oder großen Temperaturbereichen. [All24]

MOS-FETs lassen sich grundsätzlich in zwei Hauptgruppen unterteilen: n-Kanal- und p-Kanal-MOS-FETs. Diese Bezeichnungen beziehen sich auf die Art der Dotierung des

Halbleitermaterials. Die Dotierung erfolgt entweder mit Donatoren (n-Typ, negativ) oder Akzeptoren (p-Typ, positiv), wodurch sich die elektrische Leitfähigkeit des Siliziums erhöht. [RS 25]

10.2.1. N-Kanal-MOS-FET

- Geläufige Bezeichnungen: N-Channel-MOSFET, NMOS, NMOSFET
- Dotierung: Elektronen-Donatoren – das Fremdatom besitzt ein zusätzliches Elektron, das als Ladungsträger fungiert
- Majoritätsladungsträger: Elektronen
- Funktionsweise: Der Transistor leitet bei positiver Spannung am Gate-Anschluss
- Schaltverhalten: Das Source-Potential ist niedriger als das Drain-Potential

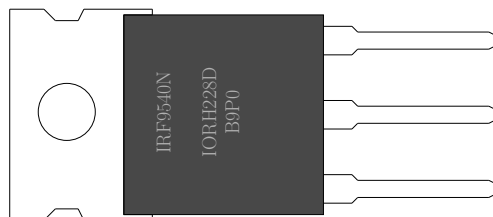
10.2.2. P-Kanal-MOS-FET

- Geläufige Bezeichnungen: P-Channel-MOSFET, PMOS, PMOSFET
- Dotierung: Elektronen-Akzeptoren – dem Fremdatom fehlt ein Elektron, es entsteht ein sogenanntes „Loch“
- Majoritätsladungsträger: Löcher (Defektelektronen)
- Funktionsweise: Der Transistor leitet bei negativer Gate-Spannung relativ zur Source
- Schaltverhalten: Das Source-Potential ist höher als das Drain-Potential

10.3. MOS-FET IRF9540N

Der COM-MOSFET ist ein p-Kanal MOSFET, der für die Steuerung von höheren Spannungen entwickelt wurde. Er eignet sich besonders für Anwendungen, bei denen eine effektive Spannung mit Hilfe von Pulsweitenmodulation (PWM) gesenkt werden soll, wie z. B. beim Dimmen von LED-Beleuchtungen.[Rei25a] 10.1

Figure 10.1.: Skizze des MOS-FETs



10.4. Spezifikation

Hauptmerkmale [Rei25a]

- Modell: IRF9540N
- Farbe der Platine: Schwarz
- Kompatibilität: Arduino, Raspberry Pi, etc.

- Besonderheiten: Der MOSFET bezieht Spannung auch über den SBC, wenn keine Source-Spannung verfügbar ist.
- Effektivspannung lässt sich mit PWM senken.
- Abmessungen: 40 mm x 22 mm x 20 mm
- Maximale Eingangsspannung: 36 V
- Maximale Eingangsstrom: 2 A

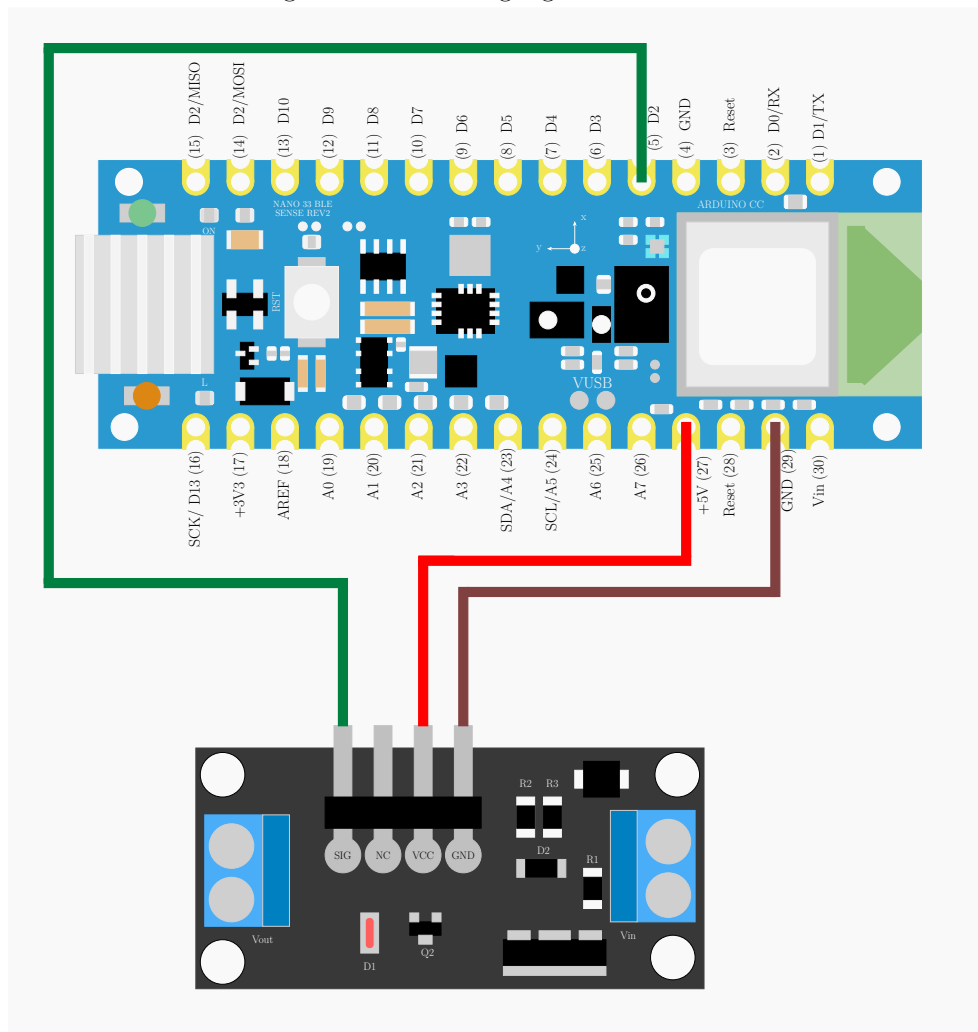
Steuerung und Anschlüsse [Rei25a]

- Steuerspannung: Mindestspannung 2 V
- Anschlüsse:
 - 2x 2 Pin Anschlussblöcke
 - 4 Pin female JST-Buchse
- Pinbelegung gemäß der Tabelle 10.1 10.2

Table 10.1.: Pinbelegung für das MOS-FET [Rei22]

Arduino	MOS-FET
GND	GND (Pin1)
5V	VCC (Pin2)
NC	NC (Pin3)
Digital Pin 2	Signal (Pin4)

Figure 10.2.: Pinbelegung des MOS-FETs



10.5. Einfacher Code

Dies ist ein einfacher Code zur Ansteuerung eines MOSFETs. Wird der Signal-Pin auf High gesetzt, aktiviert sich der Ausgang. Im folgenden Codebeispiel wird der Ausgang alle 15 Sekunden für eine Dauer von 10 Sekunden eingeschaltet. [Rei22]

Listing 10.1.: Einfacher Code zum Testen des MOS-FETs

```

1000 from time import sleep
      import RPi.GPIO as GPIO
1002
      # Set the pin numbering mode to BOARD (uses physical pin numbers)
1004      GPIO.setmode(GPIO.BOARD)
1006
      # Define the signal pin number
      Signal_Pin = 16
1008
      # Set the signal pin as an output
1010      GPIO.setup(Signal_Pin, GPIO.OUT)
1012
      try:
          # Infinite loop to toggle the output
1014          while True:
              # Turn the output ON (set pin HIGH)
1016              GPIO.output(Signal_Pin, GPIO.HIGH)
              print("Output activated")
1018              sleep(10) # Keep output on for 10 seconds
1020
              # Turn the output OFF (set pin LOW)
1022              GPIO.output(Signal_Pin, GPIO.LOW)
              print("Output deactivated")
1024              sleep(5) # Wait 5 seconds before turning it on again
      except KeyboardInterrupt:
1026          # Clean up GPIO settings when the program is interrupted (e.g., with
              Ctrl+C)
              GPIO.cleanup()

```

../Code/Arduino/MOSFET/simpleMOSFET.ino

10.6. Tests

Um einen MOS-FET zu testen, kann man eine einfache Widerstandsmessung mit einem Multimeter durchführen. Zuerst stellt man das Multimeter auf Widerstandsmessung und verbindet die Messspitzen mit den Drain- und Source-Anschlüssen des MOS-FETs. Ist keine Spannung am Gate angelegt, sollte der Widerstand zwischen Drain und Source hoch sein, was bedeutet, dass der MOS-FET „aus“ ist und keinen Strom fließen lässt. Wird jedoch eine Spannung (z. B. 9V) zwischen Gate und Source angelegt, sollte der Widerstand zwischen Drain und Source stark sinken, da der MOS-FET nun „ein“ ist und den Stromfluss ermöglicht. Diese einfache Prüfung bestätigt, ob der MOS-FET ordnungsgemäß funktioniert.

10.7. Einfache Anwendung

In dieser einfachen Schaltung wird ein MOS-FET (z.B. IRF9540N) verwendet, um eine LED ein- und auszuschalten. Der Gate-Anschluss des MOS-FETs wird direkt von einem digitalen Ausgang eines Mikrocontrollers (z.B. Arduino) angesteuert. Wenn das Gate auf einen bestimmten Pegel geschaltet wird, leitet der MOS-FET und die LED leuchtet. Wird das Gate-Signal entfernt oder umgekehrt, unterbricht der MOS-FET den Stromfluss und die LED geht aus. Diese Anwendung zeigt das grundlegende Prinzip: ein kleiner Steuerstrom am Gate kann eine größere Last im Drain-Source-Zweig ein- oder ausschalten.

Listing 10.2.: Einfache Anwendung zum Verwenden des MOS-FETs

```
1000 /**
1001  * @brief Simple MOSFET control example using an LED
1002  * Turns an LED on and off using a MOSFET and Arduino pin.
1003  */
1004 #define MOSFET_GATE_PIN 3
1005
1006 void setup() {
1007     pinMode(MOSFET_GATE_PIN, OUTPUT);
1008 }
1009
1010 void loop() {
1011     digitalWrite(MOSFET_GATE_PIN, LOW); // For P-Kanal MOSFET: Turn ON
1012     delay(1000);                        // LED ON for 1 second
1013
1014     digitalWrite(MOSFET_GATE_PIN, HIGH); // Turn OFF
1015     delay(1000);                        // LED OFF for 1 second
1016 }
```

../Code/Arduino/MOSFET/testMOSFET.ino

11. Externe LED

11.1. Einleitung

Leuchtdioden, kurz LEDs (Light Emitting Diodes), sind aus der heutigen Elektronik nicht mehr wegzudenken. Sie bieten eine energieeffiziente, langlebige und kompakte Lösung zur Lichterzeugung – sei es in Haushaltsgeräten, Automobiltechnik, Signalanlagen oder modernen Beleuchtungssystemen. Aufgrund ihrer hohen Schaltgeschwindigkeit und Robustheit sind LEDs auch in der Kommunikationstechnik und im Maschinenbau weit verbreitet. [HI18] [Her+24]

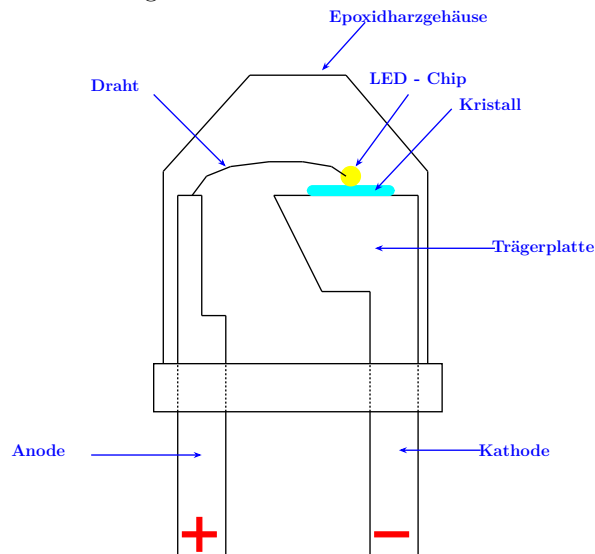
11.2. Grundlegende Informationen

Eine LED ist eine spezielle Form der Diode, die bei Durchfluss eines elektrischen Stroms Licht emittiert. Im Gegensatz zu herkömmlichen Glühlampen erzeugen LEDs Licht nicht durch Erhitzen eines Drahts, sondern durch sogenannte Elektrolumineszenz: Elektronen und „Löcher“ rekombinieren in einem Halbleitermaterial (meist Gallium- oder Siliciumverbindungen). Rekombinieren bedeutet, dass ein freies Elektron in ein Loch fällt und dessen Energieplatz im Kristallgitter besetzt. Dabei verliert das Elektron Energie – diese Energie wird nicht als Wärme, sondern als Licht (Photon) abgegeben. Die Menge an Energie, die beim Rekombinieren freigesetzt wird, entspricht der Bandlücke des Halbleitermaterials – also dem Unterschied zwischen dem Energiezustand des Elektrons und dem des Lochs. Diese Energie wird als Licht einer bestimmten Farbe (also mit bestimmter Wellenlänge) abgestrahlt. Eine große Bandlücke bedeutet kurzwelliges Licht (Blau, UV) und eine kurze Bandlücke bedeutet langwelliges Licht (Rot). [HI18] [Her+24] [Fal25]

Grundlegend ist eine LED wie in 11.1 mit den folgenden Komponenten aufgebaut:

- Anode und Kathode (positive und negative Anschlüsse)
- LED - Chip, der das Licht erzeugt
- Halbleiter-Kristall (Silicium/Germanium)
- Epoxidharzgehäuse: schützt das Bauteil und lässt das Licht ausstrahlen
- Gegebenenfalls Vorlaufwiderstand

Figure 11.1.: Aufbau einer LED



11.3. LED CML Signalleuchte, blau, ultrahell, 12 VDC

Die Signalleuchte von CML ist eine ultrahelle blaue LED mit einem Durchmesser von 8 mm. Sie eignet sich besonders für Anwendungen, bei denen eine auffällige visuelle Anzeige erforderlich ist. [Rei25c] [CML11]

11.4. Spezifikation

- Farbe: Blau
- Lichtstärke: 600 mcd
- Versorgungsspannung: 12 V AC/DC
- Einbaugröße: 8 mm Durchmesser, passend für Standard-Bohrungen in Frontplatten oder Gehäusen
- Anschluss: FASTON-Anschlüsse oder Lötanschlüsse für einfache Integration
- Besonderheiten: Integrierter Vorwiderstand, der den direkten Anschluss an 12 V ermöglicht, ohne dass zusätzliche Komponenten erforderlich sind

[Rei25c] [CML11]

11.5. Einfacher Code

Ein einfacher Code wäre die Ansteuerung einer externen LED.11.1

Listing 11.1.: Einfacher Code zur Ansteuerung einer externen LED

```
1000 /**
1001  * @file ansteuerungLED.ino
1002  * @brief Simple LED control with Arduino
1003  *
1004  * This sketch turns an LED on and off at 1-second intervals.
1005  * It demonstrates basic digital output control using the Arduino
1006  * framework.
1007  */
1008 /**
1009  * @brief Pin number where the LED is connected
1010  */
1011 const int ledPin = 13; ///< LED connected to digital pin 13
1012
1013 /**
1014  * @brief Initializes the LED pin as an output
1015  */
1016 void setup() {
1017     // Set the LED pin as output
1018     pinMode(ledPin, OUTPUT);
1019 }
1020
1021 /**
1022  * @brief Main loop: turns the LED on and off with 1-second delay
1023  */
1024 void loop() {
1025     digitalWrite(ledPin, HIGH); // Turn the LED on
1026     delay(1000);                // Wait for 1 second
1027     digitalWrite(ledPin, LOW);  // Turn the LED off
1028     delay(1000);                // Wait for 1 second
1029 }
```

../Code/Arduino/Externe LED/ansteuerungLED/ansteuerungLED.ino

11.6. Tests

Die LED ist leicht zu testen indem man sie an eine 12 V Stromversorgung anschließt, ein Vorwiderstand ist bei dieser spezifischen LED nicht nötig, da dieser bereits integriert ist.

11.7. Einfache Anwendung

Als einfache Anwendung eignet sich eine Blinkfrequenz-Schaltung. Dieser Code 11.2 lässt eine LED drei mal mit 1 Hz blinken, dann fünf mal bei 2 Hz und dann zehn mal mit 5 Hz.

Listing 11.2.: Einfacher Code zum Blinken einer externen LED

```

1000 /**
1001  * @file blinkenLED.ino
1002  * @brief LED blinking with multiple frequencies
1003  *
1004  * This sketch blinks an LED at different frequencies sequentially.
1005  * It demonstrates control of delay timing to create various blink
1006  * patterns.
1007  */
1008 /**
1009  * @brief Pin number where the LED is connected
1010  */
1011 const int ledPin = 13; ///< LED connected to digital pin 13
1012
1013 /**
1014  * @brief Initializes the LED pin as an output
1015  */
1016 void setup() {
1017     // Set the LED pin as output
1018     pinMode(ledPin, OUTPUT);
1019 }
1020
1021 /**
1022  * @brief Blinks the LED with the specified on/off delay and repetitions
1023  * @param delayTime Delay time in milliseconds between on and off states
1024  * @param repetitions Number of times the LED should blink
1025  */
1026 void blinkPattern(int delayTime, int repetitions) {
1027     for (int i = 0; i < repetitions; i++) {
1028         digitalWrite(ledPin, HIGH); // Turn the LED on
1029         delay(delayTime);           // Wait
1030         digitalWrite(ledPin, LOW);  // Turn the LED off
1031         delay(delayTime);           // Wait
1032     }
1033 }
1034
1035 /**
1036  * @brief Main loop: executes multiple blink patterns in sequence
1037  */
1038 void loop() {
1039     blinkPattern(1000, 3); // Blink slowly (1 Hz) three times
1040     blinkPattern(500, 5);  // Blink at medium speed (2 Hz) five times
1041     blinkPattern(200, 10); // Blink fast (5 Hz) ten times
1042     delay(2000);           // Wait before repeating the sequence
1043 }

```

../Code/Arduino/Externe LED/blinkenLED/blinkenLED.ino

11.8. Further Readings

Haeberle and Isele. Tabellenbuch Elektrotechnik. 28th ed. Europa Lehrmittel, 2018. isbn: 978-3-808-53490-8.[HI18]

- Ein tolles Tabellenbuch in dem viel Grundlagenwissen vermittelt wird. Zudem sind die einfache Skizzen und Erklärungen hervorzuheben.

Falcon Illumination. Wie funktioniert eine LED? [pdf]. 2025. URL: <https://www.falconillumination.de/de/wissenswertes/wie-funktioniert-eine-led.html> [Fal25]

- Sehr spannende Website mit einfachen Erklärungen rund um das Thema Beleuchtungstechnik.

12. Materialliste Magic Faucet Fountain

Anzahl	Bezeichnung	Link	Preis
1	4DUNIO Wireless Modul HC-05 Bluetooth (4-Pin)	www.reichelt.de - HC-05 Bluetooth-Modul	23,30 €
			
1	Schaltnetzteil 50 W (5 V / 12 V, 6 A) geschlossen	www.reichelt.de - Schaltnetzteil 5/12V	17,10 €
			
1	COMET Niedervolt Tauchpumpe 12 V, 600 l/h, 5,5 m	www.conrad.de - COMET Tauchpumpe	14,99 €
			
1	Blaue LED-Signalleuchte 12 V, 8 mm, 600 mcd	www.reichelt.de - blaue LED-Signalleuchte	6,75 €
			
1	Grüne LED-Signalleuchte 12 V, 8 mm, 1300 mcd	www.reichelt.de - grüne LED-Signalleuchte	6,75 €
			
3	N-Kanal MOSFET IRF9540N	www.reichelt.de - IRF9540N MOSFET	3,60 €
			
	(3 × 1,20 €)		
1	Miniatur-Kippschalter EIN/AUS 3 A, 125 V	www.reichelt.de - Kippschalter	0,60 €



Gesamtpreis: 85,09 €

Stand: 07.05.2025

13. Projekt: Magic Faucet Fountain

13.1. Beschreibung

Das Projekt Magic Faucet Fountain beschäftigt sich mit der Umsetzung eines Brunnens, der die optische Täuschung eines schwebenden Wasserhahn erzeugt. Dabei scheint es, als würde Wasser kontinuierlich aus einem frei schwebenden Hahn fließen - ganz ohne sichtbare Verbindung zur Wasserquelle.

Dieser Effekt wird durch ein transparentes Acrylrohr realisiert, das sowohl den Wasserfluss leitet als auch die tragende Struktur für den Hahn darstellt. Da das Rohr hinter dem gleichmäßig fließenden Wasser optisch kaum wahrnehmbar ist, entsteht der Eindruck eines schwebenden Hahns.

Der Wasserstrom wird über eine kleine elektrische Pumpe erzeugt, die über ein MOS-FET angesteuert wird. Der MOS-FET fungiert als elektronischer Schalter und wird von einem digitalen Ausgang des Arduino-Boards gesteuert. Der Arduino ist über Bluetooth mit einer mobilen App verbunden. Über die App kann der Benutzer den Brunnen bequem ein- und ausschalten.

Ein weiterer Bestandteil des Systems ist ein Ultraschallsensor, der den Wasserstand im Vorratsbehälter misst. Der aktuelle Füllstand wird ebenfalls in der App angezeigt, wodurch der Benutzer rechtzeitig erkennen kann, wann Wasser nachgefüllt werden muss.

13.2. Aufbau

Die mechanische Struktur des Magic Faucet Fountain basiert auf einem dekorativen Blumentopf, der gleichzeitig als Wasserreservoir und Trägerelement für den sichtbaren Aufbau dient. Der Blumentopf wird durch einen funktional gestalteten Deckel verschlossen, der sowohl mechanische als auch gestalterische Funktionen erfüllt.

Zentral im Deckel befindet sich eine Durchgangsbohrung, durch die ein transparentes Acrylrohr senkrecht nach oben geführt wird. Am oberen Ende des Rohrs ist ein dekorativer Wasserhahn angebracht. Direkt unterhalb dieses Hahns befinden sich feine Austrittsbohrungen, durch die Wasser kontrolliert herausfließt. Aufgrund der Transparenz des Rohrs und des gleichmäßigen Wasserstroms entsteht die Illusion eines frei schwebenden, endlos fließenden Wasserhahns.

Seitlich im Deckel befinden sich zusätzliche Rücklaufbohrungen, über die das Wasser zurück in den Blumentopf gelangt. Eine im Inneren platzierte Wasserpumpe sorgt für den geschlossenen Wasserkreislauf, indem sie das Wasser wieder nach oben in das Acrylrohr befördert.

An der Unterseite des Deckels ist eine Gewindeaufnahme integriert, an der ein senkrecht montiertes Führungsrohr angebracht ist. In diesem Rohr befindet sich ein frei beweglicher Schwimmer, der den Wasserstand abbildet. Das Rohr ist unten mit einem fest montierten Deckel verschlossen, in den ein Gitter direkt eingearbeitet ist. Dieses verhindert das Herausfallen des Schwimmers, ermöglicht aber den Wassereintritt. Über der Rohröffnung ist ein Ultraschallsensor montiert, der den Abstand zum Schwimmer misst und so den Wasserstand erkennt.

Die elektronischen und steuerungstechnischen Komponenten wie Arduino, MOS-FET, Spannungsversorgung, Bluetooth-Modul sowie die Status-LEDs und der Kippschalter befinden sich in einem separaten Gehäuse, das als elektronischer Kasten (E-Kasten) ausgeführt ist. Dieser Kasten schützt die Elektronik vor Feuchtigkeit und erlaubt

einen strukturierten, servicefreundlichen Aufbau. Die Leitungen zur Pumpe, zum Ultraschallsensor und zu den LEDs werden durch entsprechende Kabeldurchführungen zwischen dem E-Kasten und der Hauptbaugruppe verbunden.

Zur optischen Aufwertung kann die Oberseite des Deckels mit dekorativen Steinen oder anderen Gestaltungselementen versehen werden, wodurch die technische Funktion harmonisch in ein ästhetisches Gesamtbild eingebettet wird.

13.2.1. Abmaß

Physische Abmessungen des Aufbaus

«««< HEAD Platzbedarf und Gestaltung des Systems (z.B. Kompaktheit, Gehäuse)

===== »»»> fd2eb29dc083a54fde54f8c23761d66ce1e161e2

Gewicht und andere relevante Maße

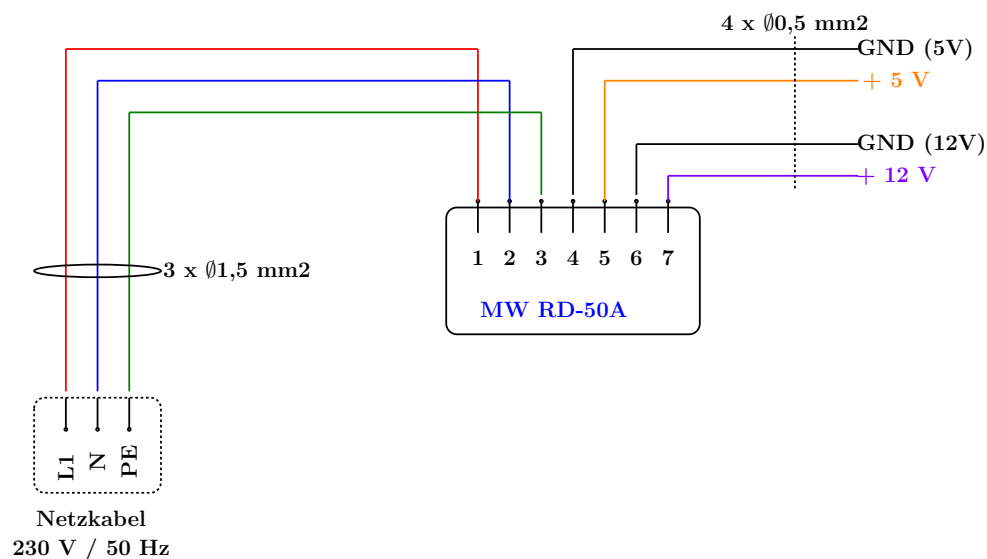
13.3. Hardware-Schnittstellen

«««< HEAD Erklärung der einzelnen Schnittstellen

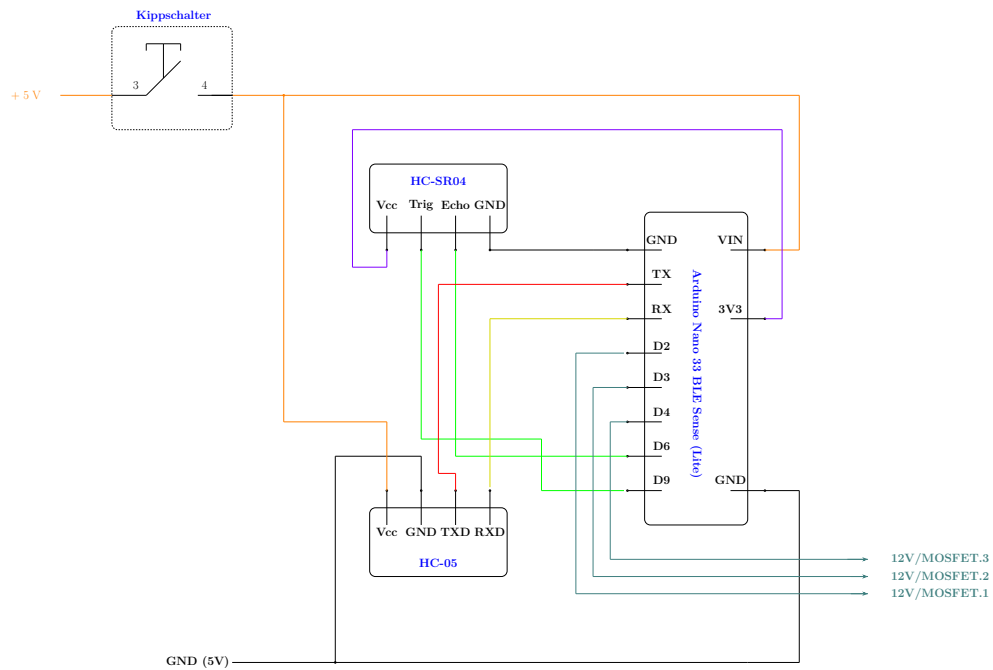
Power-Management (Stromversorgung und -verbrauch)

13.3.1. Schaltpläne

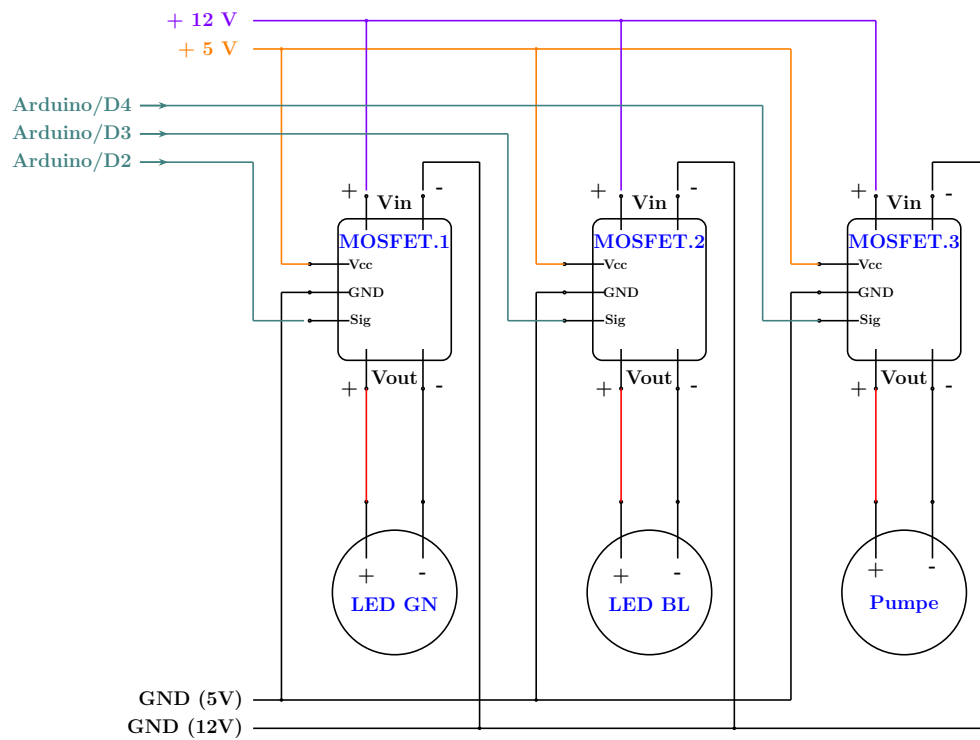
Versorgungsspannungen 5 V und 12 V



5 V Netz mit Arduino und BT-Modul

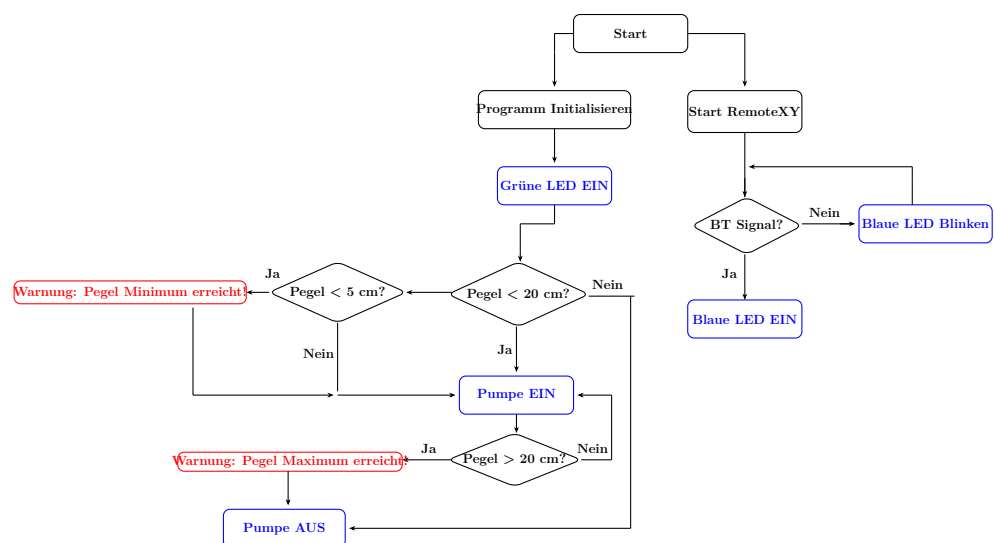


5/12 V Netz mit Aktoren



13.4. Software-Schnittstellen

13.4.1. Ablaufdiagramm der Anwendung



13.4.2. Code der Anwendung

```

1000 /**
1001  * @file ablaufApplikation.ino
1002  * @brief Automated flowerpot with Bluetooth status, pump control, and
1003  *        water level measurement.
1004  */
1005
1006 #include <SoftwareSerial.h>
1007 #include <RemoteXY.h>
1008
1009 // RemoteXY Bluetooth (HC-05) via SoftwareSerial
1010 #define REMOTEXY_SERIAL_RX 10
1011 #define REMOTEXY_SERIAL_TX 11
1012 #define REMOTEXY_SERIAL_SPEED 9600
1013
1014 #pragma pack(push, 1)
1015 uint8_t RemoteXY_CONF[] =
1016 { 255,1,0,0,0,60,0,13,13,1,
1017   2,0,7,7,18,8,2,26,31,80,
1018   117,109,112,101,0,1,0,30,7,
1019   40,10,2,31,65,110,0,79,117,
1020   115,0 };
1021
1022 struct {
1023     uint8_t pumpSwitch;          ///< 0 = Off, 1 = On
1024     char statusText[128];       ///< Text display for warnings
1025     uint8_t connect_flag;       ///< Bluetooth connection status
1026 } RemoteXY;
1027 #pragma pack(pop)
1028
1029 // Pin definitions
1030 const int PIN_LED_GREEN = 2;
1031 const int PIN_LED_BLUE = 3;
1032 const int PIN_PUMP = 4;
1033 const int PIN_TRIGGER = 9;
1034 const int PIN_ECHO = 6;
1035
1036 const int MAX_WATER_LEVEL = 20;
1037 const int MIN_WATER_LEVEL = 5;
1038
1039 unsigned long lastBlinkTime = 0;
1040 bool blueLedState = false;
1041
1042 /**
1043  * @brief Initializes pins and RemoteXY
1044  */
1045 void setup() {
1046     pinMode(PIN_LED_GREEN, OUTPUT);
1047     pinMode(PIN_LED_BLUE, OUTPUT);
1048     pinMode(PIN_PUMP, OUTPUT);
1049     pinMode(PIN_TRIGGER, OUTPUT);
1050     pinMode(PIN_ECHO, INPUT);
1051
1052     digitalWrite(PIN_LED_GREEN, HIGH); // Green LED always on
1053
1054     RemoteXY_Init();
1055 }
1056
1057 /**
1058  * @brief Main loop: water level measurement, display update, Bluetooth
1059  *        status, and pump control
1060  */
1061 void loop() {
1062     RemoteXY_Handler();
1063     updateBluetoothStatusLED();
1064
1065     int distance = measureWaterLevel();
1066
1067     // Basic warning message
1068     if (distance > MAX_WATER_LEVEL) {
1069         strcpy(RemoteXY.statusText, "WARNING: No water left!");
1070     }
1071     else if (distance < MIN_WATER_LEVEL) {
1072         strcpy(RemoteXY.statusText, "WARNING: Low level!");
1073         if (RemoteXY.pumpSwitch) {
1074             strcat(RemoteXY.statusText, " - Pump deactivated!");
1075         }
1076     }
1077 }

```

13.5. Installation

Die Installation des Magic Faucet Fountain gliedert sich in zwei Hauptbereiche: den mechanischen Aufbau der Baugruppe und die elektrische Inbetriebnahme der Steuerungskomponenten. Im Folgenden werden beide Teile Schritt für Schritt erläutert.

13.5.1. Mechanischer Aufbau

- Der Blumentopf wird als zentrales Bauteil aufgestellt und mit einem speziell angefertigten Deckel verschlossen. Dieser beinhaltet:
 - Eine zentrale Öffnung zur Durchführung des Acrylrohrs, das den Wasserfluss führt.
 - Mehrere Rücklaufbohrungen, durch die Wasser in den Topf zurückfließt.
 - Eine Gewindeaufnahme auf der Unterseite zur Befestigung des Führungsrohrs für den Schwimmer.
- Das transparente Acrylrohr wird vertikal durch den Deckel geführt und oben mit einem dekorativen Wasserhahn verbunden. Kleine Bohrungen im Rohr direkt unter dem Hahn ermöglichen den sichtbaren Wasseraustritt.
- Das Führungsrohr wird senkrecht unter dem Deckel montiert. Im Inneren befindet sich ein Schwimmer, der sich mit dem Wasserstand bewegt.
- Der untere Abschluss des Führungsrohrs besteht aus einem fest verschlossenen Deckel mit integriertem Gitter, das den Schwimmer sichert und den Wassereintritt erlaubt.
- Der Ultraschallsensor wird oberhalb des Führungsrohrs an der Aufnahme befestigt und misst den Abstand zum Schwimmer zur Bestimmung des Füllstands.
- Auf dem Deckel können zur Gestaltung Dekosteine oder andere Elemente platziert werden.

13.5.2. Elektrische Installation

Alle elektronischen Komponenten sind im separaten E-Kasten untergebracht, der außerhalb des Blumentopfs montiert wird und Feuchtigkeitsschutz sowie einfache Wartung ermöglicht.

Folgende Komponenten sind dort integriert:

- Arduino-Mikrocontroller, als zentrale Steuereinheit.
- MOS-FET-Schaltkreis zur Ansteuerung der Tauchpumpe, die das Wasser durch das Acrylrohr fördert.
- Ultraschallsensor, über digitale Pins mit dem Arduino verbunden.
- Bluetooth-Modul, über serielle Schnittstelle angebunden.
- Zwei Status-LEDs:
 - Grüne LED: zeigt den Betriebszustand an – leuchtet, wenn der Kippschalter eingeschaltet ist.
 - Blaue LED: zeigt den Bluetooth-Zustand – blinkt bei Verbindungssuche, leuchtet bei bestehender Verbindung.
- Kippschalter als Hauptschalter für das gesamte System.
- Stromversorgung über externes Netzteil, angeschlossen an den E-Kasten.

- Alle Verbindungen zur Pumpe, zu den Sensoren und zu den LEDs sind über robuste Steckverbindungen aus dem E-Kasten herausgeführt.

13.5.3. Software und App-Verbindung

- Nach dem Einschalten über den Kippschalter (→ grüne LED leuchtet), startet der Arduino.
- Die Bluetooth-Verbindung wird automatisch aktiviert (→ blaue LED blinkt).
- Über eine passende Smartphone-App kann:
 - Die Pumpe ein- und ausgeschaltet werden.
 - Der aktuelle Wasserstand basierend auf den Sensordaten angezeigt werden.
 - Der Bluetooth-Status überwacht werden.

13.5.4. Software und App-Verbindung

1. Wasser einfüllen: Der Topf wird so weit befüllt, dass die Pumpe vollständig untergetaucht ist.
2. System einschalten: Kippschalter betätigen → grüne LED zeigt Betriebsbereitschaft.
3. Bluetooth koppeln: Verbindung mit der App herstellen → blaue LED bleibt dauerhaft an.
4. Funktionstest: Pumpe über App starten, Wasserfluss beobachten.

13.6. Konfiguration

Die Konfiguration des Magic Faucet Fountain umfasst die Inbetriebnahme und Abstimmung der elektronischen Komponenten sowie die Kopplung mit der mobilen App. Ziel ist es, die Steuerung des Brunnens per Bluetooth zuverlässig zu ermöglichen und eine präzise Füllstandsanzeige über den Ultraschallsensor bereitzustellen.

1. Elektrische Inbetriebnahme
 - Vor dem Einschalten müssen alle elektrischen Verbindungen überprüft werden: Die Pumpe, der Ultraschallsensor, die Status-LEDs, der MOS-FET, der Kippschalter und das Bluetooth-Modul müssen korrekt mit dem Arduino verbunden sein.
 - Die Stromversorgung wird über den Kippschalter aktiviert. Beim Einschalten leuchtet die grüne Betriebs-LED zur Bestätigung.
2. Bluetooth-Konfiguration
 - Nach dem Einschalten beginnt die blaue LED zu blinken, was signalisiert, dass das Bluetooth-Modul aktiv ist und auf eine Verbindung wartet.
 - Über die App wird das Bluetooth-Modul des Brunnens gesucht und gekoppelt. Nach erfolgreicher Verbindung wechselt die blaue LED von Blinken auf Dauerlicht.
 - Die App ermöglicht das manuelle Ein- und Ausschalten der Pumpe sowie die Anzeige des aktuellen Wasserstands.
3. Füllstanderfassung

- Der Ultraschallsensor misst kontinuierlich den Abstand zum Schwimmer im Führungsrohr. Die Werte werden vom Arduino verarbeitet und über Bluetooth an die App gesendet.
- In der App wird der Wasserstand durch einen Ladebalken visualisiert und zusätzlich als Zahlenwert in Millilitern angegeben.

4. Software-Einstellungen

- Die Arduino-Firmware enthält die Steuerlogik für:
 - Die Pumpensteuerung (an/aus via MOS-FET).
 - Die Auswertung der Ultraschallsensordaten.
 - Die Ansteuerung der Status-LEDs.

5. Systemtest

- Nach der vollständigen Konfiguration sollte ein Testlauf durchgeführt werden, bei dem:
 - Die Bluetooth-Verbindung getestet wird.
 - Der Wasserstand korrekt in der App angezeigt wird.
 - Die Pumpe zuverlässig über die App geschaltet werden kann.
 - Die LEDs den korrekten Verbindungs- und Betriebsstatus anzeigen.
- Über die App wird das Bluetooth-Modul des Brunnens gesucht und gekoppelt. Nach erfolgreicher Verbindung wechselt die blaue LED von Blinken auf Dauerlicht.
- Die App ermöglicht das manuelle Ein- und Ausschalten der Pumpe sowie die Anzeige des aktuellen Wasserstands.

13.7. Einschränkungen

Identifikation von Problemen oder Einschränkungen (z.B. Schwierigkeiten bei der Hardwareintegration oder Softwarebugs)

14. Schlussfolgerungen

14.1. Zusammenfassung der Projektergebnisse

14.2. Reflexion über die Zielerreichung

**14.3. Mögliche Weiterentwicklungen und zukünftige
Anwendungen**

Part III.

Anhang

Literaturverzeichnis

- [AG24] A. AG. *NewPing – Library Documentation*. 2024. URL: <https://docs.arduino.cc/libraries/newping/>.
- [All24] M. Alles. *Einführung in die elektronische Schaltungstechnik*. 1st ed. Springer, 2024. ISBN: 978-3-662-69278-3.
- [Ard21] Arduino, ed. *Arduino Nano 33 BLE Sense Rev2 with headers*. 2021. URL: <https://store.arduino.cc/products/nano-33-ble-sense-rev2-with-headers>.
- [Ard21] Arduino, ed. *Check out Arduino Docs!* 2021. URL: <https://www.arduino.cc/en/Guide>.
- [Ard25] Arduino AG. *Arduino IDE 2.0*. [pdf]. 2025. URL: <https://docs.arduino.cc/software/ide/#ide-v2>.
- [CML11] CML. *Handbuch - CML LED-Signalleuchte*. [pdf]. 2011.
- [Com04] Comet. *Handbuch - Comet Tauchpumpe 12 V Elegant*. [pdf]. 2004.
- [Con25] Conrad. *Niedervolt Tauchpumpe Comet 12 V Elegant*. 2025. URL: <https://www.conrad.de/de/p/comet-1300-01-00-niedervolt-tauchpumpe-600-l-h-5-5-m-1435461.html?refresh=true#productDownloads>.
- [FAD18] M. Fezari and A. Al Dahoud. “Integrated Development Environment “IDE” For Arduino”. In: (Oct. 2018).
- [FD22] P. Filipi and Dozie. *A Difference between Arduino Nano 33 BLE Sense vs. Arduino Nano 33 BLE Sense Lite*. [pdf]. 2022. URL: <https://forum.arduino.cc/t/a-difference-between-a-n-33-ble-sense-vs-sense-lite/1030305>.
- [FLU25] FLUKE. *Anleitung zum Messen von Widerstand*. 2025. URL: <https://www.fluke.com/de-de/mehr-erfahren/blog/digitalmultimeter/anleitung-zum-messen-von-widerstand>.
- [Fal25] Falcon Illumination. *Wie funktioniert eine LED?* [pdf]. 2025. URL: <https://www.falconillumination.de/de/wissenswertes/wie-funktioniert-eine-led.html> (visited on 05/03/2025).
- [GG20] J. F. Gülich and J. F. Gülich. “Bauarten und Leistungsdaten”. In: *Kreiselpumpen: Handbuch für Entwicklung, Anlagenplanung und Betrieb* (2020), pp. 45–82.
- [HEG21] E. Hering, J. Endres, and J. Gutekunst. *Elektronik für Ingenieure und Naturwissenschaftler*. 8. [pdf]. Springer Vieweg, 2021. ISBN: 978-3-662-62697-9. DOI: 10.1007/978-3-662-62698-6.
- [HI18] Haeberle and Isele. *Tabellenbuch Elektrotechnik*. 28th ed. Europa Lehrmittel, 2018. ISBN: 978-3-808-53490-8.
- [HLG19] B. Heinrich, P. Linke, and M. Glöckler. *Grundlagen Automatisierung: Erfassen-Steuern-Regeln*. Springer-Verlag, 2019.
- [HS09] S. Hesse and G. Schnell. *Sensoren für die Prozess-und Fabrikautomation*. Vol. 5. Springer, 2009.

- [HS+12] E. Hering, G. Schönfelder, et al. *Sensoren in Wissenschaft und Technik*. Springer, 2012.
- [HS23] E. Hering and G. Schönfelder. *Sensoren in Wissenschaft und Technik*. 3. [pdf]. Springer Vieweg, 2023. ISBN: 978-3-658-39490-5. DOI: 10.1007/978-3-658-39491-2_978-3-662-62697-9.
- [Her+24] E. Hering et al. *Elektrotechnik und Elektronik in Maschinenbau und Mechatronik*. 5th ed. Springer, 2024. ISBN: 978-3-662-67538-0.
- [Ind25] Indicatorlight. *So verdrahten Sie Kippschalter*. 2025. URL: <https://de.indicatorlight.com/faq/toggle-button-switches/>.
- [MM17] G. Müller and M. Möser. *Ultraschall in Medizin und Technik*. Springer, 2017.
- [Mea] *Handbuch MW RD-50A Series*. [pdf]. 2014.
- [NKK25] NKK Switches. *Schaltergrundlagen*. 2025. URL: https://www.nkkswitches.de/learning/fundamentals/series_01.html.
- [RS 25] RS Components, ed. *MOSFET Leitfaden*. [pdf]. 2025. URL: <https://de.rs-online.com/web/content/discovery-portal/produktanbieter/mosfet-leitfaden>.
- [Raj19] A. Raj. *Arduino Nano 33 BLE Sense Review - What's New and How to Get Started?* 2019. URL: <https://circuitdigest.com/microcontroller-projects/arduino-nano-33-ble-sense-board-review-and-getting-started-guide>.
- [Rei22] Reichelt Online. *Anleitung - COM-MOSFET Modul*. [pdf]. 2022. URL: https://cdn-reichelt.de/documents/datenblatt/A300/COM-MOSFET_ANLEITUNG_2022-07-20.pdf (visited on 05/06/2025).
- [Rei25a] Reichelt Online, ed. *COM - MOS-FET*. [pdf]. 2025. URL: https://www.reichelt.de/de/de/shop/produkt/entwicklerboards_-_mosfet_irf9540n-316199 (visited on 05/04/2025).
- [Rei25b] Reichelt Online, ed. *Kippschalter - GOOBAY 10013*. [pdf]. 2025. URL: https://www.reichelt.de/de/de/shop/produkt/miniatur-kippschalter_ein-aus_3_a_125_v-359360 (visited on 05/02/2025).
- [Rei25c] Reichelt Online, ed. *LED - CML*. [pdf]. 2025. URL: https://www.reichelt.de/de/de/shop/produkt/led-signalleuchte_blaue_ultrahell_12_vac_vdc_8_mm_600_mcd-271924 (visited on 05/04/2025).
- [Rei25d] Reichelt Online, ed. *Schaltnetzteil - MW HRP-75*. [pdf]. 2025. URL: https://www.reichelt.de/de/de/shop/produkt/schaltnetzteil_geschlossen_50_w_5_12_v_6_a-137098#closemodal (visited on 05/02/2025).
- [Sau22] M. Sauter, ed. *Grundkurs Mobile Kommunikationssysteme*. 8th ed. Köln, Deutschland: Springer Vieweg, 2022. ISBN: 978-3-658-36963-7.
- [Sch13] H. Schulz. *Die Pumpen: Arbeitsweise, Berechnung, Konstruktion*. Springer-Verlag, 2013.
- [Sim] Joy-IT® Ultrasonic Distance Sensor. SEN-US01. [pdf]. Simac Electronics GmbH. Aug. 2017.
- [Sur21] D. Surek. *Pumpen*. Springer, 2021, pp. 1039–1053.
- [TM24] P. A. Tipler and G. Mosca. *Tipler Physik*. 9th ed. Springer Spektrum Berlin, May 2024, pp. 423–427. ISBN: 978-3-662-67936-4.

- [TR14] H.-R. Tränkler and L. Reindl, eds. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2nd ed. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, pp. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: 10.1007/978-3-642-29942-1.
- [TR18] H.-R. Tränkler and L. M. Reindl, eds. *Sensortechnik Handbuch für Praxis und Wissenschaft*. 2nd ed. Berlin, Heidelberg: Springer Vieweg, 2018. ISBN: 978-3-642-29942-1.
- [Wil22] W. M. Willems. *Lehrbuch der Bauphysik*. 9th ed. Springer Vieweg Wiesbaden, Nov. 2022, pp. 567–568. ISBN: 978-3-658-34093-3.
- [goo23] goobay. *Handbuch - goobay 10013*. [\[pdf\]](#). 2023.

Index

File

- .ino, 24
- Arduino, 24
- blink.ino, 37, 38
- blnik.ino, 33
- Fade, 38
- File/Examples/01.Basics, 38
- libraries, 27–29
- MySketch, 24
- MySketch.ino, 24
- Nano33BLESenseLE, 27
- Nano33BLESenseLED, 28, 29

Inertial Measurement Unit

- see* IMU, 11, 44

General Purpose Input Output

- see* GPIO, 11, 54

GPIO, 11, 54, 55

I²C, 11, 54, 55

IDE, 11, 38

IMU, 11, 44

Integrated Development Environment

- see* IDE, 11, 38

Inter-Integrated Circuit

- see* I²C, 11, 54

SCL, 11, 54, 55

SDA, 11, 54, 55

Serial Clock

- see* SCL, 11, 54

Serial Data

- see* SDA, 11, 54

UART, 11, 54, 55

Universal Asynchronous Receiver Transmitter

- see* UART, 11, 54

VCC, 11, 54, 55

Voltage at the Common Collector

- see* VCC, 11, 54